Date Published: April 01, 2026

Visit the Zebra Product Support page for the most up-to-date version of this documentation.

Mblob functions

Synopsis

The functions prefixed with Mblob make up the Blob Analysis module. The Blob Analysis module allows you to identify connected regions of pixels (blobs) within an image, and then calculate selected features of these blobs. The Blob Analysis module can measure a wide assortment of blob features, such as the blob area, perimeter, Feret diameter at a given angle, minimum bounding box, and center of gravity. The module also allows you to perform some image processing operations, such as reconstructing or eliminating blobs.

Functions

MblobAlloc
MblobAllocResult
MblobCalculate
MblobControl
MblobDraw
MblobFree
MblobGetLabel
MblobGetResult
MblobInquire
MblobLabel
MblobMerge
MblobReconstruct
MblobRestore
MblobSave
MblobSelect
MblobStream
MblobTransform

MblobAlloc

Synopsis

Allocate a blob analysis context.

Syntax

```
AIL_ID MblobAlloc(
    AIL_ID SysId,            //in
    AIL_INT64 ContextType,   //in
    AIL_INT64 ControlFlag,   //in
    AIL_ID *ContextBlobIdPtr //out
)
```

Description

This function allocates a blob analysis context. A blob analysis context contains all the information necessary to perform an MblobCalculate() operation, including global processing settings, and the blob features to calculate. Immediately after allocation, the context has only a few features enabled for calculation (for example, a blob's area). Use MblobControl() to enable addition features for calculation.

When the blob analysis context is no longer required, release it using MblobFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the blob analysis context's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the system on which to allocate the context.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ContextType

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

ContextBlobIdPtr

Specifies the address of the variable in which to write the blob analysis context identifier or specifies the data type that the function should use to return the blob analysis context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated blob analysis context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated blob analysis context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_BLOB_ID* is returned instead of a standard Aurora Imaging Library identifier. This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the blob analysis context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated blob analysis context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the blob analysis context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_BLOB_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailblob11.lib.
DLL	Requires ail11.dll; ailblob11.dll.

MblobAllocResult

Synopsis

Allocate a blob analysis result buffer.

Syntax

```
AIL_ID MblobAllocResult(
    AIL_ID SysId,           //in
    AIL_INT64 ResultType,   //in
    AIL_INT64 ControlFlag,  //in
    AIL_ID *ResultBlobIdPtr //out
)
```

Description

This function allocates a result buffer that can store blob analysis results.

Each blob creates a separate result entry in the blob analysis result buffer. You can retrieve blob analysis results from a result buffer, using MblobGetResult(). To determine the label value of a blob at a specified position, you can call MblobGetLabel().

When the blob analysis result buffer is no longer required, release it using MblobFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the blob analysis result buffer's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the system on which to allocate the result buffer.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ResultType

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

ResultBlobIdPtr

Specifies the address of the variable in which to write the blob analysis result buffer identifier or specifies the

data type that the function should use to return the blob analysis result buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated blob analysis result buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated blob analysis result buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_BLOB_ID* is returned instead of a standard Aurora Imaging Library identifier. This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the blob analysis result buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated blob analysis result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the blob analysis result buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_BLOB_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailblob11.lib.

DLL	Requires ail11.dll; ailblob11.dll.

MblobCalculate

Synopsis

Perform blob analysis calculations.

Syntax

```
void MblobCalculate(
    AIL_ID ContextBlobId,       //in
    AIL_ID BlobIdentImageBufId, //in
    AIL_ID GrayImageBufId,      //in
    AIL_ID ResultBlobId         //out
)
```

Description

This function calculates the specified features for all currently included blobs in the blob identifier image, and stores results in the specified result buffer. To specify the features to calculate, pass a blob analysis context that has the required features enabled for calculation. Use MblobControl() to enable the features. You can select the blobs on which to perform the calculations using MblobSelect().

Calculations of features with a binary definition (such as M_BREADTH) are performed using only the blob identifier image (BlobIdentImageBufId). You can limit blob analysis calculations to a region of the blob identifier image using a region of interest (ROI) set using MbufSetRegion(). In this case, the specified features are only calculated for included blobs (or parts of blobs) lying within the ROI. For more information, see the Regions of interest subsection of the Child buffers, regions of interest, and fixturing section of Chapter 2: Key information and building an application.

If a feature with a grayscale definition (such as M_MAX_PIXEL) is enabled for calculation, you must pass a grayscale image buffer to the GrayImageBufId parameter. In this case, the blob identifier image is used to identify the blobs and the grayscale image is used to provide the blob pixel values. Note that the blob identifier image and the grayscale image must be the same size. Instead of passing the blob identifier image separately, you can pass an image buffer (or container) with an ROI that identifies the blobs to the GrayImageBufId parameter. In this case, the BlobIdentImageBufId parameter must be set to **M_NULL**.

When calculating depth map related features (using MblobControl() with M_DEPTH_MAP_...), you must set BlobIdentImageBufId to **M_NULL**, and GrayImageBufId to a fully corrected depth map image buffer or 3D-processable depth map container that is associated with an ROI that acts as the blob identifier image as well as identifies invalid pixels. Note that if you don't pass a fully corrected depth map image buffer or 3D-processable depth map container, you will obtain an error if you try to retrieve a depth map feature result using MblobGetResult(); **MblobCalculate**() will not generate an error. Use MbufSetRegion() with M_RASTERIZE_DEPTH_MAP_VALID_PIXELS to associate an ROI with the specified depth map image. Note that for a depth map container, the ROI is stored in the M_COMPONENT_REGION_AIL component of the container.

The blob identifier image identifies each blob as a group of touching pixels in the current foreground state (zero or non-zero, based on MblobControl() with M_FOREGROUND_VALUE). The current M_CONNECTIVITY processing mode (also set with MblobControl()), determines when to consider pixels as touching.

If the blob identifier image has been previously binarized so that it contains only two extreme values (0 and 1 for 1-bit images, 0 and 0xff for 8-bit images, and 0 and 0xffff for 16-bit images), blob analysis can be performed a little faster. However, you must first set MblobControl() with M_IDENTIFIER_TYPE to M_BINARY, to let Aurora Imaging Library know that the blob identifier image has only two states.

Depending on the blob identification mode (MblobControl() with M_BLOB_IDENTIFICATION_MODE), **MblobCalculate**() either treats each blob individually (M_INDIVIDUAL), groups all blobs together

(M_WHOLE_IMAGE), groups blobs according to their actual pixel value in the blob identifier image (M_LABELED or M_LABELED_TOUCHING). Both M_LABELED and M_LABELED_TOUCHING treat blobs with the same pixel value as being grouped together. However, M_LABELED treats touching blobs as the same blob (regardless of their label); while M_LABELED_TOUCHING treats them as separate.

If you are calculating chain features (enabled using MblobControl() with M_CHAINS or M_ALL_FEATURES), you cannot use MblobControl() with M_BLOB_IDENTIFICATION_MODE set to M_LABELED_TOUCHING.

If several calls are made to **MblobCalculate**() with the same image or container, context, and result buffer, features calculated in one call remain in the result buffer and are not recalculated in subsequent calls. However, if you then use the result buffer with different images or a different container, or if you change the blob analysis context's processing settings using MblobControl() (constants from the For global processing settings of a blob analysis context that cause results to be recalculated table), any results already in the buffer become invalid and are discarded. Therefore, it is more efficient to use a result buffer exclusively in one processing mode with the same images/container.

If you have associated the blob identifier image with a camera calibration context, and GrayImageBufId is set to **M_NULL**, the result buffer will be associated with the same camera calibration context as the blob identifier image. However, if you provide a grayscale image that is not calibrated or does not have the same camera calibration context as the blob identifier image, the result buffer will not be calibrated (both the identifier and grayscale images must have the same camera calibration context for the result buffer to be calibrated).

Most feature calculations are initially performed in the pixel coordinate system. It is typically possible to transform the result of these calculations to the relative coordinate system using MblobControl() with M_RESULT_OUTPUT_UNITS. Those feature types whose calculation are not performed in the pixel coordinate system, or that cannot be transformed into the relative coordinate system, will have that fact stated in the result type's description in MblobGetResult(). Refer to the description of a particular feature's result, in MblobGetResult(), to determine if it can be transformed and returned in the relative coordinate system. Note that some feature results such as M_NUMBER_OF_HOLES do not have a different meaning in either the pixel coordinate system or the relative coordinate system.

Parameters

ContextBlobId

Specifies the identifier of the blob analysis context, that specifies the feature(s) to calculate, and other settings related to the blob analysis operation. The blob analysis context must have been previously allocated on the required system with MblobAlloc().

BlobIdentImageBufId

Specifies the identifier of the image buffer to use as the blob identifier image. This image is used to distinguish blob pixels from background pixels and which pixels belong to which blob. The blob identifier image buffer must be a packed binary, 8-, 16-, or 32-bit unsigned, single-band image buffer. 32-bit image buffers are only supported when the blob identification mode is set to M_LABELED_TOUCHING.

This image buffer can have a region of interest (ROI) defined using MbufSetRegion(). The region must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR).

You can set this parameter to **M_NULL** to use the ROI of the grayscale buffer as the blob identifier image. An error is generated if this parameter is set to **M_NULL** and there is either no grayscale buffer, or the grayscale buffer has no associated ROI.

GrayImageBufId

Specifies the identifier of the depth map container or the image buffer to use as the grayscale image. This buffer is used to supply the intensity of blob pixels when calculating features with a grayscale definition. If this

parameter is set to **M_NULL**, features with a grayscale definition, such as M_SUM_PIXEL, cannot be calculated. This parameter is ignored when calculating features with only a binary definition.

The depth map container must have been previously allocated using MbufAllocContainer() with M_PROC, and must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE). This container must have a region component (M_COMPONENT_REGION_AIL). Using a container without a region component will cause an error.

The grayscale image buffer must be an 8-bit or 16-bit, unsigned, single band image buffer. This image buffer can have a region of interest (ROI) defined using MbufSetRegion(). The region must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error will be generated if the ROI is only in vector format (M_VECTOR).

ResultBlobId

Specifies the identifier of a blob analysis result buffer, in which to store calculated results. The blob analysis result buffer must have been previously allocated on the required system using MblobAllocResult().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailblob11.lib.
DLL	Requires ail11.dll; ailblob11.dll.

MblobControl

Synopsis

Control a blob analysis setting.

Syntax

```
void MblobControl(
    AIL_ID ContextOrResultBlobId, //out
    AIL_INT64 ControlType,        //in
    AIL_DOUBLE ControlValue       //in
)
```

Description

This function allows you to control a setting of the specified blob analysis context or result buffer.

For blob analysis contexts, these settings control the execution of MblobCalculate(), and which blob features are enabled for calculation. For blob analysis result buffers, these settings control the post-manipulation of results. If **MblobControl**() is not called, default processing controls and values are used in calculations.

You can inquire about most of these settings using MblobInquire().

Changing certain control type settings will cause MblobCalculate() to discard all results currently in your result buffer prior to calculation. If this is the case, it will be mentioned in the preamble to the table containing these control types.

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **MblobControl()** internally calls the *AIL_DOUBLE* version of this function (MblobControlDouble()). If you need to pass large integer values (including pointers), call the *AIL_INT64* version of this function (MblobControlInt64()).

Parameters

ContextOrResultBlobId

Specifies the identifier of the blob analysis context or result buffer.

ControlType

Specifies the setting to change.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the setting's new value.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **ControlType** and **ControlValue** parameters.

- For global processing settings of a blob analysis context that cause results to be recalculated
- For global processing settings of a blob analysis context that do not cause results to be recalculated
- For specifying to calculate all features in a single call
- For specifying to calculate features with only a binary definition
- For specifying to calculate features with only a grayscale definition

- For specifying to calculate features with two definitions (binary and grayscale)
- For specifying to calculate features related to depth maps
- For a blob analysis result buffer

For the following global processing setting control types, you must set the ContextOrResultBlobId parameter to a blob analysis context.

Changing the value of the following control types between calls to MblobCalculate() will cause all the results currently in your Blob analysis result buffer to be discarded upon the next call to MblobCalculate().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For global processing settings of a blob analysis context that cause results to be recalculated																		
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																	
M_BLOB_IDENTIFICATION_MODE		Sets whether to calculate features for each blob, or to treat groups of blobs as single blobs and calculate the features of these blobs as if they were one blob. INQ																
	M_DEFAULT	Same as M_INDIVIDUAL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INDIVIDUAL	Specifies that all blobs are measured individually.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LABELED	Specifies that blobs with the same label are grouped together, and that touching blobs with different labels are also grouped together. The foreground value of the blob identifier image will be used as the label value of the blob; therefore, when using M_LABELED, the M_FOREGROUND_VALUE control type cannot be set to M_ZERO, and the identifier image cannot be binary (1-bit). M_LABELED is not supported for merge operations using MblobMerge().	c		e	f	g	a		i		j	k	l	m	n	b

	M_LABELED_TOUCHING	Specifies that blobs with the same label are grouped together, and that touching blobs with different labels are measured individually. The foreground value of the blob identifier image will be used as the label value of the blob; therefore, when using M_LABELED_TOUCHING, the M_FOREGROUND_VALUE control type cannot be set to M_ZERO. In addition, the blob identifier image cannot be binary (1-bit). M_LABELED_TOUCHING is not supported with the following: • MblobCalculate(), when chain features are enabled for calculation (**MblobControl**() with M_CHAINS, or M_ALL_FEATURES). • MblobDraw() with M_DRAW_BLOBS_CONTOUR, M_DRAW_HOLES_CONTOUR, or a combination of the two. • MblobSelect() with M_MERGE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_WHOLE_IMAGE	Specifies that all blobs are grouped together. If the blob identifier image is already binarized (for example, pixel values for an 8-bit image are either 0 or 0xff), you can set M_IDENTIFIER_TYPE to M_BINARY to calculate features faster.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONNECTIVITY		Sets the image lattice for blob analysis. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_8_CONNECTED.	c		e	f	g	a		i		j	k	l	m	n	b
	M_4_CONNECTED	Specifies that each pixel has 4 neighbors.	c		e	f	g	a		i		j	k	l	m	n	b
	M_8_CONNECTED	Specifies that each pixel has 8 neighbors.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_ANGLE_SEARCH_END		Sets the upper limit of the angular search range in which to search for the maximum (M_FERET_MAX_DIAMETER) or minimum (M_FERET_MIN_DIAMETER) Feret diameters. Note that MblobCalculate() only searches through the specified range when calculating Feret features. Better results for the features can exist outside of this search range. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 180.	c		e	f	g	a		i		j	k	l	m	n	b

	0 <= *Value* <= 180	Specifies the upper limit of the angular search range. The end value must be greater than the start value.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_ANGLE_SEARCH_START		Sets the lower limit of the angular search range in which to search for the maximum (M_FERET_MAX_DIAMETER) or minimum (M_FERET_MIN_DIAMETER) Feret diameters. Note that MblobCalculate() only searches through the specified range when calculating Feret features. Better results for the features can exist outside of this search range. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 180	Specifies the lower limit of the angular search range. The start value must be less than the end value.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_VALUE		Sets which pixel values are considered to be in the foreground. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_NONZERO.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NONZERO	Specifies the blobs consisting of non-zero pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ZERO	Specifies the blobs consisting of zero pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_FERETS		Sets the number of Feret angles to use when calculating a Feret feature (M_FERET_...) or a feature based on Ferets (that is, M_RECTANGULARITY, M_ROUGHNESS, or M_CONVEX_PERIMETER). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 8.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFINITE	Specifies the use of a high precision algorithm to accurately calculate Feret features (and features based on Feret features). > The M_FERET_DIAMETERS result type is not supported when you use this setting.	c		e	f	g	a		i		j	k	l	m	n	b

	M_MIN_FERETS	Specifies the minimum number of Feret angles. The minimum number of Feret angles is 2.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > M_MIN_FERETS	Specifies the number of Feret angles. The first Feret angle used is always 0°, and the difference between successive angles is 180° / number of Ferets.	c		e	f	g	a		i		j	k	l	m	n	b

For the following global processing setting control types, you must set the ContextOrResultBlobId parameter to a blob analysis context. These control types do not cause any results currently in the result buffer to be discarded when MblobCalculate() is called.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For global processing settings of a blob analysis context that do not cause results to be recalculated																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_FERET_CONTACT_POINTS		Sets whether to calculate Feret contact points for features that support it (for example, M_FERET_AT_PRINCIPAL_AXIS_ANGLE) if the feature has been enabled for calculation. This enables the use of the M_FERET_CONTACT_POINTS_... combination constants in MblobGetResult(). INQ															
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that the Feret contact points for supported features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that the Feret contact points for supported features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_GENERAL_ANGLE		Sets the angle to use when calculating M_FERET_GENERAL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	-360.0 <= *Value* <= 360.0	Specifies the angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b

M_IDENTIFIER_TYPE	Sets the values that non-zero pixels in the blob identifier image can have. If the blob identifier image is already binarized (for example, pixel values for an 8-bit image are either 0 or 0xff), you can set this control type to M_BINARY to calculate features faster. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_GRAYSCALE.	c		e	f	g	a		i		j	k	l	m	n	b
M_BINARY	Specifies that non-zero pixels must have the maximum value of the buffer (for example, 0xff for an 8-bit image).	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAYSCALE	Specifies that non-zero pixels can have any value.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_BLOBS	Sets the maximum number of blobs to process. If this number is reached, processing is stopped, and results available in the result buffer become invalid and are discarded unless M_RETURN_PARTIAL_RESULTS is enabled. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that there is no limit on the maximum number of blobs.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the maximum number of blobs.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_GENERAL_MODE	Sets the type of moment to calculate when calculating a user-specified moment (M_MOMENT_GENERAL). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ORDINARY.	c		e	f	g	a		i		j	k	l	m	n	b

	M_CENTRAL	Specifies a central moment. A central moment is defined as $\sum_{i}^{n}(x_i-\bar{x})^q(y_i-\bar{y})^r\ p_i$ where *i* runs from 1 to the number of pixels in the blob, p_i = value of a pixel (always 1 for binary moments), x_i = its X-coordinate, y_i = its Y-coordinate, $\bar{x}$ = the weighted mean of x_i , $\bar{y}$ = the weighted mean of y_i , *q* and *r* are the order of the moment in X and Y respectively. The weighted mean is the first-order ordinary moment divided by the sum of the pixel values (M_SUM_PIXEL for grayscale definitions, M_AREA for binary definitions). Central moments use coordinates relative to each blob's center of gravity, and are therefore independent of a blob's position within an image.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ORDINARY	Specifies an ordinary moment. An ordinary moment is defined as $\sum_{i}^{n}x_i^q y_i^r\ p_i$ where *i* runs from 1 to the number of pixels in the blob, p_i = value of a pixel (always 1 for binary moments), x_i = its X-coordinate, y_i = its Y-coordinate, and (*q* + *r*) are the order of the moment in X and Y respectively. Ordinary moments are affected by the blob position because they use coordinates relative to the image origin (top-left corner).	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_GENERAL_ORDER_X		Sets the order of the X-component of the user-specified moment. Use this control type when calculating a user-specified moment (M_MOMENT_GENERAL). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the X-order of the moment.	c		e	f	g	a		i		j	k	l	m	n	b

M_MOMENT_GENERAL_ORDER_Y	Sets the order of the Y-component of the user-specified moment. Use this control type when calculating a user-specified moment (M_MOMENT_GENERAL). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the Y-order of the moment.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_THIRD_ORDER_MODE	Sets how to calculate third-order moments. M_FAST is faster than M_PRECISE, but it might cause some imprecisions. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_FAST.	c		e	f	g	a		i		j	k	l	m	n	b
M_FAST	Specifies that third-order moments will be calculated quickly.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRECISE	Specifies that third-order moments will be calculated accurately.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_ASPECT_RATIO	Sets the pixel aspect ratio of the image(s). This is only useful if the image has a uniform pixel aspect ratio with different X- and Y-scales. For non-uniform distortion, you can use McalGrid() or McalList() to calibrate the image, and McalTransformImage() to physically correct the image. All results returned by MblobGetResult() are affected by M_PIXEL_ASPECT_RATIO, including those that are simply positions within the image. For more information, see the Pixel aspect ratio subsection of the Adjusting blob analysis processing controls section of Chapter 6: Blob analysis. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the pixel width/pixel height.	c		e	f	g	a		i		j	k	l	m	n	b

M_RETURN_PARTIAL_RESULTS	Sets whether results from partially scanned images will be available after a stop condition is met. The three stop conditions are: • The maximum number of blobs (M_MAX_BLOBS) has been processed. • The timeout limit (M_TIMEOUT) is exceeded. • The calculation has been stopped by the user (M_STOP_CALCULATE). When a stop condition is met, the image is only partially scanned and results about blobs found in the partially scanned image are discarded. When M_RETURN_PARTIAL_RESULTS is set to M_ENABLE, you can access the results of those blobs that are completely defined (those blobs whose entire perimeter was scanned) in the partially scanned image. Note that when using multi-processing (MappControlMp() with M_MP_USE set to M_ENABLE) and this control type is enabled, it is possible that the number of blobs returned can exceed the number set using M_MAX_BLOBS. This is because multi-processing might divide the image into pieces and process each piece independently; in this case, there is no global number of blobs counted for the entire image until everything is processed. When a stop condition is met in one of the image pieces, processing is stopped everywhere. The total number of blobs is then combined and returned, which can be greater than M_MAX_BLOBS. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to discard results of partially scanned images when processing is interrupted.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to make results of partially scanned images available when processing is interrupted.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE_RUNS	Sets whether to save run information when calling MblobCalculate(). INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to save run information. Disabling saves time when performing the first call to MblobCalculate() and reduces the memory requirements for the result buffer. However, you cannot use MblobLabel(), or MblobGetLabel(), and certain draw operations are disabled. See MblobDraw() for more information on which draw operations require run information to be saved. In addition, using **MblobControl**(), you cannot enable for calculation the chained pixels features (M_CHAINS), the convex hull features (M_CONVEX_HULL), M_NUMBER_OF_FERETS when it is set to M_INFINITE, and the Feret features (M_FERETS).	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to save run information. Calls to MblobCalculate() will save run information from the blob identifier image in the result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_SORT*n*		Sets the feature to use as the *n* th sorting key for results, where *n* stands for an integer between 1 and 3. Changes to this control type take effect upon the next call to MblobCalculate(). If you select a feature as a sorting key that is not enabled, an error will occur on the next call to MblobCalculate(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_NO_SORT.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AREA	Specifies to sort by the number of foreground pixels in each blob (holes are not counted). INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_AXIS_PRINCIPAL_ANGLE +	Specifies to sort by the angle, in degrees, at which each blob has the least moment of inertia. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_AXIS_SECONDARY_ANGLE +	Specifies to sort by the angle perpendicular to M_AXIS_PRINCIPAL_ANGLE of each blob, in degrees. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BLOB_CONTRAST	Specifies to sort by the difference between the maximum and minimum pixel values of a blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_BLOB_TOUCHING_IMAGE_BORDERS	Specifies to sort by whether each blob is touching the borders of the image. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BOX_AREA	Specifies to sort by the area covered by the image-axis-aligned bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BOX_ASPECT_RATIO	Specifies to sort by the ratio between the horizontal size and the vertical size of the image-axis-aligned bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BOX_FILL_RATIO	Specifies to sort by the ratio between the area of the blob and the area of the image-axis-aligned bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BOX_X_MAX	Specifies to sort by the extreme right X-coordinate of each blob, of the image-axis-aligned bounding box. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BOX_X_MIN	Specifies to sort by the extreme left X-coordinate of each blob, of the image-axis-aligned bounding box. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BOX_Y_MAX	Specifies to sort by the extreme bottom Y-coordinate of each blob, of the image-axis-aligned bounding box. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BOX_Y_MIN	Specifies to sort by the extreme top Y-coordinate of each blob, of the image-axis-aligned bounding box. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BREADTH	Specifies to sort by the breadth of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CENTER_OF_GRAVITY_X +	Specifies to sort by the X-position of the center of gravity of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CENTER_OF_GRAVITY_Y +	Specifies to sort by the Y-position of the center of gravity of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPACTNESS	Specifies to sort by the compactness of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_CONVEX_HULL_AREA	Specifies to sort by the area of the convex hull of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CONVEX_HULL_COG_X	Specifies to sort by the X-component of the center of gravity of the convex hull of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CONVEX_HULL_COG_Y	Specifies to sort by the Y-component of the center of gravity of the convex hull of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CONVEX_HULL_FILL_RATIO	Specifies to sort by the ratio between the blob's area and the area of its convex hull. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CONVEX_HULL_PERIMETER	Specifies to sort by the perimeter of the convex hull of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CONVEX_PERIMETER	Specifies to sort by an approximation of the perimeter of the convex hull of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEPTH_MAP_MAX_ELEVATION	Specifies to sort by the maximum elevation of each blob.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEPTH_MAP_MEAN_ELEVATION	Specifies to sort by the mean elevation of each blob.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEPTH_MAP_MIN_ELEVATION	Specifies to sort by the minimum elevation of each blob.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEPTH_MAP_SIZE_Z	Specifies to sort by the Z-size (difference between maximum and minimum elevation) of each blob.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEPTH_MAP_VOLUME	Specifies to sort by the volume of each blob.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ELONGATION	Specifies to sort by a value that is equal to M_LENGTH / M_BREADTH of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_EULER_NUMBER	Specifies to sort by the number of blobs minus the number of holes (number of blobs - number of holes). INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_FERET_AT_PRINCIPAL_AXIS_ANGLE +	Specifies to sort by the Feret diameter at the principal axis of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_AT_SECONDARY_AXIS_ANGLE +	Specifies to sort by the Feret diameter at the secondary axis of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_ELONGATION	Specifies to sort by the measure of the shape of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_GENERAL	Specifies to sort by the Feret diameter at the user-specified angle (set using **MblobControl**() with M_FERET_GENERAL_ANGLE). INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_MAX_ANGLE	Specifies to sort by the angle, in degrees, at which the maximum Feret diameter is found for each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_MAX_DIAMETER	Specifies to sort by the largest Feret diameter found after checking a certain number of angles. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_MAX_DIAMETER_ELONGATION	Specifies to sort by the ratio between the maximum Feret diameter of each blob and its perpendicular Feret diameter. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_MEAN_DIAMETER	Specifies to sort by the average of the Feret diameters at the angles checked for each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_MIN_ANGLE	Specifies to sort by the angle, in degrees, at which the minimum Feret diameter is found for each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_MIN_DIAMETER	Specifies to sort by the minimum Feret diameter found after checking a certain number of angles. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_MIN_DIAMETER_ELONGATION	Specifies to sort by the ratio between the minimum Feret diameter of each blob and its perpendicular Feret diameter. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_PERPENDICULAR_TO_MAX_DIAMETER	Specifies to sort by the Feret diameter that is perpendicular to the maximum Feret diameter of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_FERET_PERPENDICULAR_TO_MIN_DIAMETER	Specifies to sort by the Feret diameter that is perpendicular to the minimum Feret diameter of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_PRINCIPAL_AXIS_ELONGATION +	Specifies to sort by the ratio between the Feret diameter at the principal axis and the Feret diameter at the secondary axis of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_X	Specifies to sort by the dimension of the image-axis-aligned minimum bounding box of each blob, along the X-axis of the pixel coordinate system (that is, M_BOX_X_MAX - M_BOX_X_MIN + 1). INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_Y	Specifies to sort by the dimension of the image-axis-aligned minimum bounding box of each blob, along the Y-axis of the pixel coordinate system (that is, M_BOX_Y_MAX - M_BOX_Y_MIN + 1). INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FIRST_POINT_X	Specifies to sort by the X-coordinate of a unique point (along with M_FIRST_POINT_Y) for each blob, that is on the perimeter of the blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FIRST_POINT_Y	Specifies to sort by the Y-coordinate of a unique point (along with M_FIRST_POINT_X) for each blob, that is on the perimeter of the blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTERCEPT_0	Specifies to sort by the number of times a transition from background to foreground (not vice versa) occurs in the horizontal direction for the entire blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTERCEPT_45	Specifies to sort by the number of times that the neighborhood configuration $\begin{bmatrix} \bullet & F \\ B & \bullet \end{bmatrix}$ occurs in a blob, where *F* is a foreground pixel, *B* is a background pixel, and a dot can be any pixel value. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_INTERCEPT_90	Specifies to sort by the number of times that the neighborhood configuration $\begin{bmatrix} F \\ B \end{bmatrix}$ occurs in a blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTERCEPT_135	Specifies to sort by the number of times that the neighborhood configuration $\begin{bmatrix} F & \bullet \\ \bullet & B \end{bmatrix}$ occurs in a blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_LABEL_VALUE	Specifies to sort by the label value of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_LENGTH	Specifies to sort by the length of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MAX_PIXEL	Specifies to sort by the maximum pixel value found in each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MEAN_PIXEL	Specifies to sort by the mean pixel value in each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_AREA_BOX_ANGLE	Specifies to sort by the angle of the minimum-area bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_AREA_BOX_AREA	Specifies to sort by the area of the minimum-area bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_AREA_BOX_CENTER_X	Specifies to sort by the X-coordinate of the center of the minimum-area bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_AREA_BOX_CENTER_Y	Specifies to sort by the Y-coordinate of the center of the minimum-area bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_AREA_BOX_HEIGHT	Specifies to sort by the height (shortest side) of the minimum-area bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_AREA_BOX_PERIMETER	Specifies to sort by the perimeter of the minimum-area bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_MIN_AREA_BOX_WIDTH	Specifies to sort by the width (longest side) of the minimum-area bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_PERIMETER_BOX_ANGLE	Specifies to sort by the angle of the minimum-perimeter bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_PERIMETER_BOX_AREA	Specifies to sort by the area of the minimum-perimeter bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_PERIMETER_BOX_CENTER_X	Specifies to sort by the X-coordinate of the center of the minimum-perimeter bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_PERIMETER_BOX_CENTER_Y	Specifies to sort by the Y-coordinate of the center of the minimum-perimeter bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_PERIMETER_BOX_HEIGHT	Specifies to sort by the height (shortest side) of the minimum-perimeter bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_PERIMETER_BOX_PERIMETER	Specifies to sort by the perimeter of the minimum-perimeter bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_PERIMETER_BOX_WIDTH	Specifies to sort by the width (longest side) of the minimum-perimeter bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_PIXEL	Specifies to sort by the minimum pixel value found in each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_CENTRAL_X0_Y2 +	Specifies to sort by the central moment of each blob where the order of X equals 0 and the order of Y equals 2. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_CENTRAL_X0_Y3 +	Specifies to sort by the central moment of each blob where the order of X equals 0 and the order of Y equals 3. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_MOMENT_CENTRAL_X1_Y1 +	Specifies to sort by the central moment of each blob where the order of X equals 1 and the order of Y equals 1. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_CENTRAL_X1_Y2 +	Specifies to sort by the central moment of each blob where the order of X equals 1 and the order of Y equals 2. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_CENTRAL_X2_Y0 +	Specifies to sort by the central moment of each blob where the order of X equals 2 and the order of Y equals 0. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_CENTRAL_X2_Y1 +	Specifies to sort by the central moment of each blob where the order of X equals 2 and the order of Y equals 1. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_CENTRAL_X3_Y0 +	Specifies to sort by the central moment of each blob where the order of X equals 3 and the order of Y equals 0. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_GENERAL +	Specifies to sort by the moment calculation, based on the settings specified using **MblobControl**() with M_MOMENT_GENERAL_MODE, M_MOMENT_GENERAL_ORDER_X, and M_MOMENT_GENERAL_ORDER_Y. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_HU_*n* +	Specifies to sort by the nth Hu moment of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_X0_Y1 +	Specifies to sort by the ordinary moment of each blob where the order of X equals 0 and the order of Y equals 1. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_X0_Y2 +	Specifies to sort by the ordinary moment of each blob where the order of X equals 0 and the order of Y equals 2. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_X0_Y3 +	Specifies to sort by the ordinary moment of each blob where the order of X equals 0 and the order of Y equals 3. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_MOMENT_X1_Y0 +	Specifies to sort by the ordinary moment of each blob where the order of X equals 1 and the order of Y equals 0. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_X1_Y1 +	Specifies to sort by the ordinary moment of each blob where the order of X equals 1 and the order of Y equals 1. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_X1_Y2 +	Specifies to sort by the ordinary moment of each blob where the order of X equals 1 and the order of Y equals 2. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_X2_Y0 +	Specifies to sort by the ordinary moment of each blob where the order of X equals 2 and the order of Y equals 0. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_X2_Y1 +	Specifies to sort by the ordinary moment of each blob where the order of X equals 2 and the order of Y equals 1. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_X3_Y0 +	Specifies to sort by the ordinary moment of each blob where the order of X equals 3 and the order of Y equals 0. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_NO_SORT	Specifies to remove the sorting key.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NUMBER_OF_CHAINED_PIXELS	Specifies to sort by the number of chained pixels in each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_NUMBER_OF_CONVEX_HULL_POINTS	Specifies to sort by the number of points on the convex perimeter of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_NUMBER_OF_HOLES	Specifies to sort by the number of holes in each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_NUMBER_OF_RUNS	Specifies to sort by the total number of runs in each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_PERIMETER	Specifies to sort by the total length of edges in each blob (including the edges of any holes), whereby an allowance made for the staircase effect that is produced when diagonal edges are digitized (inside corners are counted as 1.414, rather than 2.0). INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_RECTANGULARITY	Specifies to sort by the degree to which each blob is similar to a rectangle. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ROUGHNESS	Specifies to sort by the roughness and irregularity of a blob is, and is equal to M_PERIMETER / M_CONVEX_PERIMETER. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SIGMA_PIXEL	Specifies to sort by the standard deviation of pixel values in each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SUM_PIXEL	Specifies to sort by the sum of all pixel values in each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SUM_PIXEL_SQUARED	Specifies to sort by the sum of the squares of each pixel value in each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD_BOX_X_MAX	Specifies to sort by the extreme right X-coordinate of the world-axis-aligned bounding box, of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD_BOX_X_MIN	Specifies to sort by the extreme left X-coordinate of the world-axis-aligned bounding box, of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD_BOX_Y_MAX	Specifies to sort by the extreme bottom Y-coordinate of the world-axis-aligned bounding box, of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD_BOX_Y_MIN	Specifies to sort by the extreme top Y-coordinate of the world-axis-aligned bounding box, of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_WORLD_FERET_X	Specifies to sort by the dimension of the world-axis-aligned minimum bounding box, along the X-axis of the relative coordinate system (that is, M_WORLD_BOX_X_MAX - M_WORLD_BOX_X_MIN + 1). INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD_FERET_Y	Specifies to sort by the dimension of the world-axis-aligned minimum bounding box, along the Y-axis of the relative coordinate system (that is, M_WORLD_BOX_Y_MAX - M_WORLD_BOX_Y_MIN + 1). INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD_X_AT_Y_MAX	Specifies to sort by the X-coordinate at the maximum Y-coordinate of each blob, of the world-axis-aligned bounding box. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD_X_AT_Y_MIN	Specifies to sort by the X-coordinate at the minimum Y-coordinate of each blob, of the world-axis-aligned bounding box. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD_Y_AT_X_MAX	Specifies to sort by the Y-coordinate at the maximum X-coordinate of each blob, of the world-axis-aligned bounding box. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD_Y_AT_X_MIN	Specifies to sort by the Y-coordinate at the minimum X-coordinate of each blob, of the world-axis-aligned bounding box. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_X_MAX_AT_Y_MAX	Specifies to sort by the maximum X-coordinate at the maximum Y-coordinate of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_X_MAX_AT_Y_MIN	Specifies to sort by the maximum X-coordinate at the minimum Y-coordinate of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_X_MIN_AT_Y_MAX	Specifies to sort by the minimum X-coordinate at the maximum Y-coordinate of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_X_MIN_AT_Y_MIN	Specifies to sort by the minimum X-coordinate at the minimum Y-coordinate of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_Y_MAX_AT_X_MAX	Specifies to sort by the maximum Y-coordinate at the maximum X-coordinate of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_Y_MAX_AT_X_MIN	Specifies to sort by the maximum Y-coordinate at the minimum X-coordinate of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_Y_MIN_AT_X_MAX	Specifies to sort by the minimum Y-coordinate at the maximum X-coordinate of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_Y_MIN_AT_X_MIN	Specifies to sort by the minimum Y-coordinate at the minimum X-coordinate of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SORT*n*_DIRECTION	Sets the direction in which the *n* th sorting key is sorted, where *n* stands for an integer between 1 and 3. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_SORT_UP.	c		e	f	g	a		i		j	k	l	m	n	b
M_SORT_DOWN	Specifies the feature as being sorted in descending order.	c		e	f	g	a		i		j	k	l	m	n	b
M_SORT_UP	Specifies the feature as being sorted in ascending order.	c		e	f	g	a		i		j	k	l	m	n	b
M_TIMEOUT	Sets the maximum processing time, in msec. If the timeout limit is exceeded, processing is stopped, and results already available in the result buffer become invalid and will be discarded, unless M_RETURN_PARTIAL_RESULTS is enabled. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that there is no maximum processing time.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the maximum processing time, in msec.	c		e	f	g	a		i		j	k	l	m	n	b

The following control type allows you to enable or disable all features for calculation using a single call. For the following control type, you must set the ContextOrResultBlobId parameter to a blob analysis context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying to calculate all features in a single call																		
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																	
M_ALL_FEATURES		Sets whether to calculate all features. You can then selectively enable or disable specific features for calculation. It is not recommended to use M_ALL_FEATURES in your final application. This is because programs using M_ALL_FEATURES set to M_ENABLE might become slower when new Aurora Imaging Library versions are released, if new features have been added.																
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable the calculation of all features, regardless of their previous setting. This can be used as a quick method to disable the calculation of all features.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that all of the features will be calculated, regardless of their previous setting.	c		e	f	g	a		i		j	k	l	m	n	b

The following control types establish which of the blob features, that have only a binary definition, are calculated for each blob; these blob features are calculated using only the blob identifier image. For the following control types, you must set the ContextOrResultBlobId parameter to a blob analysis context. Note that, unless otherwise stated, these features are calculated in pixel units; although you can retrieve them in world units using MblobGetResult() if calculated on a calibrated blob identifier image and M_RESULT_OUTPUT_UNITS is set to M_WORLD or M_ACCORDING_TO_CALIBRATION.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying to calculate features with only a binary definition																		
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																	
M_BOX		Sets whether to calculate all image-axis-aligned bounding box features plus X- and Y-Ferets. The image-axis-aligned bounding box is the bounding box that is aligned with the pixel coordinate system's axes. You can retrieve results for these features using MblobGetResult() with M_BOX_X_MIN, M_BOX_Y_MIN, M_BOX_X_MAX, M_BOX_Y_MAX, M_BLOB_TOUCHING_IMAGE_BORDERS, M_BOX_AREA, M_BOX_ASPECT_RATIO, M_BOX_FILL_RATIO, M_FIRST_POINT_X, M_FIRST_POINT_Y, M_FERET_X, and M_FERET_Y. INQ																
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_BREADTH		Sets whether to calculate the breadth of a blob. This feature has the same advantages and disadvantages as M_LENGTH; that is, this feature is more accurate for long thin blobs because it is derived from the perimeter (P) and area (A) assuming that P = 2(length + breadth) and A = length x breadth. You can retrieve results for this feature using MblobGetResult() with M_BREADTH. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAINS		Sets whether to calculate all 4 chain features. You can retrieve results for these features using MblobGetResult() with M_CHAIN_X, M_CHAIN_Y, M_CHAIN_INDEX, M_NUMBER_OF_CHAINED_PIXELS, and M_TOTAL_NUMBER_OF_CHAINED_PIXELS. Note that the chain code is computed depending on the lattice and foreground values set with M_CONNECTIVITY and M_FOREGROUND_VALUE, respectively. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPACTNESS		Sets whether to calculate the compactness value of a blob. This is defined as the ratio between the area of a circle with the same perimeter as the blob in question, and the area of the blob itself. You can retrieve results for this feature using MblobGetResult() with M_COMPACTNESS. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONTACT_POINTS		Sets whether to calculate the contact features of the image-axis-aligned bounding box. The image-axis-aligned bounding box is the bounding box that is aligned with the pixel coordinate system's axes. You can retrieve results for these features using MblobGetResult() with M_X_MIN_AT_Y_MIN, M_X_MAX_AT_Y_MIN, M_X_MAX_AT_Y_MAX, M_X_MIN_AT_Y_MAX, M_Y_MIN_AT_X_MAX, M_Y_MAX_AT_X_MAX, M_Y_MAX_AT_X_MIN, and M_Y_MIN_AT_X_MIN. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONVEX_HULL		Sets whether to calculate all convex hull features. You can retrieve results for these features using MblobGetResult() with M_CONVEX_HULL_AREA, M_CONVEX_HULL_COG_X, M_CONVEX_HULL_COG_Y, M_CONVEX_HULL_FILL_RATIO, M_CONVEX_HULL_PERIMETER, M_CONVEX_HULL_X, M_CONVEX_HULL_XY_PACKED, M_CONVEX_HULL_Y, M_NUMBER_OF_CONVEX_HULL_POINTS, and M_TOTAL_NUMBER_OF_CONVEX_HULL_POINTS. For a quicker result, you can select to calculate M_CONVEX_PERIMETER. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

M_CONVEX_PERIMETER		Sets whether to calculate the approximation of the perimeter of the convex hull of a blob. It is derived from several Feret diameters; so, a larger number of Ferets gives a more accurate result. You can use M_NUMBER_OF_FERETS to change the number of Ferets angles used when calculating this feature. You can retrieve results for this feature using MblobGetResult() with M_CONVEX_PERIMETER. For a more accurate result, you can select to calculate M_CONVEX_HULL_PERIMETER by enabling M_CONVEX_HULL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_ELONGATION		Sets whether to calculate a value that is equal to M_LENGTH / M_BREADTH. It is similar to M_FERET_ELONGATION, except that it should be used for long thin blobs. You can retrieve results for this feature using MblobGetResult() with M_ELONGATION. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_EULER_NUMBER		Sets whether to calculate the number of blobs - number of holes. This value is more useful for M_WHOLE_IMAGE than for M_INDIVIDUAL processing mode. When using the M_WHOLE_IMAGE processing mode, each blob is treated individually, as opposed to grouping all of the blobs in an image together. When using the M_INDIVIDUAL processing mode, the number of blobs is effectively 1. You can retrieve results for this feature using MblobGetResult() with M_EULER_NUMBER. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

M_FERET_GENERAL		Sets whether to calculate a Feret diameter at a user-specified angle specified using M_FERET_GENERAL_ANGLE. You can retrieve results for this feature using MblobGetResult() with M_FERET_GENERAL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_MAX_DIAMETER_ELONGATION		Sets whether to calculate the ratio between the maximum Feret diameter and its perpendicular Feret diameter. Use the M_FERET_ANGLE_SEARCH_... constants to control the angular range in which to find the maximum Feret diameter. You can retrieve results for this feature using MblobGetResult() with M_FERET_MAX_DIAMETER_ELONGATION. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_MIN_DIAMETER_ELONGATION		Sets whether to calculate the ratio between the minimum Feret diameter and its perpendicular Feret diameter. Use the M_FERET_ANGLE_SEARCH_... constants to control the angular range in which to find the minimum Feret diameter. You can retrieve results for this feature using MblobGetResult() with M_FERET_MIN_DIAMETER_ELONGATION. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

M_FERET_PERPENDICULAR_TO_MAX_DIAMETER		Sets whether to calculate the Feret diameter that is perpendicular to the maximum Feret diameter. Use the M_FERET_ANGLE_SEARCH_... constants to control the angular range in which to find the maximum Feret diameter. You can retrieve results for this feature using MblobGetResult() with M_FERET_PERPENDICULAR_TO_MAX_DIAMETER. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_PERPENDICULAR_TO_MIN_DIAMETER		Sets whether to calculate the Feret diameter that is perpendicular to the minimum Feret diameter. Use the M_FERET_ANGLE_SEARCH_... constants to control the angular range in which to find the minimum Feret diameter. You can retrieve results for this feature using MblobGetResult() with M_FERET_PERPENDICULAR_TO_MIN_DIAMETER. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERETS		Sets whether to calculate the Feret features. You can retrieve results for these features using MblobGetResult() with M_FERET_DIAMETERS, M_TOTAL_NUMBER_OF_FERETS, M_NUMBER_OF_FERETS, M_FERET_ELONGATION, M_FERET_MAX_ANGLE, M_FERET_MIN_ANGLE, M_FERET_MAX_DIAMETER, M_FERET_MIN_DIAMETER, and M_FERET_MEAN_DIAMETER. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

M_INTERCEPT		Sets whether to calculate the intercept features. You can retrieve results for these features using MblobGetResult() with M_INTERCEPT_0, M_INTERCEPT_45, M_INTERCEPT_90, and M_INTERCEPT_135. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_LENGTH		Sets whether to calculate the length of a blob. This feature is more accurate for long thin blobs because it is derived from the perimeter (P) and area (A) assuming that P = 2(length + breadth) and A = length x breadth. You can retrieve results for this feature using MblobGetResult() with M_LENGTH. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_AREA_BOX		Sets whether to calculate all the minimum-area bounding box features. You can retrieve results for these features using MblobGetResult() with M_MIN_AREA_BOX_AREA, M_MIN_AREA_BOX_ANGLE, M_MIN_AREA_BOX_CENTER_X, M_MIN_AREA_BOX_CENTER_Y, M_MIN_AREA_BOX_HEIGHT, M_MIN_AREA_BOX_PERIMETER, M_MIN_AREA_BOX_WIDTH, M_MIN_AREA_BOX_Xn, and M_MIN_AREA_BOX_Yn. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

M_MIN_PERIMETER_BOX		Sets whether to calculate all the minimum-perimeter bounding box features. You can retrieve results for these features using MblobGetResult() with M_MIN_PERIMETER_BOX_AREA, M_MIN_PERIMETER_BOX_ANGLE, M_MIN_PERIMETER_BOX_CENTER_X, M_MIN_PERIMETER_BOX_CENTER_Y, M_MIN_PERIMETER_BOX_HEIGHT, M_MIN_PERIMETER_BOX_PERIMETER, M_MIN_PERIMETER_BOX_WIDTH, M_MIN_PERIMETER_BOX_Xn, and M_MIN_PERIMETER_BOX_Yn. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_HOLES		Sets whether to calculate the number of holes in a blob. Holes that intersect the edge of the image are not counted (they might not be holes). This value is equal to 1 - M_EULER_NUMBER and is therefore a true hole count only in M_INDIVIDUAL processing mode. You can retrieve results for this feature using MblobGetResult() with M_NUMBER_OF_HOLES. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_PERIMETER		Sets whether to calculate the total length of edges in a blob (including the edges of any holes), with an allowance made for the staircase effect that is produced when diagonal edges are digitized (inside corners are counted as 1.414, rather than 2.0). A single pixel blob (area = 1) has a perimeter of 4.0. You can retrieve results for this feature using MblobGetResult() with M_PERIMETER. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

M_RECTANGULARITY		Sets whether to calculate the degree to which a blob resembles a rectangle. You can retrieve results for this feature using MblobGetResult() with M_RECTANGULARITY. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROUGHNESS		Sets whether to calculate the roughness and irregularity of a blob, and is equal to M_PERIMETER / M_CONVEX_PERIMETER. You can retrieve results for this feature using MblobGetResult() with M_ROUGHNESS. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_RUNS		Sets whether to calculate the blob run-length encoding information. You can retrieve results for these features using MblobGetResult() with M_RUN_X, M_RUN_Y, M_RUN_LENGTH, M_NUMBER_OF_RUNS, and M_TOTAL_NUMBER_OF_RUNS. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

M_WORLD_BOX		Sets whether to calculate all of the features related to the world-axis-aligned bounding box. The world-axis-aligned bounding box is the bounding box that is aligned with the relative coordinate system's axes. You can retrieve results for these features using MblobGetResult() with M_WORLD_FERET_X, M_WORLD_FERET_Y, M_WORLD_BOX_X_MIN, M_WORLD_BOX_Y_MIN, M_WORLD_BOX_X_MAX, M_WORLD_BOX_Y_MAX, M_WORLD_X_AT_Y_MAX, M_WORLD_X_AT_Y_MIN, M_WORLD_Y_AT_X_MAX, and M_WORLD_Y_AT_X_MIN. In a case where more than one X-coordinate touches the minimum Y-coordinate of a blob, it is not possible to predict which coordinate will be calculated and returned. This also applies to a Y-coordinate touching the minimum X-coordinate of a blob. These features are actually calculated in the world coordinate system. If enabled for calculation, you should pass MblobCalculate() a calibrated blob identifier image. If an uncalibrated image is passed to MblobCalculate(), these features are the equivalent of the M_BOX feature group. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

The following control types establish which of the features, that have only a grayscale definition, are calculated for each blob. These blob features can only be calculated if you pass MblobCalculate() both a blob identifier image and a grayscale buffer (or a grayscale buffer with an ROI that acts as the blob identifier image). For the following control types, you must set the ContextOrResultBlobId parameter to a blob analysis context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying to calculate features with only a grayscale definition																		
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_BLOB_CONTRAST		Sets whether to calculate the difference between the maximum and minimum pixel values of a blob. You can retrieve results for this feature using MblobGetResult() with M_BLOB_CONTRAST. INQ															
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_PIXEL		Sets whether to calculate the maximum pixel value in a blob. You can retrieve results for this feature using MblobGetResult() with M_MAX_PIXEL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEAN_PIXEL		Sets whether to calculate the mean pixel value in a blob. You can retrieve results for this feature using MblobGetResult() with M_MEAN_PIXEL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_PIXEL		Sets whether to calculate the minimum pixel value in a blob. You can retrieve results for this feature using MblobGetResult() with M_MIN_PIXEL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGMA_PIXEL		Sets whether to calculate the standard deviation of pixel values in a blob. You can retrieve results for this feature using MblobGetResult() with M_SIGMA_PIXEL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

M_SUM_PIXEL		Sets whether to calculate the sum of all pixel values in a blob. You can retrieve results for this feature using MblobGetResult() with M_SUM_PIXEL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUM_PIXEL_SQUARED		Sets whether to calculate the sum of the squares of each pixel value in a blob. You can retrieve results for this feature using MblobGetResult() with M_SUM_PIXEL_SQUARED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

The following control types establish which of the features, that have both a binary and a grayscale definition, are calculated for each blob. The binary definition considers all pixels equal; the grayscale definition weighs pixels by their value in the grayscale buffer (the grayscale definition is much slower to calculate).

When you do not specify a combination value for the following control types, both versions of the feature (or groups of features) are enabled for calculation by default. If you do not provide a grayscale buffer to MblobCalculate(), only the binary version can be calculated. Specifying a combination value for the following control types will overwrite the settings that were set when no combination value was specified. If a combination value was initially specified for a control type, and a different setting was specified for the same control type without a combination value, it will also overwrite the previous setting. For example, if you specify M_MOMENT_GENERAL with M_ENABLE, both the binary and grayscale versions of the feature will be calculated. If you subsequently specify M_MOMENT_GENERAL + M_GRAYSCALE with M_DISABLE, only the binary version of the feature will be calculated.

For the following control types, you must set the ContextOrResultBlobId parameter to a blob analysis context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying to calculate features with two definitions (binary and grayscale)																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_CENTER_OF_GRAVITY +		Sets whether to calculate both X- and Y-coordinates of the center of gravity. You can retrieve results for these features using MblobGetResult() with M_CENTER_OF_GRAVITY_X, and M_CENTER_OF_GRAVITY_Y. INQ															

	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_AT_PRINCIPAL_AXIS_ANGLE +		Sets whether to calculate the Feret diameter at the principal axis of a blob. Use the M_NUMBER_OF_FERETS control type to set the number of Feret angles that are evaluated. You can retrieve results for this feature using MblobGetResult() with M_FERET_AT_PRINCIPAL_AXIS_ANGLE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_AT_SECONDARY_AXIS_ANGLE +		Sets whether to calculate the Feret diameter at the secondary axis of a blob. Use the M_NUMBER_OF_FERETS control type to set the number of Feret angles that are evaluated. You can retrieve results for this feature using MblobGetResult() with M_FERET_AT_SECONDARY_AXIS_ANGLE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

M_FERET_PRINCIPAL_AXIS_ELONGATION +		Sets whether to calculate the ratio between the Feret diameter at the principal axis and the Feret diameter at the secondary axis. Use the M_NUMBER_OF_FERETS control type to set the number of Feret angles that are evaluated. You can retrieve results for this feature using MblobGetResult() with M_FERET_PRINCIPAL_AXIS_ELONGATION. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_FIRST_ORDER +		Sets whether to calculate the first-order moments where the order of either the X- or Y-component equals 0 and the order of the other component equals 1. You can retrieve results for these features using MblobGetResult() with M_MOMENT_X0_Y1, and M_MOMENT_X1_Y0. To calculate higher order moments, use M_MOMENT_SECOND_ORDER for predefined second-order moments, M_MOMENT_THIRD_ORDER for predefined third-order moments, or M_MOMENT_GENERAL for user-specified moments. Note that first-order moment features are calculated faster when using M_MOMENT_FIRST_ORDER than when using M_MOMENT_GENERAL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

M_MOMENT_GENERAL +		Sets whether to calculate a user-specified moment. Use the M_MOMENT_GENERAL_MODE, M_MOMENT_GENERAL_ORDER_X, and M_MOMENT_GENERAL_ORDER_Y control types to specify the settings of the user-specified moment. You can retrieve results for this feature using MblobGetResult() with M_MOMENT_GENERAL. You could alternatively enable the calculation of M_MOMENT_FIRST_ORDER, M_MOMENT_SECOND_ORDER, or M_MOMENT_THIRD_ORDER to calculate a set of first, second, or third-order moments, respectively. Note that first and second-order moment features are calculated faster when using M_MOMENT_FIRST_ORDER or M_MOMENT_SECOND_ORDER than when using M_MOMENT_GENERAL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_SECOND_ORDER +		Sets whether to calculate the second-order moments. You can retrieve results for these features using MblobGetResult() with M_MOMENT_X0_Y2, M_MOMENT_X1_Y1, M_MOMENT_X2_Y0, M_MOMENT_CENTRAL_X0_Y2, M_MOMENT_CENTRAL_X1_Y1, M_MOMENT_CENTRAL_X2_Y0, M_AXIS_PRINCIPAL_ANGLE, and M_AXIS_SECONDARY_ANGLE. To calculate higher order moments, use M_MOMENT_THIRD_ORDER for predefined third-order moments, or M_MOMENT_GENERAL for user-specified moments. Note that second-order moment features are calculated faster when using M_MOMENT_SECOND_ORDER than when using M_MOMENT_GENERAL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_THIRD_ORDER +		Sets whether to calculate the third-order moments. Calculating these moments makes the Hu moment invariants available. You can retrieve results for these features using MblobGetResult() with M_MOMENT_X1_Y2, M_MOMENT_X3_Y0, M_MOMENT_X0_Y3, M_MOMENT_CENTRAL_X1_Y2, M_MOMENT_CENTRAL_X2_Y1, M_MOMENT_CENTRAL_X3_Y0, M_MOMENT_CENTRAL_X0_Y3, and M_MOMENT_HU_n, where *n* ranges from 1 to 7. To calculate higher order moments, use M_MOMENT_GENERAL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For specifying to calculate features with two definitions (binary and grayscale); and for the following values: [when ControlType=M_SORTn: M_AXIS_PRINCIPAL_ANGLE; M_AXIS_SECONDARY_ANGLE; M_CENTER_OF_GRAVITY_X; M_CENTER_OF_GRAVITY_Y; M_FERET_AT_PRINCIPAL_AXIS_ANGLE; M_FERET_AT_SECONDARY_AXIS_ANGLE; M_FERET_PRINCIPAL_AXIS_ELONGATION; M_MOMENT_CENTRAL_X0_Y2; M_MOMENT_CENTRAL_X0_Y3; M_MOMENT_CENTRAL_X1_Y1; M_MOMENT_CENTRAL_X1_Y2; M_MOMENT_CENTRAL_X2_Y0; M_MOMENT_CENTRAL_X2_Y1; M_MOMENT_CENTRAL_X3_Y0; M_MOMENT_GENERAL; M_MOMENT_HU_n; M_MOMENT_X0_Y1; M_MOMENT_X0_Y2; M_MOMENT_X0_Y3; M_MOMENT_X1_Y0; M_MOMENT_X1_Y1; M_MOMENT_X1_Y2; M_MOMENT_X2_Y0; M_MOMENT_X2_Y1; M_MOMENT_X3_Y0].

You can add one of the following values to the above-mentioned values to set whether to enable, disable, or sort on only the binary or grayscale calculation of the feature. Note that this combination is essential for some values. Refer to the individual values to determine if this is the case.

For control values of the M_SORTn control type, this combination is essential (that is, you must add one of these combination values to the control value).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For feature parameters that have two definitions																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_BINARY	Selects the binary definition of the selected feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAYSCALE	Selects the grayscale definition of the selected feature.	c		e	f	g	a		i		j	k	l	m	n	b

The following control types establish which of the features, that are related to depth maps, are calculated for each blob. These blob features can only be calculated if you pass MblobCalculate() a grayscale buffer that is a fully corrected depth map image buffer or 3D-processable depth map container, and has an ROI that acts as the blob identifier image as well as identifies invalid pixels. If you passed an image buffer, you can verify that it contains a fully corrected depth map using McalInquire() with M_DEPTH_MAP. You can use MbufSetRegion() with M_RASTERIZE_DEPTH_MAP_VALID_PIXELS to merge confidence information with the blob identifier image. If you passed a container, you can use MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP to ensure that the container contains a 3D-processable depth map. For the following control types, you must set the ContextOrResultBlobId parameter to a blob analysis context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying to calculate features related to depth maps																		
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																	
M_DEPTH_MAP_MAX_ELEVATION		Sets whether to calculate the maximum elevation of a blob. INQ																
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that the maximum elevation of the blob will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that the maximum elevation of the blob will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEPTH_MAP_MEAN_ELEVATION		Sets whether to calculate the mean elevation of a blob. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that the mean elevation of the blob will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that the mean elevation of the blob will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEPTH_MAP_MIN_ELEVATION		Sets whether to calculate the minimum elevation of a blob. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies that the minimum elevation of the blob will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that the minimum elevation of the blob will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEPTH_MAP_SIZE_Z		Sets whether to calculate the Z-size (difference between maximum and minimum elevation) of a blob. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that the Z-size of the blob will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that the Z-size of the blob will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEPTH_MAP_VOLUME		Sets whether to calculate the volume of a blob. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that the volume of the blob will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that the volume of the blob will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

For the following control types, you must set the ContextOrResultBlobId parameter to a blob analysis result buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a blob analysis result buffer																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_INDEX_MODE		Sets the index mode to use when retrieving results using MblobGetResult() and when drawing results using MblobDraw(). INQ															
	M_DEFAULT	Same as M_USE_MACRO.	c		e	f	g	a		i		j	k	l	m	n	b

	M_INDEX	Specifies that the index will be passed directly to the LabelOrIndex parameter of MblobGetResult() or MblobDraw(), without any macros.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LABEL	Specifies that the label will be passed directly to the LabelOrIndex parameter of MblobGetResult() or MblobDraw(), without any macros.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USE_MACRO	Specifies that the values passed to the LabelOrIndex parameter of MblobGetResult() or MblobDraw() can be the macros M_BLOB_INDEX() or M_BLOB_LABEL(). Note that M_USE_MACRO cannot be used in M_LABELED_TOUCHING mode with a 32-bit blob identifier image.	c		e	f	g	a		i		j	k	l	m	n	b
M_INPUT_SELECT_UNITS		Sets which units to use with the CondLow and CondHigh parameters of the MblobSelect() function, when specifying the limit values of the blob selection condition. This essentially sets the input coordinate system to use. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_PIXEL	Specifies that the limit values passed to MblobSelect() be interpreted in pixel units, with respect to the pixel coordinate system. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD	Specifies that the limit values passed to MblobSelect() be interpreted in world units, with respect to the relative coordinate system. If world units are specified, calling MblobSelect() generates an error if the specified result buffer is not associated with a camera calibration context.	c		e	f	g	a		i		j	k	l	m	n	b

M_RESULT_OUTPUT_UNITS	Sets whether to return results in pixels or world units. This essentially sets the output coordinate system to use. The setting of this control type will only affect functions within this module which return positional results. This control type can be changed at any time to return results in the required output units. Note that when returned in world units, some results are only precise if calculated on a physically corrected image. If you require precise results in world units, you should correct the image using McalTransformImage() before calling MblobCalculate(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ACCORDING_TO_CALIBRATION.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_CALIBRATION	Specifies that results are returned in world units if the result was calculated on an image associated with a camera calibration context; otherwise, specifies that results are returned in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies that results are returned in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies that results are returned in world units, with respect to the relative coordinate system. If world units are specified, calling MblobGetResult() generates an error if the result was not calculated on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b
M_STOP_CALCULATE	Stops the current blob analysis calculation. You must call **MblobControl**() with M_STOP_CALCULATE from another thread (typically of higher priority). Results already available in the result buffer become invalid and will be discarded, unless M_RETURN_PARTIAL_RESULTS is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b

Type-specific versions of the function when using a C compiler under 64-bit

```
void MblobControlDouble (AIL_ID ContextOrResultBlobId, AIL_INT64 ControlType, AIL_DOUBLE ControlValue)
```

Details

Parameters

ContextOrResultBlobId

See ContextOrResultBlobId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

```
void MblobControlInt64 (AIL_ID ContextOrResultBlobId, AIL_INT64 ControlType, AIL_INT64 ControlValue)
```

Details

Parameters

ContextOrResultBlobId

See ContextOrResultBlobId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailblob11.lib.
DLL	Requires ail11.dll; ailblob11.dll.

MblobDraw

Synopsis

Draw specific blob features in the destination image buffer or 2D graphics list.

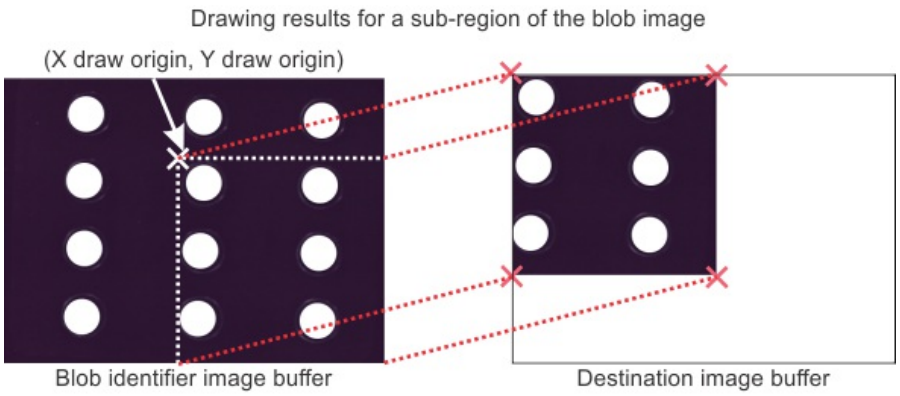
Syntax

```
void MblobDraw(
    AIL_ID ContextGraId,           //in
    AIL_ID ResultBlobId,           //in
    AIL_ID DstImageBufOrListGraId, //out
    AIL_INT64 Operation,           //in
    AIL_INT LabelOrIndex,          //in
    AIL_INT64 ControlFlag          //in
)
```

Description

This function draws specific blob features, which were calculated using MblobCalculate(), in the destination image buffer or 2D graphics list. This function can draw multiple features at a time.

You can draw results obtained relative to an offset, at the top-left corner of the destination image, using MgraControl() with M_DRAW_OFFSET_X and M_DRAW_OFFSET_Y and zoom them using MgraControl() with M_DRAW_ZOOM_X and M_DRAW_ZOOM_Y. For example, you can draw a zoomed section of the blobs found in the target image at the top-left corner of the destination image. For more information, see the Drawing graphics with offset and zoom subsection of the Drawing graphics section of Chapter 26: Generating graphics.



Parameters

ContextGraId

Specifies the identifier of the 2D graphics context to use when drawing. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the 2D graphics context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT	Specifies that the default 2D graphics context of the current Aurora Imaging Library application is used. > Note that there is a different default 2D graphics context for each thread.	c		e	f	g	a		i		j	k	l	m	n	b
2D graphics context identifier	Specifies a valid 2D graphics context identifier, which you have allocated using MgraAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ResultBlobId

Specifies the identifier of the blob analysis result buffer from which to retrieve the features to draw.

DstImageBufOrListGraId

Specifies the identifier of the destination image buffer or 2D graphics list in which to draw. The buffer can be any valid 1- or 3-band image buffer allocated using MbufAlloc...(). The 2D graphics list must be previously allocated using MgraAllocList(). By drawing into a display's overlay buffer or associating the 2D graphics list with the display, you can also annotate an image non-destructively.

If specifying an image buffer, it must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

Operation

Specifies the type of operation to perform.

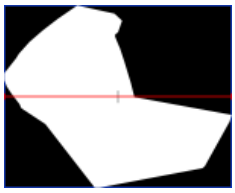
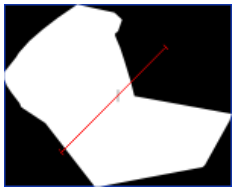
The values in the tables below can be added together to draw multiple features at a time. For example, to draw the contour, holes, and position of the blobs, you would specify M_DRAW_BLOBS_CONTOUR + M_DRAW_HOLES + M_DRAW_POSITION.

The following operations can draw in a destination image buffer or 2D graphics list.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of operation																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_AXIS	Draws a cross at the blobs' center of gravity, respecting the principal and secondary axes. Note that to perform this operation, one feature among M_AXIS_PRINCIPAL_ANGLE or M_AXIS_SECONDARY_ANGLE must have been calculated using the binary definition.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_BLOBS	Draws the blobs. Note that to perform this operation, runs must have been previously saved using MblobControl() with M_SAVE_RUNS set to M_ENABLE. When drawing in either an image buffer or a 2D graphics list, MgraControl() with M_INPUT_UNITS cannot be set to M_WORLD for this operation.	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_BLOBS_CONTOUR	Draws the external outline of the blobs. Note that to perform this operation, runs must have been previously saved using MblobControl() with M_SAVE_RUNS set to M_ENABLE. Also note that when using MblobControl() with M_BLOB_IDENTIFICATION_MODE set to either M_WHOLE_IMAGE or M_LABELED, both the blob and hole contours will be drawn when performing this operation. If M_BLOB_IDENTIFICATION_MODE is set to M_LABELED_TOUCHING, you cannot specify M_DRAW_BLOBS_CONTOUR. When drawing in either an image buffer or a 2D graphics list, MgraControl() with M_INPUT_UNITS cannot be set to M_WORLD for this operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_BOX	Draws the image-axis aligned bounding box of each blob. The image-axis-aligned bounding box is the bounding box that is aligned with the pixel coordinate system's axes. Note that to perform this operation, the M_BOX feature must have been calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_BOX_CENTER	Draws a cross at the center of the blobs' bounding box. Note that to perform this operation, the M_BOX feature must have been calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_CENTER_OF_GRAVITY	Same as M_DRAW_POSITION.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_CONVEX_HULL	Draws the convex hull of the blobs. Note that to perform this operation, the M_CONVEX_HULL feature must have been calculated, and runs must have been previously saved using MblobControl() with M_SAVE_RUNS set to M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_CONVEX_HULL_CONTOUR	Draws the convex perimeter of the blobs. Note that to perform this operation, the M_CONVEX_HULL feature must have been calculated, and runs must have been previously saved using MblobControl() with M_SAVE_RUNS set to M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_FERET_MAX	Draws the blobs' maximum Feret diameter, using an H-type line (|-|) that passes through the center of the blob's bounding box at the maximum Feret diameter's angle. The line's end points are tangent to the Feret diameter's contact points.  Note that to perform this operation, the following features must have been calculated: M_BOX, M_FERET_MAX_DIAMETER, and M_FERET_MAX_ANGLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_FERET_MIN	Draws the blobs' minimum Feret diameter, using an H-type line (|-|) that passes through the center of the blob's bounding box at the minimum Feret diameter's angle. The line's end points are tangent to the Feret diameter's contact points.  Note that to perform this operation, the following features must have been calculated: M_BOX, M_FERET_MIN_DIAMETER, and M_FERET_MIN_ANGLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_HOLES	Draws the holes of the blobs. Note that to perform this operation, runs must have been previously saved using MblobControl() with M_SAVE_RUNS set to M_ENABLE and one feature among M_BOX_X_MIN, M_BOX_X_MAX, M_BOX_Y_MIN, or M_BOX_Y_MAX must have been calculated. When drawing in either an image buffer or a 2D graphics list, MgraControl() with M_INPUT_UNITS cannot be set to M_WORLD for this operation.	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_HOLES_CONTOUR	Draws the outline of the blobs' holes. Note that to perform this operation, runs must have been previously saved using MblobControl() with M_SAVE_RUNS set to M_ENABLE. Also note that when using MblobControl() with M_BLOB_IDENTIFICATION_MODE set to either M_WHOLE_IMAGE or M_LABELED, both the blob and hole contours will be drawn when performing this operation. If M_BLOB_IDENTIFICATION_MODE is set to M_LABELED_TOUCHING, you cannot draw the blobs' holes with M_DRAW_HOLES_CONTOUR. When drawing in either an image buffer or a 2D graphics list, MgraControl() with M_INPUT_UNITS cannot be set to M_WORLD for this operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_MIN_AREA_BOX	Draws the blobs' minimum-area bounding box. Note that to perform this operation, at least one of the M_MIN_AREA_BOX features must have been calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_MIN_PERIMETER_BOX	Draws the blobs' minimum-perimeter bounding box. Note that to perform this operation, at least one of the M_MIN_PERIMETER_BOX features must have been calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_POSITION	Draws a cross at the center of gravity of the blobs. Note that to perform this operation, M_CENTER_OF_GRAVITY must have been calculated using the binary definition.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_WORLD_BOX	Draws the world-axis aligned bounding box of each blob. The world-axis-aligned bounding box is the bounding box that is aligned with the relative coordinate system's axes. Note that to perform this operation, the M_WORLD_BOX features must have been calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_WORLD_BOX_CENTER	Draws a cross at the center of the blobs' bounding box, calculated in the relative coordinate system. Note that to perform this operation, the M_WORLD_BOX features must have been calculated.	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_WORLD_FERET_X	Draws the blobs' Feret diameter, using an H-type line (|-|), centered at the blobs' position, parallel to the X-axis of the relative coordinate system. Note that to perform this operation, the M_WORLD_FERET_X feature must have been calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_WORLD_FERET_Y	Draws the blobs' Feret diameter, using an H-type line (|-|), centered at the blobs' position, parallel to the Y-axis of the relative coordinate system. Note that to perform this operation, the M_WORLD_FERET_Y feature must have been calculated.	c		e	f	g	a		i		j	k	l	m	n	b

LabelOrIndex

Specifies the label or index of the blob or blobs whose information to draw.

This parameter should be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the label of the blob or blobs to draw																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_ALL_BLOBS.	c		e	f	g	a		i		j	k	l	m	n	b
M_BLOB_INDEX(AIL_INT Index)	Specifies the index of the blob.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Index Specifies the index. Valid indices range from 0 (the first blob) to the number of blobs (M_NUMBER) minus 1.															
M_BLOB_LABEL(AIL_INT Label)	Specifies the label of the blob for which to get results. You can get a list of valid blob label values with M_LABEL_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Label Specifies the label.															
M_ALL_BLOBS	Specifies all blobs.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXCLUDED_BLOBS	Specifies all currently excluded blobs.	c		e	f	g	a		i		j	k	l	m	n	b

M_INCLUDED_BLOBS	Specifies all currently included blobs.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailblob11.lib.
DLL	Requires ail11.dll; ailblob11.dll.

MblobFree

Synopsis

Free the blob analysis context or result buffer.

Syntax

```
void MblobFree(
    AIL_ID ContextOrResultBlobId //in
)
```

Description

This function deletes the specified blob analysis context or result buffer and releases any memory associated with it.

All blob contexts and all blob result buffers allocated on a particular system must be freed before the system can be freed, unless M_UNIQUE_ID was specified during allocation.

Parameter

ContextOrResultBlobId

Specifies the identifier of the blob analysis context or result buffer to free. The buffer must have been successfully allocated, using MblobAllocResult() or MblobAlloc(), prior to calling this function.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailblob11.lib.
DLL	Requires ail11.dll; ailblob11.dll.

MblobGetLabel

Synopsis

Get the label value of a blob at a specified position.

Syntax

```
AIL_INT MblobGetLabel(
    AIL_ID ResultBlobId, //in
    AIL_INT XPos,        //in
    AIL_INT YPos,        //in
    AIL_INT *LabelVarPtr //out
)
```

Description

This function gets the label value of a specified blob. Label values are used, for example, to obtain calculation results for single blobs. Blob label values must have been generated by calling MblobCalculate().

Parameters

ResultBlobId

Specifies the identifier of the blob analysis result buffer where the labels are stored. Note, this function cannot obtain the label values of blobs that have been deleted from the result buffer with MblobSelect().

XPos

Specifies the X-coordinate of the blob.

YPos

Specifies the Y-coordinate of the blob.

LabelVarPtr

Specifies the address of the variable in which to write the label value. If there is no blob at the specified location, the blob has been deleted, or if the blob's XPos and YPos lie outside the original identifier image, **M_NULL** is returned instead of the label value. Also, since **MblobGetLabel**() returns the label value, you can set this parameter to **M_NULL**.

Return value

The returned value is the label value of the specified blob.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailblob11.lib.
DLL	Requires ail11.dll; ailblob11.dll.

MblobGetResult

Synopsis

Get results for a feature of the included blobs, from a blob result buffer.

Syntax

```
void MblobGetResult(
    AIL_ID ResultBlobId,  //in
    AIL_INT LabelOrIndex, //in
    AIL_INT64 ResultType, //in
    void *ResultArrayPtr  //out
)
```

Description

This function retrieves the results for a specified feature of one or all included blobs, from the specified blob analysis result buffer.

If your target image was associated with a camera calibration context, positional and dimensional results are, by default, returned with respect to the relative coordinate system of the image. Otherwise, these results are returned in pixels, relative to the top-left pixel in the target image.

> If your target image was associated with a camera calibration context but you want to retrieve positional and dimensional results in pixel units, use MblobControl() with the M_RESULT_OUTPUT_UNITS control type set to M_PIXEL. Note that if you set M_RESULT_OUTPUT_UNITS to M_WORLD without having specified a calibrated image from which to calculate the results, **MblobGetResult**() will generate an error. Also note that when using world units, some results are only precise if calculated on a physically corrected image. If you require precise results in world units, you should correct the image using McalTransformImage() before calling MblobCalculate().

When calculated in pixel units, the pixel aspect ratio, specified using MblobControl() with M_PIXEL_ASPECT_RATIO, is taken into account horizontally. Results are returned in units of "pixel height" since the pixel width is adjusted to be equal to the pixel height.

Note that **MblobGetResult**() generates an error if you try to retrieve a result whose corresponding feature, or the group of features to which it belongs, was not enabled for calculation. You can add the M_AVAILABLE combination constant to a feature to determine if the result is available for retrieval.

Parameters

ResultBlobId

Specifies the identifier of the blob analysis result buffer from which to get results. The specified feature(s) must have already been calculated with MblobCalculate().

LabelOrIndex

Specifies the label or index of the blob for which to get results, or specifies that you are getting general results related to all blobs. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the label or index of the blob																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT	Same as M_INCLUDED_BLOBS for features that apply to one or all occurrences. Same as M_GENERAL for general result types.	c		e	f	g	a		i		j	k	l	m	n	b
M_BLOB_INDEX(AIL_INT Index)	Specifies the index of the blob for which to get results. You can get the number of included blobs in the result buffer using M_NUMBER.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Index Specifies the index.															
	0 <= *Value* < M_NUMBER Specifies the valid indices of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_BLOB_LABEL(AIL_INT Label)	Specifies the label of the blob for which to get results. You can get a list of valid blob label values with M_LABEL_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Label Specifies the label.															
M_GENERAL	Retrieves a result relating to the entire blob result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_INCLUDED_BLOBS	Retrieves all included blob results.	c		e	f	g	a		i		j	k	l	m	n	b

ResultType

Specifies the type of result to retrieve.

See the Parameter associations section for possible values that can be specified.

ResultArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- AIL_DOUBLE
- AIL_FLOAT
- AIL_INT
- AIL_INT16

- AIL_INT32
- AIL_INT64

Specifies the address of the array in which to write results.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **ResultType** parameter and possible values returned to the **ResultArrayPtr** parameter.

- For retrieving general results from the blob result buffer
- For retrieving results of a feature with a binary definition, and that cannot be used as a sorting key
- For retrieving results of a feature with a binary definition, and that can be used as a sorting key
- For retrieving results for a feature with a grayscale definition, and that can be used as a sorting key
- For retrieving results for a result of a feature that has two different definitions, and that can be used as a sorting key
- For retrieving results of a depth map specific feature, and that can be used as a sorting key

To retrieve a general result, the ResultType parameter should be set to one of the following values. In this case, you must set LabelOrIndex to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving general results from the blob result buffer																	
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ResultArrayPtr - Possible values returned																
M_BLOB_IDENTIFICATION_MODE +		Retrieves whether features were calculated for each blob, or for groups of blobs treated as single blobs.															
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_INDIVIDUAL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INDIVIDUAL	Specifies that all blobs were measured individually.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LABELED	Specifies that blobs with the same label were grouped together, and that touching blobs with different labels were also grouped together.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LABELED_TOUCHING	Specifies that blobs with the same label were grouped together, and that touching blobs with different labels were measured individually.	c		e	f	g	a		i		j	k	l	m	n	b

	M_WHOLE_IMAGE	Specifies that all blobs were grouped together.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALCULATION_TYPE +		Retrieves the type of calculation that was performed during the last call to MblobCalculate(), based on the types of images that were passed (that is, only a binary image, or both a binary and grayscale image).	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_BINARY	Specifies that the previous call to MblobCalculate() was only performed on a binary image, and that enabled features with a binary definition are available for retrieval.	c		e	f	g	a		i		j	k	l	m	n	b
	M_BINARY_AND_GRAYSCALE	Specifies that the previous call to MblobCalculate() was performed on both a binary and a grayscale image, and that enabled feature results with a binary and grayscale definition are available for retrieval.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NOT_CALCULATED	Specifies that MblobCalculate() has not been called, and that feature results are not available for retrieval.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_BLOBS_END +		Retrieves whether the maximum number of blobs was reached. You can use MblobControl() with M_MAX_BLOBS to set the limit.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the maximum number of blobs was not reached.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the maximum number of blobs was reached.	c		e	f	g	a		i		j	k	l	m	n	b

M_MAX_LABEL_VALUE +	Retrieves the maximum label value of all blobs in the result buffer, regardless of included or excluded status. Label values for blobs are generated when a call to MblobCalculate() is made. The maximum label value is not necessarily the same as the total number of blobs because some label values might not be used, depending on the blob's shape. One of the uses for obtaining the maximum label value is to determine if an 8 or 16-bit image buffer is needed for MblobLabel(). Note that if zero is returned, no blobs were found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER +	Retrieves the number of currently included blobs. All blobs are included unless their status is changed using MblobSelect(). Included blobs will be included in future operations and result retrievals.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TIMEOUT_END +	Retrieves whether the timeout limit was reached. You can set the timeout limit using MblobControl() with M_TIMEOUT.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the timeout limit was not reached.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the timeout limit was reached.	c		e	f	g	a		i		j	k	l	m	n	b

M_TOTAL_NUMBER_OF_CHAINED_PIXELS +	Retrieves the total number of chained pixels in all blobs in the target image. The calculation of this result type can be enabled using MblobControl() with M_CHAINS.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TOTAL_NUMBER_OF_CONVEX_HULL_POINTS +	Retrieves the total number of convex hull points in all blobs in the target image. The calculation of this result type can be enabled using MblobControl() with M_CONVEX_HULL.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TOTAL_NUMBER_OF_FERETS +	Retrieves the total number of Ferets for all blobs in the target image. The calculation of this result type can be enabled using MblobControl() with M_FERETS.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TOTAL_NUMBER_OF_RUNS +	Retrieves the total number of runs in all blobs in the target image. The calculation of this result type can be enabled using MblobControl() with M_RUNS.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve one or more results for a feature that has only a binary definition and cannot be used as a sorting key, the ResultType parameter should be set to one of the following values. In this case, you must set LabelOrIndex to a specific blob, or all blobs, using M_BLOB_INDEX() or M_BLOB_LABEL() for individual blobs, or M_INCLUDED_BLOBS for all blobs.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving results of a feature with a binary definition, and that cannot be used as a sorting key																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_BLOB_INCLUSION_STATE +	Retrieves whether each blob is included in calculations and retrieval of results.															

ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_EXCLUDED	Specifies that the blob is not included in calculations and retrieval of results.	c		e	f	g	a		i		j	k	l	m	n	b
M_INCLUDED	Specifies that the blob is included in calculations and retrieval of results.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAIN_INDEX +	Retrieves the indices which indicate the chain to which each chained pixel belongs. Each blob's border chain is identified as index 1. Chains that delimit holes in the blob are identified by subsequent indices. When retrieving the chain index of each chained pixel for all included blobs in the image, you can use M_NUMBER_OF_CHAINED_PIXELS to determine the number of chained pixels in each blob, and therefore determine to which blob each chained pixel belongs. The calculation of this feature can be enabled using MblobControl() with M_CHAINS.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAIN_X +	Retrieves the X-coordinate of each chained pixel in the specified blob, for all chains contained within the blob (both the pixels bordering a blob and those delimiting its holes). Chained pixels always form a closed chain. This implies that the starting pixel in the chain is also the closing one. If your blob has regions which are 1 pixel wide, these pixels are chained twice, once in the forward direction and then in the opposite direction. The calculation of this feature can be enabled using MblobControl() with M_CHAINS. Blobs with holes have multiple chains. To determine the chain to which a pixel's X-coordinate belongs, use M_CHAIN_INDEX. Results are only available in pixel units. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_CHAIN_Y +	Retrieves the Y-coordinate of each chained pixel in the specified blob, for all chains contained within the blob (both the pixels bordering a blob and those delimiting its holes). Chained pixels always form a closed chain. This implies that the starting pixel in the chain is also the closing one. If your blob has regions which are 1 pixel wide, these pixels are chained twice, once in the forward direction and then in the opposite direction. The calculation of this feature can be enabled using MblobControl() with M_CHAINS. Blobs with holes have multiple chains. To determine the chain to which a pixel's Y-coordinate belongs, use M_CHAIN_INDEX. Results are only available in pixel units. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CONVEX_HULL_X +	Retrieves the X-coordinate of each point on the convex perimeter of the specified blob. Convex perimeter pixels always form a closed figure. This implies that the starting pixel on the convex perimeter is also the closing one. The calculation of this feature can be enabled using MblobControl() with M_CONVEX_HULL. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_CONVEX_HULL_XY_PACKED +	Retrieves the packed coordinates (X, Y) of each point on the perimeter of the specified blob. Convex perimeter pixels always form a closed figure. This implies that the starting pixel on the convex perimeter is also the closing one. The calculation of this feature can be enabled using MblobControl() with M_CONVEX_HULL. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CONVEX_HULL_Y +	Retrieves the Y-coordinate of each point on the convex perimeter of the specified blob. Convex perimeter pixels always form a closed figure. This implies that the starting pixel on the convex perimeter is also the closing one. The calculation of this feature can be enabled using MblobControl() with M_CONVEX_HULL. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_DIAMETERS +	Retrieves all the Feret diameters of the specified blob in the image. This feature is not available if you have previously set M_NUMBER_OF_FERETS to M_INFINITE using MblobControl(). The calculation of this feature can be enabled using MblobControl() with M_FERETS. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b

	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_INDEX_VALUE +		Retrieves the index value for the specified blob(s) in the image. Note that when blob's are excluded or deleted, the index of blobs with indices greater than that of the excluded/deleted blob are reduced by one. Therefore, if you need to specify a specific blob, you might want to use its blob label instead since it doesn't change.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_AREA_BOX_X*n* +		Retrieves the X-coordinate of the *n* th vertex of the minimum-area bounding box of each blob, where *n* stands for an integer between 1 and 4. The calculation of this feature can be enabled using MblobControl() with M_MIN_AREA_BOX. Note that the labeling of the vertices might be inconsistent from one blob to another within the blob identifier image. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_MIN_AREA_BOX_Y*n* +	Retrieves the Y-coordinate of the *n* th vertex of the minimum-area bounding box of each blob, where *n* stands for an integer between 1 and 4. The calculation of this feature can be enabled using MblobControl() with M_MIN_AREA_BOX. Note that the labeling of the vertices might be inconsistent from one blob to another within the blob identifier image. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_PERIMETER_BOX_X*n* +	Retrieves the X-coordinate of the *n* th vertex of the minimum-perimeter bounding box of each blob, where *n* stands for an integer between 1 and 4. The calculation of this feature can be enabled using MblobControl() with M_MIN_PERIMETER_BOX. Note that the labeling of the vertices might be inconsistent from one blob to another within the blob identifier image. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_MIN_PERIMETER_BOX_Y*n* +	Retrieves the Y-coordinate of the *n* th vertex of the minimum-perimeter bounding box of each blob, where *n* stands for an integer between 1 and 4. The calculation of this feature can be enabled using MblobControl() with M_MIN_PERIMETER_BOX. Note that the labeling of the vertices might be inconsistent from one blob to another within the blob identifier image. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_FERETS +	Retrieves the number of Feret diameters calculated for each blob. The calculation of this feature can be enabled using MblobControl() with M_FERETS.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_RUN_LENGTH +	Retrieves the length of each run in the specified blob(s). A run is defined as a horizontal series of consecutive foreground pixels. When retrieving the length of each run for all included blobs in the image, you can use M_NUMBER_OF_RUNS to determine the number of runs each blob has, and therefore determine to which blob each length corresponds. The calculation of this feature can be enabled using MblobControl() with M_RUNS.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_RUN_X +	Retrieves the X-coordinate of the start (left-most pixel) of each run in the specified blob(s). The calculation of this feature can be enabled using MblobControl() with M_RUNS.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_RUN_Y +	Retrieves the Y-coordinate of the start (left-most pixel) of each run in the specified blob(s). The calculation of this feature can be enabled using MblobControl() with M_RUNS.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve one or more results for a feature that has only a binary definition and can be used as a sorting key, the ResultType parameter should be set to one of the following values. In this case, you must set LabelOrIndex to a specific blob, or all blobs, using M_BLOB_INDEX() or M_BLOB_LABEL() for individual blobs, or M_INCLUDED_BLOBS for all blobs. Unless otherwise stated, the following features are calculated in the pixel coordinate system.

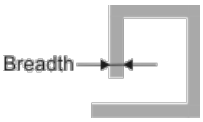
Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving results of a feature with a binary definition, and that can be used as a sorting key																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																	
M_AREA +	Retrieves the number of foreground pixels in each blob (holes are not counted). This feature is always calculated. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().																
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_BLOB_TOUCHING_IMAGE_BORDERS +	Retrieves whether each blob is touching the borders of the image. The calculation of this feature can be enabled using MblobControl() with M_BOX.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_NO	Specifies that the blob is not touching the borders of the image.	c		e	f	g	a		i		j	k	l	m	n	b
M_YES	Specifies that the blob is touching one or more borders of the image.	c		e	f	g	a		i		j	k	l	m	n	b

M_BOX_AREA +	Retrieves the area covered by the image-axis-aligned bounding box of each blob. The image-axis-aligned bounding box is the bounding box that is aligned with the pixel coordinate system's axes. The calculation of this feature can be enabled using MblobControl() with M_BOX. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_ASPECT_RATIO +	Retrieves the ratio between the horizontal size and the vertical size of the image-axis-aligned bounding box of each blob. The image-axis-aligned bounding box is the bounding box that is aligned with the pixel coordinate system's axes. The calculation of this feature can be enabled using MblobControl() with M_BOX. Even if you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS, this result is calculated and returned in the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_FILL_RATIO +	Retrieves the ratio between the area of the blob and the area of the image-axis-aligned bounding box of each blob. The image-axis-aligned bounding box is the bounding box that is aligned with the pixel coordinate system's axes. The calculation of this feature can be enabled using MblobControl() with M_BOX. Even if you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS, this result is calculated and returned in the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

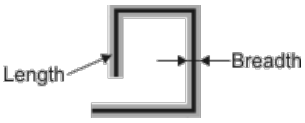
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_X_MAX +		Retrieves the extreme right X-coordinate of each blob, of the image-axis-aligned bounding box. The image-axis-aligned bounding box is the bounding box that is aligned with the pixel coordinate system's axes. The calculation of this feature can be enabled using MblobControl() with M_BOX. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_X_MIN +		Retrieves the extreme left X-coordinate of each blob, of the image-axis-aligned bounding box. The image-axis-aligned bounding box is the bounding box that is aligned with the pixel coordinate system's axes. The calculation of this feature can be enabled using MblobControl() with M_BOX. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

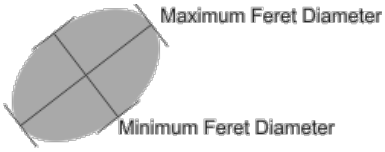
M_BOX_Y_MAX +	Retrieves the extreme bottom Y-coordinate of each blob, of the image-axis-aligned bounding box. The image-axis-aligned bounding box is the bounding box that is aligned with the pixel coordinate system's axes. The calculation of this feature can be enabled using MblobControl() with M_BOX. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_Y_MIN +	Retrieves the extreme top Y-coordinate of each blob, of the image-axis-aligned bounding box. The image-axis-aligned bounding box is the bounding box that is aligned with the pixel coordinate system's axes. The calculation of this feature can be enabled using MblobControl() with M_BOX. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

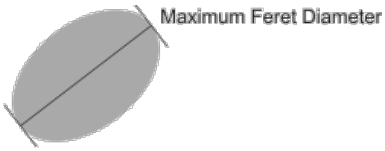
M_BREADTH +	Retrieves the breadth of each blob. This feature is only accurate for certain blob types (for example, long thin blobs) because it is derived from the perimeter (P) and area (A) assuming that P = 2(length + breadth) and A = length x breadth. The calculation of this feature can be enabled using MblobControl() with M_BREADTH.  If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPACTNESS +	Retrieves the compactness of each blob. This is the ratio between the area of a circle with the same perimeter as the blob in question, and the area of the blob itself. The minimum theoretical value of 1.0 is obtained if the blob is a perfect circle. In practice, the minimum obtainable value is slightly above 1. This is due to the effect of square pixel discretization. The more convoluted the shape, the greater the value. The formula used is equal to $\frac{p^2}{(4\pi A)}$ where *A* is the area of the blob and *p* is the perimeter of the blob. In the illustration above, the blobs have similar sizes but can be distinguished by their shapes. The compactness of the left blob is slightly above 1.0, while the compactness of the right blob is 1.24. The calculation of this feature can be enabled using MblobControl() with M_COMPACTNESS. Even if you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS, this result is calculated and returned in the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

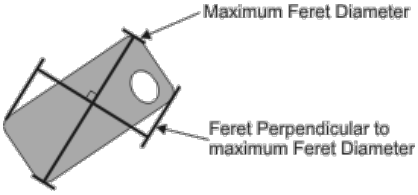
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CONVEX_HULL_AREA +	Retrieves the area of the convex hull of each blob. The calculation of this feature can be enabled using MblobControl() with M_CONVEX_HULL. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CONVEX_HULL_COG_X +	Retrieves the X-component of the center of gravity of the convex hull of each blob. The calculation of this feature can be enabled using MblobControl() with M_CONVEX_HULL. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CONVEX_HULL_COG_Y +	Retrieves the Y-component of the center of gravity of the convex hull of each blob. The calculation of this feature can be enabled using MblobControl() with M_CONVEX_HULL. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b

ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CONVEX_HULL_FILL_RATIO +	Retrieves the ratio between the blob's area and the area of its convex hull. In the example above, the left blob has a convex hull fill ratio of 1.0, whereas the middle and right-most blobs have a convex hull fill ratio of 0.9. The calculation of this feature can be enabled using MblobControl() with M_CONVEX_HULL.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CONVEX_HULL_PERIMETER +	Retrieves the perimeter of the convex hull of each blob. The perimeter is calculated by summing up the distance between every 2 consecutive points on the convex hull of a blob. The calculation of this feature can be enabled using MblobControl() with M_CONVEX_HULL. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CONVEX_PERIMETER +	Retrieves an approximation of the perimeter of the convex hull of each blob. It is derived from several Feret diameters; so, a larger number of Ferets gives a more accurate result. The calculation of this feature can be enabled using MblobControl() with M_CONVEX_PERIMETER. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b

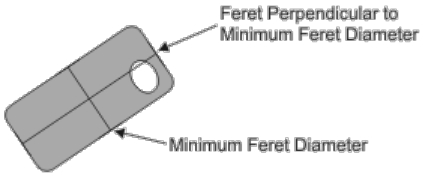
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_ELONGATION +		Retrieves a value that is equal to M_LENGTH / M_BREADTH of each blob. It is similar to M_FERET_ELONGATION, except that it should be used for long thin objects. The calculation of this feature can be enabled using MblobControl() with M_ELONGATION.  Even if you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS, this result is calculated and returned in the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_EULER_NUMBER +		Retrieves the number of blobs minus the number of holes (number of blobs - number of holes). This value is more useful if calculated using the M_WHOLE_IMAGE blob identification mode instead of the M_INDIVIDUAL blob identification mode. Unlike its usual effect, when using the M_WHOLE_IMAGE blob identification mode, each blob is treated individually, as opposed to grouping all of the blobs in an image together. So M_EULER_NUMBER returns (number of blobs - number of holes). Whereas as usual, when using the M_INDIVIDUAL blob identification mode, each blob is treated separately; therefore, the Euler number for each blob is just 1 - the number of holes. The calculation of this feature can be enabled using MblobControl() with M_EULER_NUMBER. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

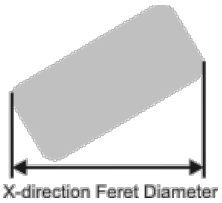
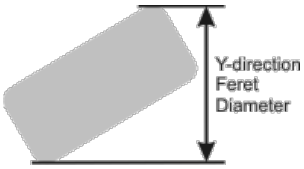
M_FERET_ELONGATION +	Retrieves the measure of the shape of each blob. It is equal to M_FERET_MAX_DIAMETER / M_FERET_MIN_DIAMETER. The calculation of this feature can be enabled using MblobControl() with M_FERETS. It is accurate for reasonably compact objects, but becomes less accurate for very elongated objects (because M_FERET_MIN_DIAMETER becomes less accurate). For very elongated objects, M_ELONGATION should be used.  Even if you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS, this result is calculated and returned in the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_GENERAL +	Retrieves the Feret diameter at the user-specified angle (set using MblobControl() with M_FERET_GENERAL_ANGLE). The calculation of this feature can be enabled using MblobControl() with M_FERET_GENERAL. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_FERET_MAX_ANGLE +	Retrieves the angle, in degrees, at which the maximum Feret diameter is found for each blob. The calculation of this feature can be enabled using MblobControl() with M_FERETS. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate(). An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_MAX_DIAMETER +	Retrieves the largest Feret diameter found after checking a certain number of angles. More angles will give a more accurate result, but will take longer to calculate. With regards to the number of angles evaluated, the maximum Feret diameter is less sensitive than the minimum Feret diameter; generally, evaluating the potential maximum Feret diameter over 8 angles will give an accurate result. The calculation of this feature can be enabled using MblobControl() with M_FERETS.  If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_FERET_MAX_DIAMETER_ELONGATION +	Retrieves the ratio between the maximum Feret diameter of each blob and its perpendicular Feret diameter. The calculation of this feature can be enabled using MblobControl() with M_FERET_MAX_DIAMETER_ELONGATION.  Even if you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS, this result is calculated and returned in the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_MEAN_DIAMETER +	Retrieves the average of the Feret diameters at the angles checked for each blob. The calculation of this feature can be enabled using MblobControl() with M_FERETS. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

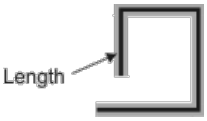
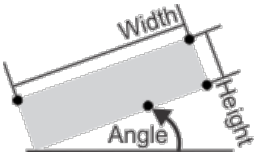
M_FERET_MIN_ANGLE +	Retrieves the angle, in degrees, at which the minimum Feret diameter is found for each blob. The calculation of this feature can be enabled using MblobControl() with M_FERETS. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate(). An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_MIN_DIAMETER +	Retrieves the minimum Feret diameter found after checking a certain number of angles. More angles will give a more accurate result, but will take longer to calculate. Note that this feature will not be very accurate for long thin blobs. However, you can get an accurate measure of the breadth of long thin blobs more quickly using M_BREADTH. The calculation of this feature can be enabled using MblobControl() with M_FERETS. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_FERET_MIN_DIAMETER_ELONGATION +	Retrieves the ratio between the minimum Feret diameter of each blob and its perpendicular Feret diameter. The calculation of this feature can be enabled using MblobControl() with M_FERET_MIN_DIAMETER_ELONGATION.  Even if you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS, this result is calculated and returned in the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_PERPENDICULAR_TO_MAX_DIAMETER +	Retrieves the Feret diameter that is perpendicular to the maximum Feret diameter of each blob. The calculation of this feature can be enabled using MblobControl() with M_FERET_PERPENDICULAR_TO_MAX_DIAMETER. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_PERPENDICULAR_TO_MIN_DIAMETER +	Retrieves the Feret diameter that is perpendicular to the minimum Feret diameter of each blob. The calculation of this feature can be enabled using MblobControl() with M_FERET_PERPENDICULAR_TO_MIN_DIAMETER. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_FERET_X +	Retrieves the dimension of the image-axis-aligned minimum bounding box of each blob, along the X-axis of the pixel coordinate system (that is, M_BOX_X_MAX - M_BOX_X_MIN + 1). The image-axis-aligned bounding box is the bounding box that is aligned with the pixel coordinate system's axes. The calculation of this feature can be enabled using MblobControl() with M_BOX.  If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_Y +	Retrieves the dimension of the image-axis-aligned minimum bounding box of each blob, along the Y-axis of the pixel coordinate system (that is, M_BOX_Y_MAX - M_BOX_Y_MIN + 1). The image-axis-aligned bounding box is the bounding box that is aligned with the pixel coordinate system's axes. The calculation of this feature can be enabled using MblobControl() with M_BOX.  If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b

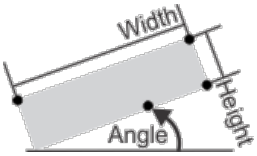
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FIRST_POINT_X +	Retrieves the X-coordinate of a unique point (along with M_FIRST_POINT_Y) for each blob, that is on the perimeter of the blob. The X-coordinate is that of the left-most pixel on the top-most line of the blob. The calculation of this feature can be enabled using MblobControl() with M_BOX. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FIRST_POINT_Y +	Retrieves the Y-coordinate of a unique point (along with M_FIRST_POINT_X) for each blob, that is on the perimeter of the blob. The Y-coordinate is that of the top-most line of the blob. The calculation of this feature can be enabled using MblobControl() with M_BOX. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERCEPT_0 +	Retrieves the number of times a transition from background to foreground (not vice versa) occurs in the horizontal direction for the entire blob. In other words, it is equal to the number of times the neighborhood configuration [B, F] occurs in a blob, where B is a background pixel and F is a foreground pixel. The calculation of this feature can be enabled using MblobControl() with M_INTERCEPT. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b

ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERCEPT_45 +	Retrieves the number of times that the neighborhood configuration $\begin{bmatrix} \bullet & F \\ B & \bullet \end{bmatrix}$ occurs in a blob, where *F* is a foreground pixel, *B* is a background pixel, and a dot can be any pixel value. The calculation of this feature can be enabled using MblobControl() with M_INTERCEPT. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERCEPT_90 +	Retrieves the number of times that the neighborhood configuration $\begin{bmatrix} F \\ B \end{bmatrix}$ occurs in a blob. The calculation of this feature can be enabled using MblobControl() with M_INTERCEPT. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERCEPT_135 +	Retrieves the number of times that the neighborhood configuration $\begin{bmatrix} F & \bullet \\ \bullet & B \end{bmatrix}$ occurs in a blob. The calculation of this feature can be enabled using MblobControl() with M_INTERCEPT. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_LABEL_VALUE +	Retrieves the label value of each blob. You can retrieve the label value of included or excluded blobs, but not deleted blobs. The label value is a positive integer (>= 1) that is unique for each blob.	c		e	f	g	a		i		j	k	l	m	n	b

	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_LENGTH +		Retrieves the length of each blob. This feature is only accurate for certain object types (for example, long thin blobs) because it is derived from the perimeter (P) and area (A) assuming that P = 2(length + breadth) and A = length x breadth. It complements M_FERET_MAX_DIAMETER because it is accurate for different blob types (for example, long thin blobs). Note, it is calculated much faster than the maximum Feret diameter. The calculation of this feature can be enabled using MblobControl() with M_LENGTH.  If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_AREA_BOX_ANGLE +		Retrieves the angle of the minimum-area bounding box of each blob. The angle is always measured from the X-axis of the pixel coordinate system, to the side of the box from which the width is measured. The calculation of this feature can be enabled using MblobControl() with M_MIN_AREA_BOX.  If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b

	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_AREA_BOX_AREA +		Retrieves the area of the minimum-area bounding box of each blob. The calculation of this feature can be enabled using MblobControl() with M_MIN_AREA_BOX. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_AREA_BOX_CENTER_X +		Retrieves the X-coordinate of the center of the minimum-area bounding box of each blob. The calculation of this feature can be enabled using MblobControl() with M_MIN_AREA_BOX. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_AREA_BOX_CENTER_Y +		Retrieves the Y-coordinate of the center of the minimum-area bounding box of each blob. The calculation of this feature can be enabled using MblobControl() with M_MIN_AREA_BOX. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

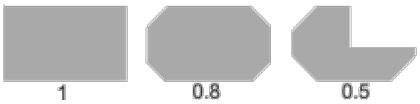
M_MIN_AREA_BOX_HEIGHT +	Retrieves the height (shortest side) of the minimum-area bounding box of each blob. The calculation of this feature can be enabled using MblobControl() with M_MIN_AREA_BOX. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_AREA_BOX_PERIMETER +	Retrieves the perimeter of the minimum-area bounding box of each blob. The calculation of this feature can be enabled using MblobControl() with M_MIN_AREA_BOX. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_AREA_BOX_WIDTH +	Retrieves the width (longest side) of the minimum-area bounding box of each blob. The calculation of this feature can be enabled using MblobControl() with M_MIN_AREA_BOX. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_MIN_PERIMETER_BOX_ANGLE +	Retrieves the angle of the minimum-perimeter bounding box of each blob. The angle is always measured from the X-axis of the pixel coordinate system to the side of the box from which the width is measured. The calculation of this feature can be enabled using MblobControl() with M_MIN_PERIMETER_BOX.  If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_PERIMETER_BOX_AREA +	Retrieves the area of the minimum-perimeter bounding box of each blob. The calculation of this feature can be enabled using MblobControl() with M_MIN_PERIMETER_BOX. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_MIN_PERIMETER_BOX_CENTER_X +	Retrieves the X-coordinate of the center of the minimum-perimeter bounding box of each blob. The calculation of this feature can be enabled using MblobControl() with M_MIN_PERIMETER_BOX. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_PERIMETER_BOX_CENTER_Y +	Retrieves the Y-coordinate of the center of the minimum-perimeter bounding box of each blob. The calculation of this feature can be enabled using MblobControl() with M_MIN_PERIMETER_BOX. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_PERIMETER_BOX_HEIGHT +	Retrieves the height (shortest side) of the minimum-perimeter bounding box of each blob. The calculation of this feature can be enabled using MblobControl() with M_MIN_PERIMETER_BOX. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_MIN_PERIMETER_BOX_PERIMETER +	Retrieves the perimeter of the minimum-perimeter bounding box of each blob. The calculation of this feature can be enabled using MblobControl() with M_MIN_PERIMETER_BOX. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_PERIMETER_BOX_WIDTH +	Retrieves the width (longest side) of the minimum-perimeter bounding box of each blob. The calculation of this feature can be enabled using MblobControl() with M_MIN_PERIMETER_BOX. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_CHAINED_PIXELS +	Retrieves the number of chained pixels in each blob. The calculation of this feature can be enabled using MblobControl() with M_CHAINS.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_CONVEX_HULL_POINTS +	Retrieves the number of points on the convex perimeter of each blob. A better approximation of convex hull features can be taken by increasing the number of Feret diameters evaluated using MblobControl() with M_NUMBER_OF_FERETS; this has the effect of increasing the number of convex hull points evaluated. The calculation of this feature can be enabled using MblobControl() with M_CONVEX_HULL.	c		e	f	g	a		i		j	k	l	m	n	b

	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_HOLES +		Retrieves the number of holes in each blob. Holes that intersect the edge of the image are not counted (they might not be holes). This value is equal to 1 - M_EULER_NUMBER (1 - (1 - number of holes)) and is therefore a true hole count if calculated using the M_INDIVIDUAL blob identification mode. The calculation of this feature can be enabled using MblobControl() with M_NUMBER_OF_HOLES.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_RUNS +		Retrieves the total number of runs in each blob. A run is defined as a horizontal series of consecutive foreground pixels. The calculation of this feature can be enabled using MblobControl() with M_RUNS.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_PERIMETER +		Retrieves the total length of edges in each blob (including the edges of any holes), whereby an allowance made for the staircase effect that is produced when diagonal edges are digitized (inside corners are counted as 1.414, rather than 2.0). A single pixel blob (area = 1) has a perimeter of 4.0. The calculation of this feature can be enabled using MblobControl() with M_PERIMETER. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_RECTANGULARITY +	Retrieves the degree to which each blob is similar to a rectangle. To do this, **MblobGetResult**() calculates the ratio between the blob's area and the product of its minimum Feret diameter and the Feret diameter perpendicular to the minimum Feret diameter.  The three blobs above have the same minimum Feret diameter and the same Feret diameter perpendicular to the minimum Feret diameter. Only their shape and area differs. The rectangularity of the left blob is 1.0, indicating a perfect rectangle. The rectangularity of the middle blob is 0.8 and the rectangularity of the right-most blob is 0.5. The calculation of this feature can be enabled using MblobControl() with M_RECTANGULARITY. Even if you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS, this result is calculated and returned in the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_ROUGHNESS +	Retrieves the roughness and irregularity of a blob is, and is equal to M_PERIMETER / M_CONVEX_PERIMETER. A smooth convex blob will have the minimum roughness of 1.0. In the example above, the left blob has a roughness of 1.0, the middle blob has a roughness of 1.1, and the right blob has a roughess of 1.5. The calculation of this feature can be enabled using MblobControl() with M_ROUGHNESS. Even if you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS, this result is calculated and returned in the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_WORLD_BOX_X_MAX +	Retrieves the extreme right X-coordinate of the world-axis-aligned bounding box, of each blob. The world-axis-aligned bounding box is the bounding box that is aligned with the relative coordinate system's axes. The calculation of this feature can be enabled using MblobControl() with M_WORLD_BOX. The M_WORLD_BOX... features are actually calculated in the world coordinate system. If these features were enabled for calculation in MblobControl(), you should have passed MblobCalculate() a calibrated blob identifier image. If an uncalibrated image was passed to MblobCalculate(), these features are the equivalent to the corresponding M_BOX_... features.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD_BOX_X_MIN +	Retrieves the extreme left X-coordinate of the world-axis-aligned bounding box, of each blob. The world-axis-aligned bounding box is the bounding box that is aligned with the relative coordinate system's axes. The calculation of this feature can be enabled using MblobControl() with M_WORLD_BOX. The M_WORLD_BOX... features are actually calculated in the world coordinate system. If these features were enabled for calculation in MblobControl(), you should have passed MblobCalculate() a calibrated blob identifier image. If an uncalibrated image was passed to MblobCalculate(), these features are the equivalent to the corresponding M_BOX_... features.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_WORLD_BOX_Y_MAX +	Retrieves the extreme bottom Y-coordinate of the world-axis-aligned bounding box, of each blob. The world-axis-aligned bounding box is the bounding box that is aligned with the relative coordinate system's axes. The calculation of this feature can be enabled using MblobControl() with M_WORLD_BOX. The M_WORLD_BOX... features are actually calculated in the world coordinate system. If these features were enabled for calculation in MblobControl(), you should have passed MblobCalculate() a calibrated blob identifier image. If an uncalibrated image was passed to MblobCalculate(), these features are the equivalent to the corresponding M_BOX_... features.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD_BOX_Y_MIN +	Retrieves the extreme top Y-coordinate of the world-axis-aligned bounding box, of each blob. The world-axis-aligned bounding box is the bounding box that is aligned with the relative coordinate system's axes. The calculation of this feature can be enabled using MblobControl() with M_WORLD_BOX. The M_WORLD_BOX... features are actually calculated in the world coordinate system. If these features were enabled for calculation in MblobControl(), you should have passed MblobCalculate() a calibrated blob identifier image. If an uncalibrated image was passed to MblobCalculate(), these features are the equivalent to the corresponding M_BOX_... features.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_WORLD_FERET_X +	Retrieves the dimension of the world-axis-aligned minimum bounding box, along the X-axis of the relative coordinate system (that is, M_WORLD_BOX_X_MAX - M_WORLD_BOX_X_MIN + 1). The world-axis-aligned bounding box is the bounding box that is aligned with the relative coordinate system's axes. The calculation of this feature can be enabled using MblobControl() with M_WORLD_BOX. The M_WORLD_FERET... features are actually calculated in the world coordinate system. If these features were enabled for calculation in MblobControl(), you should have passed MblobCalculate() a calibrated blob identifier image. If an uncalibrated image was passed to MblobCalculate(), these features are the equivalent to the corresponding M_FERET_... features.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD_FERET_Y +	Retrieves the dimension of the world-axis-aligned minimum bounding box, along the Y-axis of the relative coordinate system (that is, M_WORLD_BOX_Y_MAX - M_WORLD_BOX_Y_MIN + 1). The world-axis-aligned bounding box is the bounding box that is aligned with the relative coordinate system's axes. The calculation of this feature can be enabled using MblobControl() with M_WORLD_BOX. The M_WORLD_FERET... features are actually calculated in the world coordinate system. If these features were enabled for calculation in MblobControl(), you should have passed MblobCalculate() a calibrated blob identifier image. If an uncalibrated image was passed to MblobCalculate(), these features are the equivalent to the corresponding M_FERET_... features.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_WORLD_X_AT_Y_MAX +	Retrieves the X-coordinate at the maximum Y-coordinate of each blob, of the world-axis-aligned bounding box. The world-axis-aligned bounding box is the bounding box that is aligned with the relative coordinate system's axes. The calculation of this feature can be enabled using MblobControl() with M_WORLD_BOX. The M_WORLD_BOX... features are actually calculated in the world coordinate system. If these features were enabled for calculation in MblobControl(), you should have passed MblobCalculate() a calibrated blob identifier image. If an uncalibrated image was passed to MblobCalculate(), these features are the equivalent to the corresponding M_BOX_... features.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD_X_AT_Y_MIN +	Retrieves the X-coordinate at the minimum Y-coordinate of each blob, of the world-axis-aligned bounding box. The world-axis-aligned bounding box is the bounding box that is aligned with the relative coordinate system's axes. The calculation of this feature can be enabled using MblobControl() with M_WORLD_BOX. The M_WORLD_BOX... features are actually calculated in the world coordinate system. If these features were enabled for calculation in MblobControl(), you should have passed MblobCalculate() a calibrated blob identifier image. If an uncalibrated image was passed to MblobCalculate(), these features are the equivalent to the corresponding M_BOX_... features.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_WORLD_Y_AT_X_MAX +	Retrieves the Y-coordinate at the maximum X-coordinate of each blob, of the world-axis-aligned bounding box. The world-axis-aligned bounding box is the bounding box that is aligned with the relative coordinate system's axes. The calculation of this feature can be enabled using MblobControl() with M_WORLD_BOX. The M_WORLD_BOX... features are actually calculated in the world coordinate system. If these features were enabled for calculation in MblobControl(), you should have passed MblobCalculate() a calibrated blob identifier image. If an uncalibrated image was passed to MblobCalculate(), these features are the equivalent to the corresponding M_BOX_... features.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD_Y_AT_X_MIN +	Retrieves the Y-coordinate at the minimum X-coordinate of each blob, of the world-axis-aligned bounding box. The world-axis-aligned bounding box is the bounding box that is aligned with the relative coordinate system's axes. The calculation of this feature can be enabled using MblobControl() with M_WORLD_BOX. The M_WORLD_BOX... features are actually calculated in the world coordinate system. If these features were enabled for calculation in MblobControl(), you should have passed MblobCalculate() a calibrated blob identifier image. If an uncalibrated image was passed to MblobCalculate(), these features are the equivalent to the corresponding M_BOX_... features.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_X_MAX_AT_Y_MAX +	Retrieves the maximum X-coordinate at the maximum Y-coordinate of each blob. Together with M_BOX_Y_MAX, this determines one of the contact points on the convex perimeter of the blob. The calculation of this feature can be enabled using MblobControl() with M_CONTACT_POINTS. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_X_MAX_AT_Y_MIN +	Retrieves the maximum X-coordinate at the minimum Y-coordinate of each blob. Together with M_BOX_Y_MIN, this determines one of the contact points on the convex perimeter of the blob. The calculation of this feature can be enabled using MblobControl() with M_CONTACT_POINTS. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_X_MIN_AT_Y_MAX +	Retrieves the minimum X-coordinate at the maximum Y-coordinate of each blob. Together with M_BOX_Y_MAX, this determines one of the contact points on the convex perimeter of the blob. The calculation of this feature can be enabled using MblobControl() with M_CONTACT_POINTS. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b

	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_X_MIN_AT_Y_MIN +		Retrieves the minimum X-coordinate at the minimum Y-coordinate of each blob. Together with M_BOX_Y_MIN, this determines one of the contact points on the convex perimeter of the blob. The calculation of this feature can be enabled using MblobControl() with M_CONTACT_POINTS. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_Y_MAX_AT_X_MAX +		Retrieves the maximum Y-coordinate at the maximum X-coordinate of each blob. Together with M_BOX_X_MAX, this determines one of the contact points on the convex perimeter of the blob. The calculation of this feature can be enabled using MblobControl() with M_CONTACT_POINTS. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_Y_MAX_AT_X_MIN +	Retrieves the maximum Y-coordinate at the minimum X-coordinate of each blob. Together with M_BOX_X_MIN, this determines one of the contact points of the convex perimeter of the blob. The calculation of this feature can be enabled using MblobControl() with M_CONTACT_POINTS. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_Y_MIN_AT_X_MAX +	Retrieves the minimum Y-coordinate at the maximum X-coordinate of each blob. Together with M_BOX_X_MAX, this determines one of the contact points on the convex perimeter of the blob. The calculation of this feature can be enabled using MblobControl() with M_CONTACT_POINTS. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_Y_MIN_AT_X_MIN +	Retrieves the minimum Y-coordinate at the minimum X-coordinate of each blob. Together with M_BOX_X_MIN, this determines one of the contact points on the convex perimeter of the blob. The calculation of this feature can be enabled using MblobControl() with M_CONTACT_POINTS. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b

	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve the result for a feature with a grayscale definition, and that can be used as a sorting key, select one of the following. Results for these features are only available if both a blob identifier image and a grayscale buffer (or a grayscale buffer with an ROI that acts as the blob identifier image) were passed to MblobCalculate() (and the corresponding features were selected for calculation). In this case, you must set LabelOrIndex to a specific blob or all blobs, using M_BLOB_INDEX() or M_BLOB_LABEL() for a specific blob, or M_INCLUDED_BLOBS for all blobs.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving results for a feature with a grayscale definition, and that can be used as a sorting key																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_BLOB_CONTRAST +	Retrieves the difference between the maximum and minimum pixel values of a blob. It is equal to M_MAX_PIXEL - M_MIN_PIXEL. The calculation of this feature can be enabled using MblobControl() with M_BLOB_CONTRAST. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_PIXEL +	Retrieves the maximum pixel value found in each blob. The calculation of this feature can be enabled using MblobControl() with M_MAX_PIXEL. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_MEAN_PIXEL +	Retrieves the mean pixel value in each blob. It is equal to M_SUM_PIXEL / M_AREA. The calculation of this feature can be enabled using MblobControl() with M_MEAN_PIXEL. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_MIN_PIXEL +	Retrieves the minimum pixel value found in each blob. The calculation of this feature can be enabled using MblobControl() with M_MIN_PIXEL. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* | *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGMA_PIXEL +	Retrieves the standard deviation of pixel values in each blob. It is equal to $\sqrt{\frac{\sum p_i^2 - \frac{\left(\sum p_i\right)^2}{N}}{N}}$, where N = number of pixels and p = pixel value. The calculation of this feature can be enabled using MblobControl() with M_SIGMA_PIXEL. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* | *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_SUM_PIXEL +	Retrieves the sum of all pixel values in each blob. The calculation of this feature can be enabled using MblobControl() with M_SUM_PIXEL. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* | *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_SUM_PIXEL_SQUARED +	Retrieves the sum of the squares of each pixel value in each blob. The calculation of this feature can be enabled using MblobControl() with M_SUM_PIXEL_SQUARED. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* | *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve the result for a feature that has two different definitions (a binary and a grayscale definition), and that can be used as a sorting key, select one of the following values. If you did not provide both a blob identifier image and a grayscale buffer, only the binary version was calculated. If you did provide a grayscale buffer, both versions were calculated, unless otherwise specified. You must set LabelOrIndex to a specific blob or all blobs, using M_BLOB_INDEX() or M_BLOB_LABEL() for a specific blob, or M_INCLUDED_BLOBS for all blobs.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving results for a result of a feature that has two different definitions, and that can be used as a sorting key

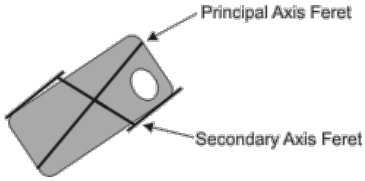
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ResultArrayPtr - Possible values returned																
M_AXIS_PRINCIPAL_ANGLE +		Retrieves the angle, in degrees, at which each blob has the least moment of inertia. When the blob identifier image is calibrated, this feature is calculated in calibrated units; otherwise, it is calculated in pixel units. For elongated blobs, the angle of the blobs' longest length is taken. The result is calculated as: $-0.5 * \mathrm{atan} \frac{(2 * M_MOMENT_CENTRAL_X1_Y1)}{M_MOMENT_CENTRAL_X2_Y0 - M_MOMENT_CENTRAL_X0_Y2}$ The result is always between -90° and +90°, relative to the output coordinate system specified using MblobControl() with M_RESULT_OUTPUT_UNITS. The calculation of this feature can be enabled using MblobControl() with M_MOMENT_SECOND_ORDER. An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup. You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature															
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_AXIS_SECONDARY_ANGLE +	Retrieves the angle perpendicular to M_AXIS_PRINCIPAL_ANGLE of each blob, in degrees. When the blob identifier image is calibrated, this feature is calculated in calibrated units; otherwise, it is calculated in pixel units. The calculation of this feature can be enabled using MblobControl() with M_MOMENT_SECOND_ORDER. The result is always between -90° and +90°, relative to the output coordinate system specified using MblobControl() with M_RESULT_OUTPUT_UNITS. An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup. You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_CENTER_OF_GRAVITY_X +	Retrieves the X-position of the center of gravity of each blob. The grayscale version is M_MOMENT_X1_Y0 / M_SUM_PIXEL. The binary version uses M_AREA instead of M_SUM_PIXEL. The calculation of this feature can be enabled using MblobControl() with M_CENTER_OF_GRAVITY. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate(). You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_CENTER_OF_GRAVITY_Y +	Retrieves the Y-position of the center of gravity of each blob. The grayscale version is M_MOMENT_X0_Y1 / M_SUM_PIXEL. The binary version uses M_AREA instead of M_SUM_PIXEL. The calculation of this feature can be enabled using MblobControl() with M_CENTER_OF_GRAVITY. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate(). You must specify a combination value from the following table: ● To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_FERET_AT_PRINCIPAL_AXIS_ANGLE +		Retrieves the Feret diameter at the principal axis of each blob. The calculation of this feature can be enabled using MblobControl() with M_FERET_AT_PRINCIPAL_AXIS_ANGLE. The principal axis is the axis at which the blob has the least moment of inertia. Also, if the blob is symmetrical, the principal axis is aligned with the blob's axis of symmetry. You can retrieve the angle of the principal axis, by calling **MblobGetResult**() with M_AXIS_PRINCIPAL_ANGLE. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate(). You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_FERET_AT_SECONDARY_AXIS_ANGLE +	Retrieves the Feret diameter at the secondary axis of each blob. The calculation of this feature can be enabled using MblobControl() with M_FERET_AT_SECONDARY_AXIS_ANGLE. The secondary axis is perpendicular to the principal axis. You can retrieve the angle of the secondary axis by calling **MblobGetResult**() with M_AXIS_SECONDARY_ANGLE. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into world units, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate(). You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_PRINCIPAL_AXIS_ELONGATION +	Retrieves the ratio between the Feret diameter at the principal axis and the Feret diameter at the secondary axis of each blob. It is equal to M_FERET_AT_PRINCIPAL_AXIS_ANGLE / M_FERET_AT_SECONDARY_AXIS_ANGLE. The calculation of this feature can be enabled using MblobControl() with M_FERET_PRINCIPAL_AXIS_ELONGATION.  This result is not affected by the setting of M_RESULT_OUTPUT_UNITS. You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b

	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_CENTRAL_X0_Y2 +		Retrieves the central moment of each blob where the order of X equals 0 and the order of Y equals 2. The calculation of this feature can be enabled using MblobControl() with M_MOMENT_SECOND_ORDER. This central moment is defined as $\sum_{i}^{n} (y_i-\bar{y})^2 p_i$ where *i* runs from 1 to the number of pixels in the blob, where *n* is the number of pixels in the blob, p_i = value of a pixel (always 1 for binary moments), y_i = its Y-coordinate, $\bar{y}$ = the weighted mean of y_i . The weighted mean is the first order ordinary moment divided by the sum of the pixel values (M_SUM_PIXEL for grayscale definitions, M_AREA for binary definitions). For central moments, coordinates are relative to each blob's center of gravity, as opposed to ordinary moments, which use coordinates that are relative to the image origin (top-left corner). This result is not affected by the setting of M_RESULT_OUTPUT_UNITS. Results are only available in pixel units. You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_MOMENT_CENTRAL_X0_Y3 +	Retrieves the central moment of each blob where the order of X equals 0 and the order of Y equals 3. The calculation of this feature can be enabled using MblobControl() with M_MOMENT_THIRD_ORDER. This central moment is defined as $\sum_{i}^{n}(y_i - \overline{y})^3 p_i$, where *i* runs from 1 to the number of pixels in the blob, where *n* is the number of pixels in the blob, p_i = value of a pixel (always 1 for binary moments), y_i = its Y-coordinate, $\bar{y}$ = the weighted mean of y_i . The weighted mean is the first order ordinary moment divided by the sum of the pixel values (M_SUM_PIXEL for grayscale definitions, M_AREA for binary definitions). For central moments, coordinates are relative to each blob's center of gravity, as opposed to ordinary moments, which use coordinates that are relative to the image origin (top-left corner). This result is not affected by the setting of M_RESULT_OUTPUT_UNITS. Results are only available in pixel units. You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_MOMENT_CENTRAL_X1_Y1 +		Retrieves the central moment of each blob where the order of X equals 1 and the order of Y equals 1. The calculation of this feature can be enabled using MblobControl() with M_MOMENT_SECOND_ORDER. This central moment is defined as $\sum_{i}^{n}(x_i-\bar{x})\ (y_i-\bar{y})\ \ p_i$ where *i* runs from 1 to the number of pixels in the blob, where *n* is the number of pixels in the blob, p_i = value of a pixel (always 1 for binary moments), x_i = its X-coordinate and, y_i = its Y-coordinate, $\bar{x}$ = the weighted mean of x_i , and $\bar{y}$ = the weighted mean of y_i . The weighted mean is the first order ordinary moment divided by the sum of the pixel values (M_SUM_PIXEL for grayscale definitions, M_AREA for binary definitions). For central moments, coordinates are relative to each blob's center of gravity, as opposed to ordinary moments, which use coordinates that are relative to the image origin (top-left corner). This result is not affected by the setting of M_RESULT_OUTPUT_UNITS. Results are only available in pixel units. You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_MOMENT_CENTRAL_X1_Y2 +	Retrieves the central moment of each blob where the order of X equals 1 and the order of Y equals 2. The calculation of this feature can be enabled using MblobControl() with M_MOMENT_THIRD_ORDER. This central moment is defined as $\sum_{i}^{n}(x_i - \overline{x})(y_i - \overline{y})^2 p_i$, where i runs from 1 to the number of pixels in the blob, where n is the number of pixels in the blob, p_i = value of a pixel (always 1 for binary moments), x_i = its X-coordinate and, y_i = its Y-coordinate, $\overline{x}$ = the weighted mean of x_i , and $\overline{y}$ = the weighted mean of y_i . The weighted mean is the first order ordinary moment divided by the sum of the pixel values (M_SUM_PIXEL for grayscale definitions, M_AREA for binary definitions). For central moments, coordinates are relative to each blob's center of gravity, as opposed to ordinary moments, which use coordinates that are relative to the image origin (top-left corner). This result is not affected by the setting of M_RESULT_OUTPUT_UNITS. Results are only available in pixel units. You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_MOMENT_CENTRAL_X2_Y0 +	Retrieves the central moment of each blob where the order of X equals 2 and the order of Y equals 0. The calculation of this feature can be enabled using MblobControl() with M_MOMENT_SECOND_ORDER. This central moment is defined as $\sum_{i}^{n}(x_i-\bar{x})^2 p_i$ where *i* runs from 1 to the number of pixels in the blob, where *n* is the number of pixels in the blob, p_i = value of a pixel (always 1 for binary moments), x_i = its X-coordinate, $\bar{x}$ = the weighted mean of x_i . The weighted mean is the first order ordinary moment divided by the sum of the pixel values (M_SUM_PIXEL for grayscale definitions, M_AREA for binary definitions). For central moments, coordinates are relative to each blob's center of gravity, as opposed to ordinary moments, which use coordinates that are relative to the image origin (top-left corner). This result is not affected by the setting of M_RESULT_OUTPUT_UNITS. Results are only available in pixel units. You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_MOMENT_CENTRAL_X2_Y1 +	Retrieves the central moment of each blob where the order of X equals 2 and the order of Y equals 1. The calculation of this feature can be enabled using MblobControl() with M_MOMENT_THIRD_ORDER. This central moment is defined as $\sum_{i}^{n}(x_i - \overline{x})^2(y_i - \overline{y})p_i$, where i runs from 1 to the number of pixels in the blob, where n is the number of pixels in the blob, p_i = value of a pixel (always 1 for binary moments), x_i = its X-coordinate and, y_i = its Y-coordinate, $\overline{x}$ = the weighted mean of x_i , and $\overline{y}$ = the weighted mean of y_i . The weighted mean is the first order ordinary moment divided by the sum of the pixel values (M_SUM_PIXEL for grayscale definitions, M_AREA for binary definitions). For central moments, coordinates are relative to each blob's center of gravity, as opposed to ordinary moments, which use coordinates that are relative to the image origin (top-left corner). This result is not affected by the setting of M_RESULT_OUTPUT_UNITS. Results are only available in pixel units. You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_MOMENT_CENTRAL_X3_Y0 +	Retrieves the central moment of each blob where the order of X equals 3 and the order of Y equals 0. The calculation of this feature can be enabled using MblobControl() with M_MOMENT_THIRD_ORDER. This central moment is defined as $\sum_{i}^{n}(x_i - \bar{x})^3 p_i$, where *i* runs from 1 to the number of pixels in the blob, where *n* is the number of pixels in the blob, p_i = value of a pixel (always 1 for binary moments), x_i = its X-coordinate, $\bar{x}$ = the weighted mean of x_i . The weighted mean is the first order ordinary moment divided by the sum of the pixel values (M_SUM_PIXEL for grayscale definitions, M_AREA for binary definitions). For central moments, coordinates are relative to each blob's center of gravity, as opposed to ordinary moments, which use coordinates that are relative to the image origin (top-left corner). This result is not affected by the setting of M_RESULT_OUTPUT_UNITS. Results are only available in pixel units. You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_MOMENT_GENERAL +	Retrieves the moment calculation, based on the settings specified using MblobControl() with M_MOMENT_GENERAL_MODE, M_MOMENT_GENERAL_ORDER_X, and M_MOMENT_GENERAL_ORDER_Y. The calculation of this feature can be enabled using MblobControl() with M_MOMENT_GENERAL. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS. You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_MOMENT_HU_*n* +	Retrieves the nth Hu moment of each blob. The calculation of these features can be enabled using MblobControl() with M_MOMENT_THIRD_ORDER. The seven Hu moments are defined as: $I_1 = \eta_{20} + \eta_{02}$ $I_2 = (\eta_{20} - \eta_{02})^2 + 4\eta_{11}^2$ $I_3 = (\eta_{30} - 3\eta_{12})^2 + (3\eta_{21} - \eta_{03})^2$ $I_4 = (\eta_{30} + \eta_{12})^2 + (\eta_{21} + \eta_{03})^2$ $I_5 = (\eta_{30} - 3\eta_{12})(\eta_{30} + \eta_{12})[(\eta_{30} + \eta_{12})^2 - 3(\eta_{21} + \eta_{03})^2] + (3\eta_{21} - \eta_{03})(\eta_{21} + \eta_{03})[3(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2]$ $I_6 = (\eta_{20} - \eta_{02})[(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2] + 4\eta_{11}(\eta_{30} + \eta_{12})(\eta_{21} + \eta_{03})$ $I_7 = (3\eta_{21} - \eta_{03})(\eta_{30} + \eta_{12})[(\eta_{30} + \eta_{12})^2 - 3(\eta_{21} + \eta_{03})^2] - (\eta_{30} - 3\eta_{12})(\eta_{21} + \eta_{03})[3(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2]$ Each scale invariant is defined with: $\eta_{qr} = \frac{\sum_i^n (x_i - \bar{x})^q (y_i - \bar{y})^r p_i}{\sum_i^n p_i^{1+\frac{i+j}{2}}}$ where i runs from 1 to the number of pixels in the blob, where n is the number of pixels in the blob, p_i = value of a pixel (always 1 for binary moments), x_i = its X-coordinate and, y_i = its Y-coordinate, $\bar{x}$ = the weighted mean of x_i , and $\bar{y}$ = the weighted mean of y_i . This result is not affected by the setting of M_RESULT_OUTPUT_UNITS. Results are only available in pixel units. You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_MOMENT_X0_Y1 +	Retrieves the ordinary moment of each blob where the order of X equals 0 and the order of Y equals 1. The calculation of this feature can be enabled using MblobControl() with M_MOMENT_FIRST_ORDER. This ordinary moment is defined as $\sum_{i}^{n} y_i \; p_i$ where *i* runs from 1 to the number of pixels in the blob, where *n* is the number of pixels in the blob, p_i = value of a pixel (always 1 for binary moments) and y_i = its Y-coordinate. This calculation returns the mean of Y for each blob. Therefore, in order to calculate the weighted arithmetic mean about *Y*, take the result returned by the calculation of this feature and divide it by the M_SUM_PIXEL feature (for a grayscale definition), for a binary definition divide it by the number of pixels in the blob (M_AREA). For ordinary moments, coordinates are relative to the image origin (top-left corner), as opposed to central moments, which use coordinates that are relative to each blob's center of gravity. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS. Results are only available in pixel units. You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_MOMENT_X0_Y2 +	Retrieves the ordinary moment of each blob where the order of X equals 0 and the order of Y equals 2. The calculation of this feature can be enabled using MblobControl() with M_MOMENT_SECOND_ORDER. This ordinary moment is defined as $\sum_{i}^{n} y_i^2 p_i$ where *i* runs from 1 to the number of pixels in the blob, where *n* is the number of pixels in the blob, p_i = value of a pixel (always 1 for binary moments) and y_i = its Y-coordinate. For ordinary moments, coordinates are relative to the image origin (top-left corner), as opposed to central moments, which use coordinates that are relative to each blob's center of gravity. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS. Results are only available in pixel units. You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_MOMENT_X0_Y3 +	Retrieves the ordinary moment of each blob where the order of X equals 0 and the order of Y equals 3. The calculation of this feature can be enabled using MblobControl() with M_MOMENT_THIRD_ORDER. This ordinary moment is defined as $\sum_{i}^{n} y_i^3 p_i$, where *i* runs from 1 to the number of pixels in the blob, where *n* is the number of pixels in the blob, p_i = value of a pixel (always 1 for binary moments) and y_i = its Y-coordinate. For ordinary moments, coordinates are relative to the image origin (top-left corner), as opposed to central moments, which use coordinates that are relative to each blob's center of gravity. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS. Results are only available in pixel units. You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_MOMENT_X1_Y0 +	Retrieves the ordinary moment of each blob where the order of X equals 1 and the order of Y equals 0. The calculation of this feature can be enabled using MblobControl() with M_MOMENT_FIRST_ORDER. This ordinary moment is defined as $\sum_{i}^{n} x_i \, p_i$ where *i* runs from 1 to the number of pixels in the blob, where *n* is the number of pixels in the blob, p_i = value of a pixel (always 1 for binary moments) and x_i = its X-coordinate. This calculation returns the mean of X for each blob. Therefore, in order to calculate the weighted arithmetic mean about *X*, take the result returned by the calculation of this feature and divide it by the M_SUM_PIXEL feature (for a grayscale definition), for a binary definition divide it by the number of pixels in the blob (M_AREA). For ordinary moments, coordinates are relative to the image origin (top-left corner), as opposed to central moments, which use coordinates that are relative to each blob's center of gravity. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS. Results are only available in pixel units. You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_MOMENT_X1_Y1 +	Retrieves the ordinary moment of each blob where the order of X equals 1 and the order of Y equals 1. The calculation of this feature can be enabled using MblobControl() with M_MOMENT_SECOND_ORDER. This ordinary moment is defined as $\sum_{i}^{n} x_i \; y_i \; p_i$ where *i* runs from 1 to the number of pixels in the blob, where *n* is the number of pixels in the blob, p_i = value of a pixel (always 1 for binary moments), x_i = its X-coordinate and y_i = its Y-coordinate. For ordinary moments, coordinates are relative to the image origin (top-left corner), as opposed to central moments, which use coordinates that are relative to each blob's center of gravity. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS. Results are only available in pixel units. You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_MOMENT_X1_Y2 +		Retrieves the ordinary moment of each blob where the order of X equals 1 and the order of Y equals 2. The calculation of this feature can be enabled using MblobControl() with M_MOMENT_THIRD_ORDER. This ordinary moment is defined as $\sum_{i}^{n} x_i y_i^2 p_i$ where *i* runs from 1 to the number of pixels in the blob, where *n* is the number of pixels in the blob, p_i = value of a pixel (always 1 for binary moments), x_i = its X-coordinate and y_i = its Y-coordinate. For ordinary moments, coordinates are relative to the image origin (top-left corner), as opposed to central moments, which use coordinates that are relative to each blob's center of gravity. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS. Results are only available in pixel units. You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_MOMENT_X2_Y0 +	Retrieves the ordinary moment of each blob where the order of X equals 2 and the order of Y equals 0. The calculation of this feature can be enabled using MblobControl() with M_MOMENT_SECOND_ORDER. This ordinary moment is defined as $\sum_{i}^{n} x_i^2 p_i$ where *i* runs from 1 to the number of pixels in the blob, where *n* is the number of pixels in the blob, p_i = value of a pixel (always 1 for binary moments) and x_i = its X-coordinate. For ordinary moments, coordinates are relative to the image origin (top-left corner), as opposed to central moments, which use coordinates that are relative to each blob's center of gravity. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS. Results are only available in pixel units. You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_MOMENT_X2_Y1 +		Retrieves the ordinary moment of each blob where the order of X equals 2 and the order of Y equals 1. The calculation of this feature can be enabled using MblobControl() with M_MOMENT_THIRD_ORDER. This ordinary moment is defined as $\sum_{i}^{n} x_i^2 y_i p_i$ where *i* runs from 1 to the number of pixels in the blob, where *n* is the number of pixels in the blob, p_i = value of a pixel (always 1 for binary moments), x_i = its X-coordinate and y_i = its Y-coordinate. For ordinary moments, coordinates are relative to the image origin (top-left corner), as opposed to central moments, which use coordinates that are relative to each blob's center of gravity. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS. Results are only available in pixel units. You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_MOMENT_X3_Y0 +	Retrieves the ordinary moment of each blob where the order of X equals 3 and the order of Y equals 0. The calculation of this feature can be enabled using MblobControl() with M_MOMENT_THIRD_ORDER. This ordinary moment is defined as $\sum_{i}^{n} x_i^3 p_i$, where *i* runs from 1 to the number of pixels in the blob, where *n* is the number of pixels in the blob, p_i = value of a pixel (always 1 for binary moments) and x_i = its X-coordinate. For ordinary moments, coordinates are relative to the image origin (top-left corner), as opposed to central moments, which use coordinates that are relative to each blob's center of gravity. This result is not affected by the setting of M_RESULT_OUTPUT_UNITS. Results are only available in pixel units. You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale definition of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve the result for a depth map specific feature, and that can be used as a sorting key, select one of the following. Results for these features are only available if a grayscale buffer that is a fully corrected depth map image buffer or 3D-processable depth map container was passed to MblobCalculate() (and the corresponding features were selected for calculation); in addition, the blob identifier image and the confidence information of the depth map must have been merged and passed as the ROI of the grayscale buffer. To retrieve these results, you must set LabelOrIndex to a specific blob or all blobs, using M_BLOB_INDEX() or M_BLOB_LABEL() for a specific blob, or M_INCLUDED_BLOBS for all blobs.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving results of a depth map specific feature, and that can be used as a sorting key																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_DEPTH_MAP_MAX_ELEVATION +	Retrieves the maximum elevation of each blob.															

	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEPTH_MAP_MEAN_ELEVATION +		Retrieves the mean elevation of each blob.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEPTH_MAP_MIN_ELEVATION +		Retrieves the minimum elevation of each blob.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEPTH_MAP_SIZE_Z +		Retrieves the Z-size (difference between maximum and minimum elevation) of each blob.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEPTH_MAP_VOLUME +		Retrieves the volume of each blob.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_FERET_GENERAL; M_FERET_MAX_DIAMETER; M_FERET_MIN_DIAMETER; M_FERET_PERPENDICULAR_TO_MAX_DIAMETER; M_FERET_PERPENDICULAR_TO_MIN_DIAMETER; M_FERET_X; M_FERET_Y; M_FERET_AT_PRINCIPAL_AXIS_ANGLE; M_FERET_AT_SECONDARY_AXIS_ANGLE.

You can add one of the following values to the above-mentioned values to specify to retrieve the contact points of the Feret.

The calculation of Feret contact points (M_FERET_CONTACT_POINTS) must have been enabled to use these combination constants.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving Feret contact points																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_FERET_CONTACT_POINTS_X1	Retrieves the X-coordinate for the first contact point of the Feret diameter of each blob. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_CONTACT_POINTS_X2	Retrieves the X-coordinate for the second contact point of the Feret diameter of each blob. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_CONTACT_POINTS_Y1	Retrieves the Y-coordinate for the first contact point of the Feret diameter of each blob. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_CONTACT_POINTS_Y2	Retrieves the Y-coordinate for the second contact point of the Feret diameter of each blob. If you requested to retrieve results in world units with M_RESULT_OUTPUT_UNITS and the calculation was performed on appropriate buffers, the results returned are transformed into the relative coordinate system, taking into account distortions as best as possible. If you require precise results, you should correct the image using McalTransformImage() before calling MblobCalculate().	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For retrieving results for a result of a feature that has two different definitions, and that can be used as a sorting key.

You must add one of the following values to the above-mentioned values to set whether the results should be returned for the binary or grayscale definition of the selected feature.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For features that have two definitions																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BINARY +	Retrieves the result for the binary definition of the selected feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAYSCALE +	Retrieves the result for the grayscale definition of the selected feature. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for

Tables:

- For retrieving general results from the blob result buffer
- For retrieving results of a feature with a binary definition, and that cannot be used as a sorting key
- For retrieving results of a feature with a binary definition, and that can be used as a sorting key
- For retrieving results for a feature with a grayscale definition, and that can be used as a sorting key
- For retrieving results for a result of a feature that has two different definitions, and that can be used as a sorting key
- For retrieving results of a depth map specific feature, and that can be used as a sorting key

the values listed in all the parameter association tables.

You can add the following value to the above-mentioned values to specify whether a result is available.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining whether results are available																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																	
M_AVAILABLE	Retrieves whether the requested result type is available for retrieval.																
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the requested result type is not available.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the requested result type is available.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For retrieving results of a feature with a binary definition, and that cannot be used as a sorting key, For retrieving results of a feature with a binary definition, and that can be used as a sorting key, For retrieving results for a result of a feature that has two different definitions, and that can be used as a sorting key, For retrieving results of a depth map specific feature, and that can be used as a sorting key; and for the following values: M_CHAIN_INDEX; M_CHAIN_X; M_CHAIN_Y; M_CONVEX_HULL_X; M_CONVEX_HULL_XY_PACKED; M_CONVEX_HULL_Y; M_FERET_DIAMETERS; M_RUN_LENGTH; M_RUN_X; M_RUN_Y.

You can add the following value to the above-mentioned values to determine the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the required array size (number of elements) to store the returned values																	
ResultType combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_NB_ELEMENTS	Retrieves the required array size (number of elements) to store the returned values.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for
Tables:

- For retrieving general results from the blob result buffer
- For retrieving results of a feature with a binary definition, and that cannot be used as a sorting key
- For retrieving results of a feature with a binary definition, and that can be used as a sorting key
- For retrieving results for a feature with a grayscale definition, and that can be used as a sorting key
- For retrieving results for a result of a feature that has two different definitions, and that can be used as a sorting key
- For retrieving results of a depth map specific feature, and that can be used as a sorting key
- For features that have two definitions

the values listed in all the parameter association tables.

You can add one of the following values to the above-mentioned values to set the requested results to a data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a data type																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested results to an *AIL_DOUBLE*.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT	Casts the requested results to an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b

ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_FLOAT* **|** *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT* **|** *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT16	Casts the requested results to an *AIL_INT16*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT16* **|** *array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT32* **|** *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested results to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT64* **|** *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]*	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailblob11.lib.
DLL	Requires ail11.dll; ailblob11.dll.

MblobInquire

Synopsis

Inquire about a blob analysis setting.

Syntax

```
AIL_INT MblobInquire(
    AIL_ID ContextOrResultBlobId, //in
    AIL_INT64 InquireType,        //in
    void *UserVarPtr              //out
)
```

Description

This function inquires about a specified setting associated with a blob analysis context or result buffer.

You can use MblobControl() to change a setting associated with a context or a result buffer.

Parameters

ContextOrResultBlobId

Specifies the identifier of a blob analysis context or result buffer.

InquireType

Specifies the type of setting about which to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_DOUBLE
- AIL_ID
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address in which to write the requested information. Since the **MblobInquire**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For inquiring the system on which the result buffer or context is allocated
- For inquiring about the global processing settings of a blob analysis context that cause results to be recalculated
- For inquiring about a global processing settings of a blob analysis context that do not cause results to be recalculated
- For inquiring about features with only a binary definition
- For inquiring about features with only a grayscale definition

- For inquiring about features with two definitions (binary and grayscale)
- For inquiring about depth map specific features
- For inquiring about a processing setting in a blob analysis result buffer

For the following inquire type, the ContextOrResultBlobId parameter can specify a blob analysis result buffer or context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the system on which the result buffer or context is allocated																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_OWNER_SYSTEM +		Inquires the identifier of the system on which the blob result buffer or context is allocated. SET SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
	System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

For the following inquire types, the ContextOrResultBlobId parameter can specify a blob analysis context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about the global processing settings of a blob analysis context that cause results to be recalculated																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_BLOB_IDENTIFICATION_MODE +		Inquires the blob identification mode that was selected. SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_INDIVIDUAL. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_INDIVIDUAL	Specifies that all blobs are measured individually. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_LABELED	Specifies that blobs with the same label are grouped together, and that touching blobs with different labels are also grouped together. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_LABELED_TOUCHING	Specifies that blobs with the same label are grouped together, and that touching blobs with different labels are measured individually. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_WHOLE_IMAGE	Specifies that all blobs are grouped together. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CONNECTIVITY +	Inquires the image lattice. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_8_CONNECTED.	c		e	f	g	a		i		j	k	l	m	n	b
M_4_CONNECTED	Specifies that each pixel has 4 neighbors.	c		e	f	g	a		i		j	k	l	m	n	b
M_8_CONNECTED	Specifies that each pixel has 8 neighbors.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_ANGLE_SEARCH_END +	Inquires the upper limit of the angular search range used in the calculation of the minimum or maximum Feret diameter. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 180.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 180	Specifies the upper limit of the angular search range. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_ANGLE_SEARCH_START +	Inquires the lower limit of the angular search range used in the calculation of the minimum or maximum Feret diameter. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 180	Specifies the lower limit of the angular search range. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_VALUE +	Inquires the pixel values that are considered to be in the foreground. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_DEFAULT	Same as M_NONZERO.	c		e	f	g	a		i		j	k	l	m	n	b
M_NONZERO	Specifies the blobs consisting of non-zero pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_ZERO	Specifies the blobs consisting of zero pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_FERETS +	Inquires the number of Feret angles set to calculate a Feret feature. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 8.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies the use of a high precision algorithm to accurately calculate Feret features (and features based on Feret features). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_FERETS	Specifies the minimum number of Feret angles. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value > M_MIN_FERETS	Specifies the number of Feret angles. INFO	c		e	f	g	a		i		j	k	l	m	n	b

For the following inquire types, the ContextOrResultBlobId parameter must specify a blob analysis context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a global processing settings of a blob analysis context that do not cause results to be recalculated																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_FERET_CONTACT_POINTS +	Inquires whether Feret contact points will be calculated. If enabled, the M_FERET_CONTACT_POINTS_... combination constants are available in MblobGetResult(). SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b

M_DISABLE	Specifies that the Feret contact points for supported features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that the Feret contact points for supported features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_GENERAL_ANGLE +	Inquires the angle to use when calculating the user-specified Feret. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
-360.0 <= *Value* <= 360.0	Specifies the angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_IDENTIFIER_TYPE +	Inquires the values that the non-zero pixels in the image have. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_GRAYSCALE.	c		e	f	g	a		i		j	k	l	m	n	b
M_BINARY	Specifies that non-zero pixels must have the maximum value of the buffer (for example, 0xff for an 8-bit image).	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAYSCALE	Specifies that non-zero pixels can have any value.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_BLOBS +	Inquires the maximum number of blobs to process. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that there is no limit on the maximum number of blobs.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the maximum number of blobs.	c		e	f	g	a		i		j	k	l	m	n	b

M_MOMENT_GENERAL_MODE +	Inquires the type of moment that will be calculated when calculating a user-specified moment. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ORDINARY.	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTRAL	Specifies a central moment. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ORDINARY	Specifies an ordinary moment. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_GENERAL_ORDER_X +	Inquires the order of the X-component of the specified general moment. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the X-order of the moment.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_GENERAL_ORDER_Y +	Inquires the order of the Y-component of the specified general moment. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the Y-order of the moment.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_THIRD_ORDER_MODE +	Inquires how third-order moments are calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_FAST.	c		e	f	g	a		i		j	k	l	m	n	b
M_FAST	Specifies that third-order moments will be calculated quickly.	c		e	f	g	a		i		j	k	l	m	n	b

M_PRECISE	Specifies that third-order moments will be calculated accurately.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_ASPECT_RATIO +	Inquires the pixel aspect ratio that you set for the image(s). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the pixel width/pixel height.	c		e	f	g	a		i		j	k	l	m	n	b
M_RETURN_PARTIAL_RESULTS +	Inquires whether results from partially scanned images will be available after a stop condition is met. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to discard results of partially scanned images when processing is interrupted.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to make results of partially scanned images available when processing is interrupted.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE_RUNS +	Inquires whether the run information will be saved when calling MblobCalculate(). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to save run information. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to save run information. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_SORT*n* +	Inquires the feature used as the *n* th sorting key, where *n* stands for an integer between 1 and 3. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_NO_SORT.	c		e	f	g	a		i		j	k	l	m	n	b
M_AREA	Specifies that the following is used as the sorting key: the number of foreground pixels in each blob (holes are not counted). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_AXIS_PRINCIPAL_ANGLE +	Specifies that the following is used as the sorting key: the angle, in degrees, at which each blob has the least moment of inertia. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
M_AXIS_SECONDARY_ANGLE +	Specifies that the following is used as the sorting key: the angle perpendicular to M_AXIS_PRINCIPAL_ANGLE of each blob, in degrees. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
M_BLOB_CONTRAST	Specifies that the following is used as the sorting key: the difference between the maximum and minimum pixel values of a blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_BLOB_TOUCHING_IMAGE_BORDERS	Specifies that the following is used as the sorting key: whether each blob is touching the borders of the image. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_BOX_AREA	Specifies that the following is used as the sorting key: the area covered by the image-axis-aligned bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BOX_ASPECT_RATIO	Specifies that the following is used as the sorting key: the ratio between the horizontal size and the vertical size of the image-axis-aligned bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BOX_FILL_RATIO	Specifies that the following is used as the sorting key: the ratio between the area of the blob and the area of the image-axis-aligned bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BOX_X_MAX	Specifies that the following is used as the sorting key: the extreme right X-coordinate of each blob, of the image-axis-aligned bounding box. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BOX_X_MIN	Specifies that the following is used as the sorting key: the extreme left X-coordinate of each blob, of the image-axis-aligned bounding box. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BOX_Y_MAX	Specifies that the following is used as the sorting key: the extreme bottom Y-coordinate of each blob, of the image-axis-aligned bounding box. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BOX_Y_MIN	Specifies that the following is used as the sorting key: the extreme top Y-coordinate of each blob, of the image-axis-aligned bounding box. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BREADTH	Specifies that the following is used as the sorting key: the breadth of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_CENTER_OF_GRAVITY_X +	Specifies that the following is used as the sorting key: the X-position of the center of gravity of each blob. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	M_CENTER_OF_GRAVITY_Y +	Specifies that the following is used as the sorting key: the Y-position of the center of gravity of each blob. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPACTNESS	Specifies that the following is used as the sorting key: the compactness of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CONVEX_HULL_AREA	Specifies that the following is used as the sorting key: the area of the convex hull of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CONVEX_HULL_COG_X	Specifies that the following is used as the sorting key: the X-component of the center of gravity of the convex hull of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CONVEX_HULL_COG_Y	Specifies that the following is used as the sorting key: the Y-component of the center of gravity of the convex hull of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CONVEX_HULL_FILL_RATIO	Specifies that the following is used as the sorting key: the ratio between the blob's area and the area of its convex hull. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_CONVEX_HULL_PERIMETER	Specifies that the following is used as the sorting key: the perimeter of the convex hull of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CONVEX_PERIMETER	Specifies that the following is used as the sorting key: an approximation of the perimeter of the convex hull of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEPTH_MAP_MAX_ELEVATION	Specifies that the following is used as the sorting key: the maximum elevation of each blob.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEPTH_MAP_MEAN_ELEVATION	Specifies that the following is used as the sorting key: the mean elevation of each blob.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEPTH_MAP_MIN_ELEVATION	Specifies that the following is used as the sorting key: the minimum elevation of each blob.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEPTH_MAP_SIZE_Z	Specifies that the following is used as the sorting key: the Z-size (difference between maximum and minimum elevation) of each blob.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEPTH_MAP_VOLUME	Specifies that the following is used as the sorting key: the volume of each blob.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ELONGATION	Specifies that the following is used as the sorting key: a value that is equal to M_LENGTH / M_BREADTH of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_EULER_NUMBER	Specifies that the following is used as the sorting key: the number of blobs minus the number of holes (number of blobs - number of holes). INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_FERET_AT_PRINCIPAL_AXIS_ANGLE +	Specifies that the following is used as the sorting key: the Feret diameter at the principal axis of each blob. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_AT_SECONDARY_AXIS_ANGLE +	Specifies that the following is used as the sorting key: the Feret diameter at the secondary axis of each blob. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_ELONGATION	Specifies that the following is used as the sorting key: the measure of the shape of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_GENERAL	Specifies that the following is used as the sorting key: the Feret diameter at the user-specified angle (set using MblobControl() with M_FERET_GENERAL_ANGLE). INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_MAX_ANGLE	Specifies that the following is used as the sorting key: the angle, in degrees, at which the maximum Feret diameter is found for each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_MAX_DIAMETER	Specifies that the following is used as the sorting key: the largest Feret diameter found after checking a certain number of angles. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_MAX_DIAMETER_ELONGATION	Specifies that the following is used as the sorting key: the ratio between the maximum Feret diameter of each blob and its perpendicular Feret diameter. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_FERET_MEAN_DIAMETER	Specifies that the following is used as the sorting key: the average of the Feret diameters at the angles checked for each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_MIN_ANGLE	Specifies that the following is used as the sorting key: the angle, in degrees, at which the minimum Feret diameter is found for each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_MIN_DIAMETER	Specifies that the following is used as the sorting key: the minimum Feret diameter found after checking a certain number of angles. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_MIN_DIAMETER_ELONGATION	Specifies that the following is used as the sorting key: the ratio between the minimum Feret diameter of each blob and its perpendicular Feret diameter. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_PERPENDICULAR_TO_MAX_DIAMETER	Specifies that the following is used as the sorting key: the Feret diameter that is perpendicular to the maximum Feret diameter of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_PERPENDICULAR_TO_MIN_DIAMETER	Specifies that the following is used as the sorting key: the Feret diameter that is perpendicular to the minimum Feret diameter of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_PRINCIPAL_AXIS_ELONGATION +	Specifies that the following is used as the sorting key: the ratio between the Feret diameter at the principal axis and the Feret diameter at the secondary axis of each blob. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b

	M_FERET_X	Specifies that the following is used as the sorting key: the dimension of the image-axis-aligned minimum bounding box of each blob, along the X-axis of the pixel coordinate system (that is, M_BOX_X_MAX - M_BOX_X_MIN + 1). INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_Y	Specifies that the following is used as the sorting key: the dimension of the image-axis-aligned minimum bounding box of each blob, along the Y-axis of the pixel coordinate system (that is, M_BOX_Y_MAX - M_BOX_Y_MIN + 1). INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FIRST_POINT_X	Specifies that the following is used as the sorting key: the X-coordinate of a unique point (along with M_FIRST_POINT_Y) for each blob, that is on the perimeter of the blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FIRST_POINT_Y	Specifies that the following is used as the sorting key: the Y-coordinate of a unique point (along with M_FIRST_POINT_X) for each blob, that is on the perimeter of the blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTERCEPT_0	Specifies that the following is used as the sorting key: the number of times a transition from background to foreground (not vice versa) occurs in the horizontal direction for the entire blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTERCEPT_45	Specifies that the following is used as the sorting key: the number of times that the neighborhood configuration $\begin{bmatrix} \bullet & F \\ B & \bullet \end{bmatrix}$ occurs in a blob, where *F* is a foreground pixel, *B* is a background pixel, and a dot can be any pixel value. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTERCEPT_90	Specifies that the following is used as the sorting key: the number of times that the neighborhood configuration $\begin{bmatrix} F \\ B \end{bmatrix}$ occurs in a blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_INTERCEPT_135	Specifies that the following is used as the sorting key: the number of times that the neighborhood configuration $\begin{bmatrix} F & \bullet \\ \bullet & B \end{bmatrix}$ occurs in a blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_LABEL_VALUE	Specifies that the following is used as the sorting key: the label value of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_LENGTH	Specifies that the following is used as the sorting key: the length of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MAX_PIXEL	Specifies that the following is used as the sorting key: the maximum pixel value found in each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MEAN_PIXEL	Specifies that the following is used as the sorting key: the mean pixel value in each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_AREA_BOX_ANGLE	Specifies that the following is used as the sorting key: the angle of the minimum-area bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_AREA_BOX_AREA	Specifies that the following is used as the sorting key: the area of the minimum-area bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_AREA_BOX_CENTER_X	Specifies that the following is used as the sorting key: the X-coordinate of the center of the minimum-area bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_AREA_BOX_CENTER_Y	Specifies that the following is used as the sorting key: the Y-coordinate of the center of the minimum-area bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_AREA_BOX_HEIGHT	Specifies that the following is used as the sorting key: the height (shortest side) of the minimum-area bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_MIN_AREA_BOX_PERIMETER	Specifies that the following is used as the sorting key: the perimeter of the minimum-area bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_AREA_BOX_WIDTH	Specifies that the following is used as the sorting key: the width (longest side) of the minimum-area bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_PERIMETER_BOX_ANGLE	Specifies that the following is used as the sorting key: the angle of the minimum-perimeter bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_PERIMETER_BOX_AREA	Specifies that the following is used as the sorting key: the area of the minimum-perimeter bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_PERIMETER_BOX_CENTER_X	Specifies that the following is used as the sorting key: the X-coordinate of the center of the minimum-perimeter bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_PERIMETER_BOX_CENTER_Y	Specifies that the following is used as the sorting key: the Y-coordinate of the center of the minimum-perimeter bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_PERIMETER_BOX_HEIGHT	Specifies that the following is used as the sorting key: the height (shortest side) of the minimum-perimeter bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_PERIMETER_BOX_PERIMETER	Specifies that the following is used as the sorting key: the perimeter of the minimum-perimeter bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_PERIMETER_BOX_WIDTH	Specifies that the following is used as the sorting key: the width (longest side) of the minimum-perimeter bounding box of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_MIN_PIXEL	Specifies that the following is used as the sorting key: the minimum pixel value found in each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_CENTRAL_X0_Y2 +	Specifies that the following is used as the sorting key: the central moment of each blob where the order of X equals 0 and the order of Y equals 2. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_CENTRAL_X0_Y3 +	Specifies that the following is used as the sorting key: the central moment of each blob where the order of X equals 0 and the order of Y equals 3. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_CENTRAL_X1_Y1 +	Specifies that the following is used as the sorting key: the central moment of each blob where the order of X equals 1 and the order of Y equals 1. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b

	M_MOMENT_CENTRAL_X1_Y2 +	Specifies that the following is used as the sorting key: the central moment of each blob where the order of X equals 1 and the order of Y equals 2. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_CENTRAL_X2_Y0 +	Specifies that the following is used as the sorting key: the central moment of each blob where the order of X equals 2 and the order of Y equals 0. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_CENTRAL_X2_Y1 +	Specifies that the following is used as the sorting key: the central moment of each blob where the order of X equals 2 and the order of Y equals 1. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_CENTRAL_X3_Y0 +	Specifies that the following is used as the sorting key: the central moment of each blob where the order of X equals 3 and the order of Y equals 0. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b

	M_MOMENT_GENERAL +	Specifies that the following is used as the sorting key: the moment calculation, based on the settings specified using MblobControl() with M_MOMENT_GENERAL_MODE, M_MOMENT_GENERAL_ORDER_X, and M_MOMENT_GENERAL_ORDER_Y. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_HU_*n* +	Specifies that the following is used as the sorting key: the nth Hu moment of each blob. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_X0_Y1 +	Specifies that the following is used as the sorting key: the ordinary moment of each blob where the order of X equals 0 and the order of Y equals 1. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_X0_Y2 +	Specifies that the following is used as the sorting key: the ordinary moment of each blob where the order of X equals 0 and the order of Y equals 2. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b

	M_MOMENT_X0_Y3 +	Specifies that the following is used as the sorting key: the ordinary moment of each blob where the order of X equals 0 and the order of Y equals 3. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_X1_Y0 +	Specifies that the following is used as the sorting key: the ordinary moment of each blob where the order of X equals 1 and the order of Y equals 0. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_X1_Y1 +	Specifies that the following is used as the sorting key: the ordinary moment of each blob where the order of X equals 1 and the order of Y equals 1. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_X1_Y2 +	Specifies that the following is used as the sorting key: the ordinary moment of each blob where the order of X equals 1 and the order of Y equals 2. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b

	M_MOMENT_X2_Y0 +	Specifies that the following is used as the sorting key: the ordinary moment of each blob where the order of X equals 2 and the order of Y equals 0. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_X2_Y1 +	Specifies that the following is used as the sorting key: the ordinary moment of each blob where the order of X equals 2 and the order of Y equals 1. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	M_MOMENT_X3_Y0 +	Specifies that the following is used as the sorting key: the ordinary moment of each blob where the order of X equals 3 and the order of Y equals 0. INFO This value is combined with a value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	M_NO_SORT	Specifies that the sorting key is removed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NUMBER_OF_CHAINED_PIXELS	Specifies that the following is used as the sorting key: the number of chained pixels in each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_NUMBER_OF_CONVEX_HULL_POINTS	Specifies that the following is used as the sorting key: the number of points on the convex perimeter of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_NUMBER_OF_HOLES	Specifies that the following is used as the sorting key: the number of holes in each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_NUMBER_OF_RUNS	Specifies that the following is used as the sorting key: the total number of runs in each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PERIMETER	Specifies that the following is used as the sorting key: the total length of edges in each blob (including the edges of any holes), whereby an allowance made for the staircase effect that is produced when diagonal edges are digitized (inside corners are counted as 1.414, rather than 2.0). INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_RECTANGULARITY	Specifies that the following is used as the sorting key: the degree to which each blob is similar to a rectangle. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ROUGHNESS	Specifies that the following is used as the sorting key: the roughness and irregularity of a blob is, and is equal to M_PERIMETER / M_CONVEX_PERIMETER. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SIGMA_PIXEL	Specifies that the following is used as the sorting key: the standard deviation of pixel values in each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SUM_PIXEL	Specifies that the following is used as the sorting key: the sum of all pixel values in each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SUM_PIXEL_SQUARED	Specifies that the following is used as the sorting key: the sum of the squares of each pixel value in each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD_BOX_X_MAX	Specifies that the following is used as the sorting key: the extreme right X-coordinate of the world-axis-aligned bounding box, of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_WORLD_BOX_X_MIN	Specifies that the following is used as the sorting key: the extreme left X-coordinate of the world-axis-aligned bounding box, of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD_BOX_Y_MAX	Specifies that the following is used as the sorting key: the extreme bottom Y-coordinate of the world-axis-aligned bounding box, of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD_BOX_Y_MIN	Specifies that the following is used as the sorting key: the extreme top Y-coordinate of the world-axis-aligned bounding box, of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD_FERET_X	Specifies that the following is used as the sorting key: the dimension of the world-axis-aligned minimum bounding box, along the X-axis of the relative coordinate system (that is, M_WORLD_BOX_X_MAX - M_WORLD_BOX_X_MIN + 1). INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD_FERET_Y	Specifies that the following is used as the sorting key: the dimension of the world-axis-aligned minimum bounding box, along the Y-axis of the relative coordinate system (that is, M_WORLD_BOX_Y_MAX - M_WORLD_BOX_Y_MIN + 1). INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD_X_AT_Y_MAX	Specifies that the following is used as the sorting key: the X-coordinate at the maximum Y-coordinate of each blob, of the world-axis-aligned bounding box. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD_X_AT_Y_MIN	Specifies that the following is used as the sorting key: the X-coordinate at the minimum Y-coordinate of each blob, of the world-axis-aligned bounding box. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_WORLD_Y_AT_X_MAX	Specifies that the following is used as the sorting key: the Y-coordinate at the maximum X-coordinate of each blob, of the world-axis-aligned bounding box. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD_Y_AT_X_MIN	Specifies that the following is used as the sorting key: the Y-coordinate at the minimum X-coordinate of each blob, of the world-axis-aligned bounding box. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_X_MAX_AT_Y_MAX	Specifies that the following is used as the sorting key: the maximum X-coordinate at the maximum Y-coordinate of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_X_MAX_AT_Y_MIN	Specifies that the following is used as the sorting key: the maximum X-coordinate at the minimum Y-coordinate of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_X_MIN_AT_Y_MAX	Specifies that the following is used as the sorting key: the minimum X-coordinate at the maximum Y-coordinate of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_X_MIN_AT_Y_MIN	Specifies that the following is used as the sorting key: the minimum X-coordinate at the minimum Y-coordinate of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_Y_MAX_AT_X_MAX	Specifies that the following is used as the sorting key: the maximum Y-coordinate at the maximum X-coordinate of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_Y_MAX_AT_X_MIN	Specifies that the following is used as the sorting key: the maximum Y-coordinate at the minimum X-coordinate of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_Y_MIN_AT_X_MAX	Specifies that the following is used as the sorting key: the minimum Y-coordinate at the maximum X-coordinate of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_Y_MIN_AT_X_MIN	Specifies that the following is used as the sorting key: the minimum Y-coordinate at the minimum X-coordinate of each blob. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SORT*n*_DIRECTION +	Inquires the direction in which the *n* th sorting key was sorted, where *n* stands for an integer between 1 and 3. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_SORT_UP.	c		e	f	g	a		i		j	k	l	m	n	b
M_SORT_DOWN	Specifies the feature as being sorted in descending order.	c		e	f	g	a		i		j	k	l	m	n	b
M_SORT_UP	Specifies the feature as being sorted in ascending order.	c		e	f	g	a		i		j	k	l	m	n	b
M_TIMEOUT +	Inquires the maximum processing time for MblobCalculate(), in msec. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that there is no maximum processing time.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the maximum processing time, in msec.	c		e	f	g	a		i		j	k	l	m	n	b

For the following inquire types, the ContextOrResultBlobId parameter can specify a blob analysis context. The following group of features do not use grayscale pixel values; they are calculated using only the blob identifier image.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about features with only a binary definition																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																

M_BOX +		Inquires whether image-axis-aligned bounding box features will be calculated. SET														
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_BREADTH +		Inquires whether the breadth of a blob will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAINS +		Inquires whether chain features will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPACTNESS +		Inquires whether a measure of the compactness of an object will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONTACT_POINTS +		Inquires whether the contact features will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONVEX_HULL +		Inquires whether all convex hull features will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONVEX_PERIMETER +		Inquires whether the approximation of the perimeter of the convex hull of a blob will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_ELONGATION +		Inquires whether a measure of the elongation of an object will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_EULER_NUMBER +		Inquires whether the number of blobs - number of holes will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_GENERAL +		Inquires whether the user-specified Feret diameter will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_MAX_DIAMETER_ELONGATION +		Inquires whether the ratio between the maximum Feret diameter and its perpendicular Feret diameter will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_MIN_DIAMETER_ELONGATION +		Inquires whether the ratio between the minimum Feret diameter and its perpendicular Feret diameter will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_PERPENDICULAR_TO_MAX_DIAMETER +		Inquires whether the Feret diameter that is perpendicular to the maximum Feret diameter will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_PERPENDICULAR_TO_MIN_DIAMETER +		Inquires whether the Feret diameter that is perpendicular to the minimum Feret diameter will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERETS +		Inquires whether the Feret features will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERCEPT +		Inquires whether the intercept features will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

M_LENGTH +		Inquires whether the length of a blob will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_AREA_BOX +		Inquires whether all the minimum-area bounding box features will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_PERIMETER_BOX +		Inquires whether all the minimum-perimeter bounding box features will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_HOLES +		Inquires whether the number of holes in a blob will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_PERIMETER +		Inquires whether the total length of edges in a blob (including the edges of any holes) will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECTANGULARITY +		Inquires whether the degree to which a blob resembles a rectangle will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROUGHNESS +		Inquires if the roughness and irregularity of a blob will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_RUNS +		Inquires whether the blob run-length encoding information will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD_BOX +		Inquires whether all of the features related to the world-axis-aligned bounding box will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

For the following inquire types, the ContextOrResultBlobId parameter can specify a blob analysis context. The following features require grayscale pixel values, and can only be calculated if you provide a grayscale buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about features with only a grayscale definition																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																

M_BLOB_CONTRAST +	Inquires whether the difference between the maximum and minimum pixel values of a blob will be calculated. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_PIXEL +	Inquires whether the maximum pixel value in a blob will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEAN_PIXEL +	Inquires whether the mean pixel value in a blob will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_PIXEL +	Inquires whether the minimum pixel value in a blob will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

M_SIGMA_PIXEL +	Inquires whether the standard deviation of pixel values in a blob will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUM_PIXEL +	Inquires whether the sum of all pixel values in a blob will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUM_PIXEL_SQUARED +	Inquires whether the sum of the squares of each pixel value in a blob will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

For the following inquire types, the ContextOrResultBlobId parameter can specify a blob analysis context. The following features have two different definitions: a binary definition, where all pixels are considered equal, and a grayscale, where pixels are weighted by their value in the grayscale buffer (the grayscale version is much slower to calculate). If you do not provide a grayscale buffer, only the binary version can be calculated. If you do provide a grayscale buffer, both versions are calculated.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about features with two definitions (binary and grayscale)																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																

M_CENTER_OF_GRAVITY +		Inquires whether both X- and Y-coordinates of the center of gravity will be calculated. SET You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_AT_PRINCIPAL_AXIS_ANGLE +		Inquires whether the Feret diameter at the principal axis of a blob will be calculated. SET You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

M_FERET_AT_SECONDARY_AXIS_ANGLE +		Inquires whether the Feret diameter at the secondary axis of a blob will be calculated. SET You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_PRINCIPAL_AXIS_ELONGATION +		Inquires whether the ratio between the Feret diameter at the principal axis and the Feret diameter at the secondary axis will be calculated. SET You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

M_MOMENT_FIRST_ORDER +		Inquires whether the first-order moments will be calculated. SET You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_GENERAL +		Inquires whether the user-specified moment will be calculated. SET You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that this feature will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that this feature will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_SECOND_ORDER +		Inquires whether the second-order moments will be calculated. SET You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_THIRD_ORDER +		Inquires whether the third-order moments and Hu moment invariants will be calculated. SET You must specify a combination value from the following table: • To set whether the results should be returned for the binary or grayscale version of the selected feature	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that these features will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that these features will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

For the following inquire types, the ContextOrResultBlobId parameter can specify a blob analysis context. These blob features can only be calculated if you pass MblobCalculate() a grayscale buffer that is a fully corrected depth map image buffer or 3D-processable depth map container, and has an ROI that acts as the blob identifier image as well as identifies invalid pixels. If you passed an image buffer, you can verify that it contains a fully corrected depth map using McalInquire() with M_DEPTH_MAP. You can use MbufSetRegion() with M_RASTERIZE_DEPTH_MAP_VALID_PIXELS to merge confidence information with the blob identifier image. If you passed a container, you can use MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP to ensure that the container contains a 3D-processable depth map.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about depth map specific features																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_DEPTH_MAP_MAX_ELEVATION +		Inquires whether the maximum elevation of a blob will be calculated. SET															

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that the maximum elevation of the blob will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that the maximum elevation of the blob will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEPTH_MAP_MEAN_ELEVATION +		Inquires whether the mean elevation of a blob will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that the mean elevation of the blob will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that the mean elevation of the blob will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEPTH_MAP_MIN_ELEVATION +		Inquires whether the minimum elevation of a blob will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that the minimum elevation of the blob will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that the minimum elevation of the blob will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEPTH_MAP_SIZE_Z +		Inquires whether the Z-size (difference between maximum and minimum elevation) of a blob will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that the Z-size of the blob will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that the Z-size of the blob will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEPTH_MAP_VOLUME +		Inquires whether the volume of a blob will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies that the volume of the blob will not be calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that the volume of the blob will be calculated.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the following values of out-parameters: [when InquireType=M_SORTn: M_AXIS_PRINCIPAL_ANGLE; M_AXIS_SECONDARY_ANGLE; M_CENTER_OF_GRAVITY_X; M_CENTER_OF_GRAVITY_Y; M_FERET_AT_PRINCIPAL_AXIS_ANGLE; M_FERET_AT_SECONDARY_AXIS_ANGLE; M_FERET_PRINCIPAL_AXIS_ELONGATION; M_MOMENT_CENTRAL_X0_Y2; M_MOMENT_CENTRAL_X0_Y3; M_MOMENT_CENTRAL_X1_Y1; M_MOMENT_CENTRAL_X1_Y2; M_MOMENT_CENTRAL_X2_Y0; M_MOMENT_CENTRAL_X2_Y1; M_MOMENT_CENTRAL_X3_Y0; M_MOMENT_GENERAL; M_MOMENT_HU_n; M_MOMENT_X0_Y1; M_MOMENT_X0_Y2; M_MOMENT_X0_Y3; M_MOMENT_X1_Y0; M_MOMENT_X1_Y1; M_MOMENT_X1_Y2; M_MOMENT_X2_Y0; M_MOMENT_X2_Y1; M_MOMENT_X3_Y0].
Combination values for the following values of in-parameters: the values listed in For inquiring about features with two definitions (binary and grayscale).

The above-mentioned in-parameter values must be added to the following values. The above-mentioned out-parameter values are returned combined with the following values. In either case, the following values are used to set whether the results should be returned for the binary or grayscale version of the selected feature.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For feature parameters that have two definitions																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BINARY	Selects the binary definition of the selected feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAYSCALE	Selects the grayscale definition of the selected feature.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for
Tables:

- For inquiring about the global processing settings of a blob analysis context that cause results to be recalculated
- For inquiring about a global processing settings of a blob analysis context that do not cause results to be recalculated
- For inquiring about features with only a binary definition
- For inquiring about features with only a grayscale definition
- For inquiring about features with two definitions (binary and grayscale)
- For inquiring about depth map specific features

the values listed in all the parameter association tables except **For inquiring the system on which the result buffer or context is allocated**, **For inquiring about a processing setting in a blob analysis result buffer**.

You can add the following value to the above-mentioned values to determine the default value of an inquire type, regardless of the current value of the inquire type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the default value of an inquire type																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

	UserVarPtr - Possible values returned															
M_DEFAULT		Inquires the default value of the specified inquire type.														
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

For the following inquire types, the ContextOrResultBlobId parameter must specify a blob analysis result buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a processing setting in a blob analysis result buffer																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_INDEX_MODE +		Inquires the index mode that will be used to retrieve results with MblobGetResult() and to draw results with MblobDraw(). SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INDEX	Specifies that the index will be passed directly to the LabelOrIndex parameter of MblobGetResult() or MblobDraw(), without any macros.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LABEL	Specifies that the label will be passed directly to the LabelOrIndex parameter of MblobGetResult() or MblobDraw(), without any macros.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USE_MACRO	Specifies that the values passed to the LabelOrIndex parameter of MblobGetResult() or MblobDraw() can be the macros M_BLOB_INDEX() or M_BLOB_LABEL(). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_INPUT_SELECT_UNITS +		Inquires the units in which the CondLow and CondHigh parameters of MblobSelect() expect the limits of the blob selection condition. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_PIXEL	Specifies that the limit values passed to MblobSelect() be interpreted in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

M_WORLD	Specifies that the limit values passed to MblobSelect() be interpreted in world units, with respect to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_RESULT_OUTPUT_UNITS +	Inquires whether results are returned in pixel or world units. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_CALIBRATION	Specifies that results are returned in world units if the result was calculated on an image associated with a camera calibration context; otherwise, specifies that results are returned in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies that results are returned in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies that results are returned in world units, with respect to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for
Tables:

- For inquiring the system on which the result buffer or context is allocated
- For inquiring about the global processing settings of a blob analysis context that cause results to be recalculated
- For inquiring about a global processing settings of a blob analysis context that do not cause results to be recalculated
- For inquiring about features with only a binary definition
- For inquiring about features with only a grayscale definition
- For inquiring about features with two definitions (binary and grayscale)
- For inquiring about depth map specific features
- For inquiring about a processing setting in a blob analysis result buffer

the values listed in all the parameter association tables.

You can add one of the following values to the above-mentioned values to cast the requested information to the required data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested information to an *AIL_DOUBLE*.															

UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested information to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested information to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested information to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested information to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailblob11.lib.
DLL	Requires ail11.dll; ailblob11.dll.

MblobLabel

Synopsis

Draw a labeled image.

Syntax

```
void MblobLabel(
    AIL_ID ResultBlobId,  //in
    AIL_ID DstImageBufId, //out
    AIL_INT64 ControlFlag //in
)
```

Description

This function draws a labeled image in which each blob existing in the specified result buffer is represented with its own unique label value.

The label values are taken from the blob analysis result buffer. A call to MblobCalculate() must have been made to generate label values for an image. Blobs that have been deleted from the result buffer are not drawn.

Parameters

ResultBlobId

Specifies the identifier of the blob analysis result buffer.

DstImageBufId

Specifies the identifier of the destination (labeled) image buffer. This must be a single band, 8-, 16-, or 32-bit unsigned buffer.

Note, this buffer need not be the same size as the original identifier image used to calculate the blobs; however, it must be 16 or 32 bits deep if the maximum label value is between 256 and 65535 inclusively, or 32 bits if the maximum label value exceeds 65535. To determine the maximum label value, call MblobGetResult() with M_MAX_LABEL_VALUE. The number of blobs alone does not tell you the maximum label value (label values are not necessarily contiguous).

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

ControlFlag

Specifies whether or not to clear the destination image buffer before drawing the labeled image into it. This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the destination image buffer																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CLEAR	Clears the destination image buffer before placing the labeled image into it.	c		e	f	g	a		i		j	k	l	m	n	b
M_NO_CLEAR	Does not clear the destination image buffer before placing the labeled image into it (background pixels will be unchanged).	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailblob11.lib.
DLL	Requires ail11.dll; ailblob11.dll.

MblobMerge

Synopsis

Merge the results of two blob result buffers.

Syntax

```
void MblobMerge(
    AIL_ID SrcResultBlobId1, //in
    AIL_ID SrcResultBlobId2, //in
    AIL_ID DstResultBlobId,  //out
    AIL_INT64 ControlFlag    //in
)
```

Description

This function merges the results of two blob result buffers into a single result buffer.

The source results are merged as if they were taken from two vertically adjacent child buffers of one image. As such, the destination result buffer uses the coordinate system of the first result buffer, and positional results from the second result buffer are translated in the Y-direction to take into account this coordinate system change. Border blobs that would touch if they were in one image are grouped into one grouped blob, assigned a single label, and recalculated as if the grouped blobs were one blob. Only features that were calculated for all the individual blobs in the group are recalculated for the new grouped blob; calculations are made using the results of the individual blobs in the group. See the Merging results section of Chapter 6: Blob analysis for more detailed explanations.

Note that after merging the results, you can perform an MblobSelect() operation to further include or exclude blobs in your destination result buffer, but if you perform an MblobCalculate() operation after the merge, all the results in the destination result buffer will be discarded.

The source blob results must have been calculated by a context that:

- Has the same run information saving mode (M_SAVE_RUNS set to M_ENABLE or M_DISABLE).
- Has the same pixel aspect ratio (M_PIXEL_ASPECT_RATIO).
- Has the same lattice (M_CONNECTIVITY).
- Has the same blob identification mode (M_BLOB_IDENTIFICATION_MODE). If the blob result buffers have M_BLOB_IDENTIFICATION_MODE set to M_LABELED, they cannot be merged.
- Has the same number of Ferets (M_NUMBER_OF_FERETS).

Note that if the source results include user-specified moments (added using MblobControl() with M_MOMENT_GENERAL), or third-order moments calculated in precise mode (added using MblobControl() with M_MOMENT_THIRD_ORDER set to M_ENABLE, and M_MOMENT_THIRD_ORDER_MODE set to M_PRECISE), **MblobMerge**() will only be able to merge the moments if they were calculated using their binary definition and M_SAVE_RUNS was enabled. If they were calculated using their grayscale definition or M_SAVE_RUNS was not enabled, the moments will not be merged.

Furthermore, if you specify a sorting key for result retrieval, the sorting order will not be respected after the merge.

If both source result buffers have the same camera calibration information (although their results might be from different child images), the destination result buffer will have the same camera calibration information. If either of the source result buffers is uncalibrated, or if they have different camera calibration information, the destination

result buffer will be uncalibrated.

Parameters

SrcResultBlobId1

Specifies the identifier of the first source result buffer. SrcResultBlobId1 and SrcResultBlobId2 cannot be the same.

SrcResultBlobId2

Specifies the identifier of the second source result buffer. SrcResultBlobId1 and SrcResultBlobId2 cannot be the same.

DstResultBlobId

Specifies the identifier of the destination result buffer. DstResultBlobId cannot be the same as SrcResultBlobId1 or SrcResultBlobId2.

ControlFlag

Specifies the control settings for the merge operation. This parameter can be set to any of the following.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of merge operation																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default control operation. The default value is M_COPY, in which the source results are copied into the destination result buffer, and the orientation of the merged result will be as if the source results were taken from two vertically adjacent child buffers of one image.	c		e	f	g	a		i		j	k	l	m	n	b
M_COPY	Copies the results of SrcResultBlobId1 and SrcResultBlobId2 into the destination result buffer. After the merge, the original results of SrcResultBlobId1 and SrcResultBlobId2 remain intact.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOVE	Moves the results of the source result buffers into the destination result buffer. After the merge, the original results of SrcResultBlobId1 and SrcResultBlobId2 are emptied.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailblob11.lib.
DLL	Requires ail11.dll; ailblob11.dll.

MblobReconstruct

Synopsis

Reconstruct blobs (or blob holes) in an image buffer.

Syntax

```
void MblobReconstruct(
    AIL_ID SrcImageBufId,  //in
    AIL_ID SeedImageBufId, //in
    AIL_ID DstImageBufId,  //out
    AIL_INT64 Operation,   //in
    AIL_INT64 ProcMode     //in
)
```

Description

This function copies (or reconstructs) blobs or blob holes from the source to the destination buffer, according to the specified operation and processing mode. By default, all non-zero pixels in the source buffer are considered to be part of a blob. Use the M_FOREGROUND_ZERO processing mode to inverse this behavior.

Parameters

SrcImageBufId

Specifies the identifier of the source image buffer. The source buffer must be a single band, packed binary, 8- or 16-bit unsigned buffer.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

SeedImageBufId

Specifies the identifier of the image buffer to use as a seed image. The seed image buffer must be a single band, packed binary, 8- or 16-bit unsigned buffer.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

A seed image is needed to perform an M_RECONSTRUCT_FROM_SEED type of operation. For any other operation type, set this parameter to **M_NULL**.

DstImageBufId

Specifies the identifier of the destination (processed blobs) image buffer. The destination buffer must be a single band, packed binary, 8- or 16-bit unsigned buffer.

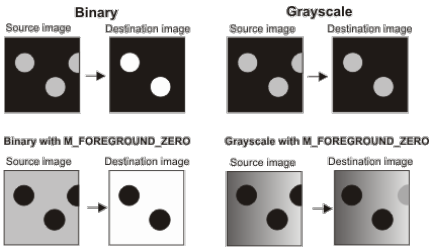
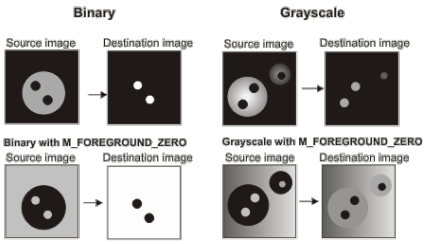
This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

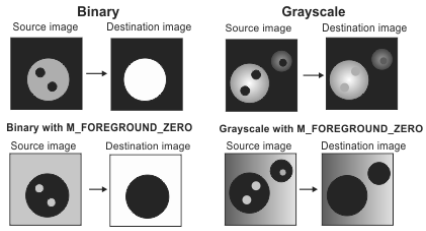
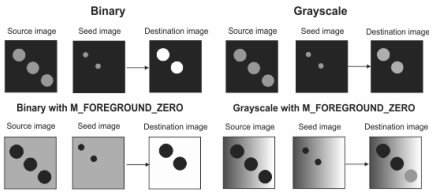
Operation

Specifies the type of operation to perform. This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of operation to perform

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ERASE_BORDER_BLOBS	All blobs that do not touch the borders of the source image are copied to the destination image buffer according to the selected processing mode. Pixels of the blobs that touch the border are replaced according to the selected processing mode.  Note that when the M_GRAYSCALE and M_FOREGROUND_ZERO processing modes are selected, the border blobs are filled with the average grayscale pixel value of the background. This operation is similar to a "border kill".	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTRACT_HOLES	All holes within the blobs of the source buffer are copied to the destination buffer with their pixel values set to their corresponding blob's pixel values, according to the selected processing mode. A hole cannot touch any image border to be considered a hole.  Note that, in the M_GRAYSCALE processing mode, the pixel values of holes copied to the destination buffer are set to the blob's average grayscale pixel value in the source buffer. Also, when the M_GRAYSCALE and M_FOREGROUND_ZERO processing modes are selected, the blobs are filled with the average grayscale pixel value of the background.	c		e	f	g	a		i		j	k	l	m	n	b

M_FILL_HOLES	All blobs in the source buffer are copied to the destination buffer according to the selected processing mode, and those blobs with holes are filled according to the processing mode. A hole must not touch the border of the image to be considered a hole.  Note that when the M_GRAYSCALE processing mode is selected, holes are filled with the average grayscale pixel value of the corresponding blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECONSTRUCT_FROM_SEED	All blobs in the source buffer that have at least one corresponding foreground seed pixel in the seed buffer are copied to the destination buffer, according to the selected processing mode. Blobs that are not seeded are replaced according to the selected processing mode.  Note that when the M_GRAYSCALE and M_FOREGROUND_ZERO processing modes are selected, blobs in the source image that are not seeded are filled with the average grayscale value of the background. Also, the value of the seed pixels must be strictly zero for this operation to be performed properly. Note that the seed image and the destination image must have identical dimensions.	c		e	f	g	a		i		j	k	l	m	n	b

ProcMode

Specifies the processing mode to use.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the processing mode to use																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default processing mode (which corresponds to M_BINARY + M_8_CONNECTED).	c		e	f	g	a		i		j	k	l	m	n	b

M_BINARY +	Specifies that non-zero pixel values will be treated as ones (1) during processing, and the resulting non-zero pixels copied to the destination buffer will be set to the maximum value of that buffer (for example, 0xff for an 8-bit buffer). Note, in general, the M_BINARY processing mode is faster.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAYSCALE +	Specifies that the values of pixels copied to the destination buffer will be changed to the values of corresponding pixels in the source buffer.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For specifying the processing mode to use.

You can add one of the following values to the above-mentioned values to set the type of connectivity (lattice).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For selecting the type of connectivity																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_4_CONNECTED	Specifies that blobs are computed on a four connected lattice.	c		e	f	g	a		i		j	k	l	m	n	b
M_8_CONNECTED	Specifies that blobs are computed on an eight connected lattice. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For specifying the processing mode to use.

You can add the following value to the above-mentioned values to set the value of the foreground pixels.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For selecting the foreground pixels																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FOREGROUND_ZERO	Specifies that the pixel values of blobs will consist of zero values and the pixels of the background will consists of non-zero values; that is, the inverse of the usual blob pixel value definition.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For specifying the processing mode to use.

You can add the following value to the above-mentioned values to set whether to accelerate blob reconstruction with seed images.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For accelerating blob reconstruction with seed images																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_SEED_PIXELS_ALL_IN_BLOBS	Optimizes the reconstruction process. Use this value in cases when all seed pixels in the seed buffer have corresponding blob pixels in the source buffer. This condition often exists when the seed buffer is an eroded (see MimErode()) version of the source buffer.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailblob11.lib.
DLL	Requires ail11.dll; ailblob11.dll.

MblobRestore

Synopsis

Restore a blob analysis context from disk.

Syntax

```
AIL_ID MblobRestore(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID SysId,                //in
    AIL_INT64 ControlFlag,       //in
    AIL_ID *ContextBlobIdPtr     //out
)
```

Description

This function restores a blob analysis context that was previously saved to a file, using MblobSave() or MblobStream(). This function restores all of the blob analysis context's settings that were in effect when it was saved.

When the restored blob analysis context is no longer required, release it using MblobFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the blob analysis context's lifetime and you must not manually free it.

Parameters

FileName

Specifies the name and path of the file from which to restore the blob analysis context. The function handles (internally) the opening and closing of the file.

This parameter should be set to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
AIL_TEXT("*FileName*") [1]	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Typically, blob analysis contexts have a MBLOB file extension. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

SysId

Specifies the identifier of the system on which to restore the blob analysis context. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ContextBlobIdPtr

Specifies the address of the variable in which to write the blob analysis context identifier or specifies the data type that the function should use to return the Blob Analysis context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated blob analysis context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated blob analysis context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_BLOB_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the blob analysis context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b

Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored blob analysis context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b	

Return value

The returned value is the blob analysis context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_BLOB_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailblob11.lib.
DLL	Requires ail11.dll; ailblob11.dll.

MblobSave

Synopsis

Save a blob context to a file.

Syntax

```
void MblobSave(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID ContextBlobId,        //in
    AIL_INT64 ControlFlag        //in
)
```

Description

This function saves all the information about the previously allocated blob context to disk, including the current enabled features. This information can be reloaded, using MblobRestore() or MblobStream().

Parameters

FileName

Specifies the name and path of the file in which to save the blob context. It is recommended that you use the MBLOB file extension for easier use with other Aurora Imaging software products. The function internally handles the opening and closing of the file. If this file already exists, it will be overwritten.

This parameter should be set to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Save As** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Typically, blob contexts have a MBLOB file extension. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

ContextBlobId

Specifies the identifier of the blob context to save.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailblob11.lib.
DLL	Requires ail11.dll; ailblob11.dll.

MblobSelect

Synopsis

Select blobs for calculations and result retrieval.

Syntax

```
void MblobSelect(
    AIL_ID ResultBlobId,          //out
    AIL_INT64 Operation,          //in
    AIL_INT64 SelectionCriterion, //in
    AIL_INT64 Condition,          //in
    AIL_DOUBLE CondLow,           //in
    AIL_DOUBLE CondHigh           //in
)
```

Description

This function selects or merges blobs that meet a specified criterion. These blobs will be included in or excluded from future operations (calculations or result retrieval), or deleted entirely from the result buffer. Selection criterion can be based on a calculated feature or on the current status of the blobs. MblobCalculate() must have been called at least once before calling this function.

If this function is not called at least once, all blobs are included by default. If there is more than one call to this function, the effect of the calls is cumulative unless M_INCLUDE_ONLY or M_EXCLUDE_ONLY is specified as the operation to perform.

If a blob is excluded or deleted, its index is reassigned; that is, the index of blobs with indices greater than that of the excluded/deleted blob are reduced by one. This ensures that included blobs have consecutive indices. Label values of included and excluded blobs, however, are left as is; therefore, it is possible to specify the label value of an excluded blob. You can get a list of valid blob label values by calling MblobGetResult() with M_LABEL_VALUE. Deleted blobs' label values are set to 0.

Once a blob has been excluded, it can normally be re-included only by specifying M_INCLUDE or M_INCLUDE_ONLY in a future call to this function (with the correct criterion). However, if you change the processing mode of a Blob Analysis context (using MblobControl()), or use the result buffer with different images, the subsequent call to MblobCalculate() will cause all results in the buffer to be discarded and all blobs will be re-included.

The limits of the blob selection criterion are set using the CondLow and CondHigh parameters. If the blobs were taken from a calibrated image, these parameters can be set in pixel units or world units. To set the units, use MblobControl() with M_INPUT_SELECT_UNITS. Note that if you set this control type to M_WORLD but you don't pass **MblobSelect**() a result buffer whose results were obtained from a calibrated target image, the function will generate an error.

Parameters

ResultBlobId

Specifies the identifier of the blob analysis result buffer to use in the blob selection process.

Operation

Specifies the operation to perform on the specified blobs. This parameter can be set to one of the following.

For specifying the operation to perform																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DELETE	Deletes blobs that meet the specified condition. M_DELETE affects only included blobs, unless otherwise stated by the SelectionCriterion parameter. M_DELETE removes blobs permanently from the result buffer and, consequently, prevents these blobs from being re-included.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXCLUDE	Excludes all blobs that meet the specified condition. M_EXCLUDE affects only the status of currently included blobs.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXCLUDE_ONLY	Excludes only those blobs that meet the specified condition and includes all others. The exclusion does not consider the present status of blobs (whether they are excluded), except for blobs that have been deleted (M_DELETE), unless otherwise stated by the SelectionCriterion parameter.	c		e	f	g	a		i		j	k	l	m	n	b
M_INCLUDE	Includes all blobs that meet the specified condition. M_INCLUDE affects only the status of currently excluded blobs.	c		e	f	g	a		i		j	k	l	m	n	b
M_INCLUDE_ONLY	Includes only those blobs that meet the specified condition and excludes all others. The inclusion does not consider the present status of blobs (whether they are included), except for blobs that have been deleted (M_DELETE), unless otherwise stated by the SelectionCriterion parameter.	c		e	f	g	a		i		j	k	l	m	n	b

M_MERGE	Groups all the blobs that meet the specified condition. Once grouped, the blobs are treated as one blob (a grouped blob). The grouped blob is assigned a unique blob label. Only features that were calculated for all the individual blobs in the group are re-calculated for the new grouped blob; calculations are made using the results of the individual blobs in the group. Note that if the source results include user-specified moments (added using MblobControl() with M_MOMENT_GENERAL) or third-order moments calculated in precise mode (added using MblobControl() with M_MOMENT_THIRD_ORDER set to M_ENABLE, and M_MOMENT_THIRD_ORDER_MODE set to M_PRECISE) M_MERGE will only be able to merge the moments if they were calculated using their binary definition and M_SAVE_RUNS was enabled. If they were calculated using their grayscale definition or M_SAVE_RUNS was not enabled, the moments will not be merged. Except when using M_EXCLUDED_BLOBS as the feature for selection, this grouping operation only applies to included blobs. If you are using M_MERGE, you cannot use MblobControl() with M_BLOB_IDENTIFICATION_MODE set to M_LABELED_TOUCHING.	c		e	f	g	a		i		j	k	l	m	n	b

SelectionCriterion

Specifies the feature to use as part of the selection criterion or the status of the blobs to affect. See MblobControl() for a full description of the features. The specified result buffer must already contain the results for the specified feature.

To use as part of the criterion a feature that has only a binary definition, select one of the following.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a selection criterion based on a feature that has only a binary definition																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_AREA	Uses the number of foreground pixels in a blob (holes are not counted).	c		e	f	g	a		i		j	k	l	m	n	b
M_BLOB_TOUCHING_IMAGE_BORDERS	Uses all blobs that touched the border of the blob identifier image.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_AREA	Uses the area covered by the bounding box of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_ASPECT_RATIO	Uses the ratio of the horizontal size to the vertical size of the bounding box of a blob.	c		e	f	g	a		i		j	k	l	m	n	b

M_BOX_FILL_RATIO	Uses the ratio of the area of a blob to the area of the bounding box of the blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_X_MAX	Uses the extreme right coordinate of the bounding box of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_X_MIN	Uses the extreme left coordinate of the bounding box of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_Y_MAX	Uses the extreme bottom coordinate of the bounding box of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_Y_MIN	Uses the extreme top coordinate of the bounding box of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_BREADTH	Uses the breadth of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPACTNESS	Uses the compactness feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONVEX_HULL_AREA	Uses the area of the convex hull of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONVEX_HULL_COG_X	Uses the X-component of the center of gravity of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONVEX_HULL_COG_Y	Uses the Y-component of the center of gravity of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONVEX_HULL_FILL_RATIO	Uses the ratio of the area of a blob to the area of its convex hull.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONVEX_HULL_PERIMETER	Uses the perimeter of the convex hull of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONVEX_PERIMETER	Uses the approximation of the perimeter of the convex hull of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_ELONGATION	Uses the elongation of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_EULER_NUMBER	Uses the number of blobs - number of holes.	c		e	f	g	a		i		j	k	l	m	n	b

M_FERET_ELONGATION	Uses the measure of the shape of a blob. It is equal to M_FERET_MAX_DIAMETER / M_FERET_MIN_DIAMETER.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_GENERAL	Uses the Feret diameter feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_MAX_ANGLE	Uses the angle at which the maximum Feret diameter is found, in degrees, relative to the input coordinate system specified using MblobControl() with M_INPUT_SELECT_UNITS. An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_MAX_DIAMETER	Uses the largest Feret diameter found after checking a certain number of angles.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_MAX_DIAMETER_ELONGATION	Uses the ratio of the maximum Feret diameter by its perpendicular Feret diameter.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_MEAN_DIAMETER	Uses the average Feret diameter at all the angles checked.	c		e	f	g	a		i		j	k	l	m	n	b

M_FERET_MIN_ANGLE	Uses the angle at which the minimum Feret diameter is found, in degrees, relative to the input coordinate system specified using MblobControl() with M_INPUT_SELECT_UNITS. An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_MIN_DIAMETER	Uses the smallest Feret diameter found after checking a certain number of angles.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_MIN_DIAMETER_ELONGATION	Uses the ratio of the minimum Feret diameter by its perpendicular Feret diameter.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_PERPENDICULAR_TO_MAX_DIAMETER	Uses the Feret diameter that is perpendicular to the maximum Feret diameter.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_PERPENDICULAR_TO_MIN_DIAMETER	Uses the Feret diameter that is perpendicular to the minimum Feret diameter.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_X	Uses the dimension of the minimum bounding box of a blob in the horizontal direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_Y	Uses the dimension of the minimum bounding box of a blob in the vertical direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIRST_POINT_X	Uses the x-coordinate of the top left-most pixel along the perimeter of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIRST_POINT_Y	Uses the y-coordinate of the top left-most pixel along the perimeter of a blob.	c		e	f	g	a		i		j	k	l	m	n	b

M_INDEX_VALUE	Uses the index value.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERCEPT_0	Uses the number of times a transition from background to foreground (not vice versa) occurs in the horizontal direction for the entire blob. In other words, it is equal to the number of times the neighborhood configuration [*B*, *F*] occurs in a blob, where *B* is a background pixel and *F* is a foreground pixel.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERCEPT_45	Determines the number of times that the neighborhood configuration $\begin{bmatrix} \bullet & F \\ B & \bullet \end{bmatrix}$ occurs in a blob, where *F* is a foreground pixel, *B* is a background pixel, and a dot can be any pixel value.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERCEPT_90	Determines the number of times that the neighborhood configuration $\begin{bmatrix} F \\ B \end{bmatrix}$ occurs in a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERCEPT_135	Determines the number of times that the neighborhood configuration $\begin{bmatrix} F & \bullet \\ \bullet & B \end{bmatrix}$ occurs in a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_LABEL_VALUE	Uses the label feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_LENGTH	Uses the measure of the true length of a blob.	c		e	f	g	a		i		j	k	l	m	n	b

M_MIN_AREA_BOX_ANGLE	Uses the angle of the minimum-area bounding box. An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_AREA_BOX_AREA	Uses the area of the minimum-area bounding box.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_AREA_BOX_CENTER_X	Uses the X-coordinate of the center of the minimum-area bounding box.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_AREA_BOX_CENTER_Y	Uses the Y-coordinate of the center of the minimum-area bounding box.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_AREA_BOX_HEIGHT	Uses the height of the minimum-area bounding box.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_AREA_BOX_PERIMETER	Uses the perimeter of the minimum-area bounding box.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_AREA_BOX_WIDTH	Uses the width of the minimum-area bounding box.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_PERIMETER_BOX_ANGLE	Uses the angle of the minimum-perimeter bounding box. An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.	c		e	f	g	a		i		j	k	l	m	n	b

M_MIN_PERIMETER_BOX_AREA	Uses the area of the minimum-perimeter bounding box.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_PERIMETER_BOX_CENTER_X	Uses the X-coordinate of the center of the minimum-perimeter bounding box.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_PERIMETER_BOX_CENTER_Y	Uses the Y-coordinate of the center of the minimum-perimeter bounding box.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_PERIMETER_BOX_HEIGHT	Uses the height of the minimum-perimeter bounding box.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_PERIMETER_BOX_PERIMETER	Uses the perimeter of the minimum-perimeter bounding box.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_PERIMETER_BOX_WIDTH	Uses the width of the minimum-perimeter bounding box.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_GENERAL	Uses the moment calculation.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_CHAINED_PIXELS	Uses the number of chained pixels in a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_HOLES	Uses the number of holes in a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_RUNS	Uses the total number of runs in a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_PERIMETER	Uses the total length of edges in a blob (including the edges of any holes), with an allowance made for the staircase effect that is produced when diagonal edges are digitized.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECTANGULARITY	Uses the degree to which a blob is similar to a rectangle. To do this, **MblobSelect**() calculates the ratio of the blob's area to the product of its minimum Feret diameter and the Feret diameter perpendicular to the minimum Feret diameter.	c		e	f	g	a		i		j	k	l	m	n	b

M_ROUGHNESS	Uses the roughness feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD_BOX_X_MAX	Uses the extreme right X-coordinate of a blob, calculated in the relative coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD_BOX_X_MIN	Uses the extreme left X-coordinate of a blob, calculated in the relative coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD_BOX_Y_MAX	Uses the extreme bottom Y-coordinate of a blob, calculated in the relative coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD_BOX_Y_MIN	Uses the extreme top Y-coordinate of a blob, calculated in the relative coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD_FERET_X	Uses the dimension of the minimum bounding box of a blob in the horizontal direction, calculated in the relative coordinate system (that is, M_WORLD_BOX_X_MAX - M_WORLD_BOX_X_MIN + 1).	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD_FERET_Y	Uses the dimension of the minimum bounding box of a blob in the vertical direction, calculated in the relative coordinate system (that is, M_WORLD_BOX_Y_MAX - M_WORLD_BOX_Y_MIN + 1).	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD_X_AT_Y_MAX	Uses the X-coordinate at the maximum Y-coordinate of a blob, calculated in the relative coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD_X_AT_Y_MIN	Uses the X-coordinate at the minimum Y-coordinate of a blob, calculated in the relative coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD_Y_AT_X_MAX	Uses the Y-coordinate at the maximum X-coordinate of a blob, calculated in the relative coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

M_WORLD_Y_AT_X_MIN	Uses the Y-coordinate at the minimum X-coordinate of a blob, calculated in the relative coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_X_MAX_AT_Y_MAX	Uses the maximum X-coordinate at the maximum Y-coordinate of the blob, calculated in the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_X_MAX_AT_Y_MIN	Uses the maximum X-coordinate at the minimum Y-coordinate of the blob, calculated in the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_X_MIN_AT_Y_MAX	Uses the minimum X-coordinate at the maximum Y-coordinate of the blob, calculated in the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_X_MIN_AT_Y_MIN	Uses the minimum X-coordinate at the minimum Y-coordinate of the blob, calculated in the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_Y_MAX_AT_X_MAX	Uses the maximum Y-coordinate at the maximum X-coordinate of the blob, calculated in the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_Y_MAX_AT_X_MIN	Uses the maximum Y-coordinate at the minimum X-coordinate of the blob, calculated in the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_Y_MIN_AT_X_MAX	Uses the minimum Y-coordinate at the maximum X-coordinate of the blob, calculated in the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_Y_MIN_AT_X_MIN	Uses the minimum Y-coordinate at the minimum X-coordinate of the blob, calculated in the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

To use as part of the criterion a feature that has only a grayscale definition, select one of the following. You can specify one of these features as part of your selection criterion only if both a blob identifier image and a grayscale buffer (or a grayscale buffer with an ROI that acts as the blob identifier image) were passed to MblobCalculate() (and the features were selected for calculation).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a selection criterion based on a feature that has only a grayscale definition																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BLOB_CONTRAST	Uses the difference between the maximum and minimum pixel values of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEPTH_MAP_MAX_ELEVATION	Uses the feature specifying the maximum elevation of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEPTH_MAP_MEAN_ELEVATION	Uses the feature specifying the mean elevation of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEPTH_MAP_MIN_ELEVATION	Uses the feature specifying the minimum elevation of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEPTH_MAP_SIZE_Z	Uses the feature specifying the Z-size (difference between maximum and minimum elevation) of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEPTH_MAP_VOLUME	Uses the feature specifying the volume of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_PIXEL	Uses the maximum pixel value in a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEAN_PIXEL	Uses the mean pixel value in a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_PIXEL	Uses the minimum pixel value in a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGMA_PIXEL	Uses the standard deviation of pixel values in a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUM_PIXEL	Uses the sum of all pixel values in a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUM_PIXEL_SQUARED	Uses the sum of the squares of the pixel values in a blob.	c		e	f	g	a		i		j	k	l	m	n	b

To use as part of the criterion a feature that has two different definitions (a binary and a grayscale definition), select one of the following.

If you did not provide both a blob identifier image and a grayscale buffer to MblobCalculate(), only the binary version was calculated. If you did provide a grayscale buffer, both versions were calculated, unless otherwise specified. In the latter case, see combination values below to specify which to use as part of the criterion. If both versions were calculated and no version is specified, then the grayscale version of the feature is selected as the criterion.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a selection criterion based on a feature that has two different definitions																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_AXIS_PRINCIPAL_ANGLE +	Uses the feature specifying the angle, in degrees, at which a blob has the least moment of inertia, relative to the input coordinate system specified using MblobControl() with M_INPUT_SELECT_UNITS. An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.	c		e	f	g	a		i		j	k	l	m	n	b
M_AXIS_SECONDARY_ANGLE +	Uses the feature specifying the angle, in degrees, perpendicular to M_AXIS_PRINCIPAL_ANGLE, relative to the input coordinate system specified using MblobControl() with M_INPUT_SELECT_UNITS. An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTER_OF_GRAVITY_X +	Uses the feature specifying the X-position of the center of gravity of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTER_OF_GRAVITY_Y +	Uses the feature specifying the Y-position of the center of gravity of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_AT_PRINCIPAL_AXIS_ANGLE +	Uses the Feret diameter at the principal axis of a blob. The principal axis is the axis at which the blob has the least moment of inertia. Also, if the blob is symmetrical, the principal axis is aligned with the blob's axis of symmetry.	c		e	f	g	a		i		j	k	l	m	n	b

M_FERET_AT_SECONDARY_AXIS_ANGLE +	Uses the Feret diameter at the secondary axis of a blob. The secondary axis is perpendicular to the principal axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_FERET_PRINCIPAL_AXIS_ELONGATION +	Uses the ratio of the Feret diameter at the principal axis to the Feret diameter at the secondary axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_CENTRAL_X0_Y2 +	Uses the feature specifying the central moment of a blob, where the order of X equals 0 and the order of Y equals 2.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_CENTRAL_X0_Y3 +	Uses the feature specifying the central moment of a blob, where the order of X equals 0 and the order of Y equals 3.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_CENTRAL_X1_Y1 +	Uses the feature specifying the central moment of a blob, where the order of X equals 1 and the order of Y equals 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_CENTRAL_X1_Y2 +	Uses the feature specifying the central moment of a blob, where the order of X equals 1 and the order of Y equals 2.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_CENTRAL_X2_Y0 +	Uses the feature specifying the central moment of a blob, where the order of X equals 2 and the order of Y equals 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_CENTRAL_X2_Y1 +	Uses the feature specifying the central moment of a blob, where the order of X equals 2 and the order of Y equals 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_CENTRAL_X3_Y0 +	Uses the feature specifying the central moment of a blob, where the order of X equals 3 and the order of Y equals 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_HU_1 +	Uses the feature specifying the first Hu moment of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_HU_2 +	Uses the feature specifying the second Hu moment of a blob.	c		e	f	g	a		i		j	k	l	m	n	b

M_MOMENT_HU_3 +	Uses the feature specifying the third Hu moment of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_HU_4 +	Uses the feature specifying the fourth Hu moment of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_HU_5 +	Uses the feature specifying the fifth Hu moment of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_HU_6 +	Uses the feature specifying the sixth Hu moment of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_HU_7 +	Uses the feature specifying the seventh Hu moment of a blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_X0_Y1 +	Uses the feature specifying the ordinary moment of a blob, where the order of X equals 0 and the order of Y equals 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_X0_Y2 +	Uses the feature specifying the ordinary moment of a blob, where the order of X equals 0 and the order of Y equals 2.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_X0_Y3 +	Uses the feature specifying the ordinary moment of a blob, where the order of X equals 0 and the order of Y equals 3.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_X1_Y0 +	Uses the feature specifying the ordinary moment of a blob, where the order of X equals 1 and the order of Y equals 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_X1_Y1 +	Uses the feature specifying the ordinary moment of a blob, where the order of X equals 1 and the order of Y equals 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_X1_Y2 +	Uses the feature specifying the ordinary moment of a blob, where the order of X equals 1 and the order of Y equals 2.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_X2_Y0 +	Uses the feature specifying the ordinary moment of a blob, where the order of X equals 2 and the order of Y equals 0.	c		e	f	g	a		i		j	k	l	m	n	b

M_MOMENT_X2_Y1 +	Uses the feature specifying the ordinary moment of a blob, where the order of X equals 2 and the order of Y equals 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_X3_Y0 +	Uses the feature specifying the ordinary moment of a blob, where the order of X equals 3 and the order of Y equals 0.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For a selection criterion based on a feature that has two different definitions.

You can add one of the following values to the above-mentioned values to set whether to use the binary or grayscale version of the selected feature.

Note that the selected feature must have both a binary definition and a grayscale definition, and both versions must have been calculated.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For features with a binary and grayscale definition (both have been calculated)																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BINARY	Uses the binary version of the selected feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAYSCALE	Uses the grayscale version of the selected feature. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

To use as part of the selection criterion the current status of the blobs, set the SelectionCriterion parameter to one of the following values. For example, you can delete all currently excluded blobs from the list of blobs on which to operate.

When the selection criterion is based on the current status of blobs, you must set the Condition, CondLow, and CondHigh parameters to **M_NULL**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a selection criterion based on the current status of blobs																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALL_BLOBS	Uses all blobs.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXCLUDED_BLOBS	Uses all excluded blobs.	c		e	f	g	a		i		j	k	l	m	n	b
M_INCLUDED_BLOBS	Uses all included blobs.	c		e	f	g	a		i		j	k	l	m	n	b

Condition

Specifies the condition for the blob selection. This parameter must be set to one of the values below.

When the selection criterion is based on the current status of blobs or M_BLOB_TOUCHING_IMAGE_BORDERS, set the Condition parameter to **M_NULL**.

When the selection criterion is based on a feature (except M_BLOB_TOUCHING_IMAGE_BORDERS), you can specify one of the following conditions that use two limits (CondLow and CondHigh).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For blob selection using two limits																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_IN_RANGE	Specifies that blobs with values for the specified feature in the range CondLow to CondHigh, inclusive, are included, excluded, or deleted from future operations on the specified result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_OUT_RANGE	Specifies that blobs with values for the specified feature less than CondLow, or greater than CondHigh, are included, excluded, or deleted from future operations on the specified result buffer.	c		e	f	g	a		i		j	k	l	m	n	b

When the selection criterion is based on a feature (except M_BLOB_TOUCHING_IMAGE_BORDERS), you can specify one of the following conditions that use only one limit (CondLow).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For blob selection using only one limit																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_EQUAL	Specifies that blobs with values for the specified feature equal to CondLow are included, excluded, or deleted from future operations on the specified result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_GREATER	Specifies that blobs with values for the specified feature greater than CondLow are included, excluded, or deleted from future operations on the specified result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_GREATER_OR_EQUAL	Specifies that blobs with values for the specified feature greater than or equal to CondLow are included, excluded, or deleted from future operations on the specified result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_LESS	Specifies that blobs with values for the specified feature less than CondLow are included, excluded, or deleted from future operations on the specified result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_LESS_OR_EQUAL	Specifies that blobs with values for the specified feature less than or equal to CondLow are included, excluded, or deleted from future operations on the specified result buffer.	c		e	f	g	a		i		j	k	l	m	n	b

M_NOT_EQUAL	Specifies that blobs with values for the specified feature not equal to CondLow are included, excluded, or deleted from future operations on the specified result buffer.	c		e	f	g	a		i		j	k	l	m	n	b

CondLow

Specifies the lower limit of the selected condition.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the lower limit of the selected condition																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that this parameter is not applicable. Specify this value when and only when the selection criterion is based on the status of blobs or M_BLOB_TOUCHING_IMAGE_BORDERS.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the lower limit of the condition, relative to the input coordinate system specified using MblobControl() with M_INPUT_SELECT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b

CondHigh

Specifies the upper limit of the selected condition.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the upper limit of the selected condition																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that this parameter is not applicable. Specify this value when: the condition uses only one limit, the selection criterion is based on the status of blobs, or the selection criterion is based on M_BLOB_TOUCHING_IMAGE_BORDERS.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the upper limit of the condition, relative to the input coordinate system specified using MblobControl() with M_INPUT_SELECT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailblob11.lib.
DLL	Requires ail11.dll; ailblob11.dll.

MblobStream

Synopsis

Load, restore, or save an Aurora Imaging Library blob context from/to a file or memory stream.

Syntax

```
void MblobStream(
    AIL_TEXT_PTR MemPtrOrFileName, //in-out
    AIL_ID SysId,                  //in
    AIL_INT64 Operation,           //in
    AIL_INT64 StreamType,          //in
    AIL_DOUBLE Version,            //in
    AIL_INT64 ControlFlag,         //in
    AIL_ID *ContextBlobIdPtr,      //in-out
    AIL_INT *SizeByteVarPtr        //out
)
```

Description

This function can load, restore, or save a blob context from/to a file or memory stream.

To inquire the number of bytes necessary to save a blob context, you should first call this function (**MblobStream**()) with M_INQUIRE_SIZE_BYTE.

The content saved to memory stream is equivalent to the content saved to file. In addition, any file saved using this function is equivalent to a file saved with MblobSave().

You can use this and other Aurora Imaging Library stream functions, for example, to save all required Aurora Imaging Library objects, as well as any other custom data, for your application to a memory stream. Once in a memory stream, you can write the stream to a single file or transfer it over a network. You are responsible for concatenating the streams and for saving the stream to file. All information about the previously allocated blob context is saved.

All of the blob context's settings that were in effect when the blob context was saved will be restored.

Using **MblobStream**(), you can choose to save a backwards-compatible version of the blob context, which will work using a version of Aurora Imaging Library that is up to one major release older than the current version (depending on which version is specified). However, all settings and features unique to the higher version will be ignored when restored using the lower version. Besides saving backwards-compatible versions, you can also load or restore blob contexts saved using any Aurora Imaging Library version with which this function was available. Settings that do not exist in the lower version will be filled with default values when the blob context is loaded or restored.

Parameters

MemPtrOrFileName
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- Optionally, in C++: a reference to a std::vector of type AIL_UINT8

Specifies the file or memory stream.

See the Parameter associations section for possible values that can be returned/specified.

SysId

Specifies the system on which to restore the blob context.

See the Parameter associations section for possible values that can be specified.

Operation

Specifies the operation to perform.

See the Parameter associations section for possible values that can be specified.

StreamType

Specifies the type of stream in which to store/from which to restore the blob context. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the type of stream																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FILE	Specifies a file stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEMORY	Specifies a memory stream. You are responsible for allocating a block of memory for the stream.	c		e	f	g	a		i		j	k	l	m	n	b

Version

Specifies the Aurora Imaging Library version of the blob context.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ContextBlobIdPtr

Specifies the address of the variable in which to write or from which to read the identifier of the blob context.

See the Parameter associations section for possible values that can be returned/specified.

SizeByteVarPtr

Specifies the address of the variable in which to write the size of the blob context, in bytes.

If the size is not required, you can set this parameter to **M_NULL**.

> The size of the blob context varies depending on the Aurora Imaging Library version that you specify.

Parameter associations

The table below lists possible values for the **MemPtrOrFileName**, **SysId**, **Operation**, **Version**, and **ContextBlobIdPtr** parameters.

- For performing the stream operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For performing the stream operation.

Operation	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
MemPtrOrFileName - Possible values passed or returned																
SysId																
Version																
ContextBlobIdPtr - Possible values passed or returned																
M_INQUIRE_SIZE_BYTE	Inquires the number of bytes required to save a blob context to memory stream. This operation is not supported when the StreamType parameter is set to M_FILE.															
MemPtrOrFileName	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	Specifies the Aurora Imaging Library version with which the blob context must be compatible when saved. The size of a blob context might differ from one Aurora Imaging Library version to another.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
ContextBlobIdPtr	Specifies the address of the variable containing the identifier of the blob context.	c		e	f	g	a		i		j	k	l	m	n	b

M_LOAD	Loads the content of a specified file or memory stream into a previously allocated blob context.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream from which to load the context.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. Blob contexts typically have a MBLOB file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ContextBlobIdPtr	Specifies the address of the variable containing the identifier of the blob context. All previous settings are overwritten with those of the loaded context, or are set to the default value if the settings didn't exist in the version of Aurora Imaging Library from which the context is being loaded.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESTORE	Restores a blob context from a file or memory stream and assigns it an Aurora Imaging Library identifier.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream from which to restore the context.	c		e	f	g	a		i		j	k	l	m	n	b

	M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
	AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. Blob contexts typically have a MBLOB file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId		Specifies the system on which to allocate the blob context.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
	System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
Version		This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ContextBlobIdPtr		Specifies the address of the variable in which to write the identifier of the blob context. If the operation is not successful, **M_NULL** is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE		Saves a blob context to a specified file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName		Specifies the file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b

		M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
		AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. For easier use with other Aurora Imaging software products, when saving a blob context to a file, use the MBLOB file extension. The function handles (internally) the opening and closing of the file. If the file already exists, it will be overwritten. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
		MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which to write, and the block of memory must be large enough to stream the entire object. To determine the required size, call this function with M_INQUIRE_SIZE_BYTE. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
	SysId		This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	Version		Specifies the Aurora Imaging Library version with which the blob context must be compatible when saved. Note that the blob context can only be up to one major release older than the current version.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b

		M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
	ContextBlobIdPtr		Specifies the address of the variable containing the identifier of the blob context to save.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailblob11.lib.
DLL	Requires ail11.dll; ailblob11.dll.

MblobTransform

Synopsis

Perform a transformation/modification on blob analysis results.

Syntax

```
void MblobTransform(
    AIL_ID SrcResultBlobId, //in
    AIL_ID DstResultBlobId, //out
    AIL_INT64 Operation,    //in
    AIL_DOUBLE Parameter1,  //in
    AIL_DOUBLE Parameter2,  //in
    AIL_DOUBLE Parameter3,  //in
    AIL_INT64 ControlFlag   //in
)
```

Description

This function performs a transformation/modification on the results of a blob analysis operation performed using MblobCalculate().

Parameters

SrcResultBlobId

Specifies the identifier of the source result buffer.

The result buffer must be a valid identifier and the results must be calculated.

DstResultBlobId

Specifies the identifier of the destination result buffer. DstResultBlobId must be set to the same result buffer as SrcResultBlobId (that is, the transformation is done in-place).

Operation

Specifies the operation to perform.

This parameter can be set to the following value.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the operation																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_RELABEL_CONSECUTIVE	Specifies to relabel blobs to get a continuous sequence of labels for included and excluded blobs. New labels are assigned based on the order of the current labels of the blobs, and not their status. For example, if you have 4 blobs, labeled 1,4,5,8, and exclude the second blob, after M_RELABEL_CONSECUTIVE, the blobs will be labeled 1,3,4. Results must have been calculated in M_INDIVIDUAL processing mode. Original blob labeling is not necessarily continuous, and the maximum label value (M_MAX_LABEL_VALUE) can be large. M_RELABEL_CONSECUTIVE can reduce the maximum label value, which potentially shrinks the bit depth needed for a destination image buffer (for example, for an MblobLabel() operation).	c		e	f	g	a		i		j	k	l	m	n	b

Parameter1

Reserved for future expansion and must be set to **M_DEFAULT**.

Parameter2

Reserved for future expansion and must be set to **M_DEFAULT**.

Parameter3

Reserved for future expansion and must be set to **M_DEFAULT**.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailblob11.lib.
DLL	Requires ail11.dll; ailblob11.dll.